

Reviewer-Revised Research Report: Transformation Vectors and Composition Diagnostics

Date

2026-06-23

Purpose

Fix the reviewer-revised research state for external verification. This version incorporates AC-level review concerns: endpoint leakage, syntax-holdout overclaiming, McNemar tests, semantic-control statistics, and the terminology change to a third-order signed permutation coherence test.

Track 1 fixed result

Delta vectors add reproducible information beyond target-only endpoints in the multiseed full-semantic setting. The syntax=1.0 result has now been directly ablated: y_only also reaches 1.0, so the syntax split is interpreted as endpoint/surface leakage rather than as geometry. BERT-large and DeBERTa-v3-base spot-checks both rank delta best under Linear SVC and logistic regression. UPAT is now reported as a hard-holdout boundary where delta is not reliably better than y_only.

Track 2 fixed result

Composition order matters. Semantic equivalence shifts noncommutativity distributions with strong statistical tests. The third-order result is now framed as signed permutation coherence, not a formal Jacobi identity. Decoder pairwise composition is now included. The 2026-06-23 multilingual max audit extends the signed-permutation probe to 7 languages and 5 multilingual encoders. All four tested triples are below signed-null in every model-language cell; therefore the older 'QMT-only' story must be narrowed to the earlier English/decoder table, not promoted as a universal claim.

Main conclusion

The evidence now supports a broader but still controlled signed-permutation signal in multilingual sentence encoders, with NQM and QMT especially strong in the new audit. This is closer to the Lie-algebra direction, but still not a proof of a Lie algebra: endpoint controls remain strong and cross-language centroid consistency is only moderate with high variance. RISE (Freenor and Alvarez, ICLR 2026) remains the closest prior work for spherical/geodesic semantic-syntactic transformations across languages and embedding models.

Delta vs endpoint ablation with McNemar evidence

model	delta_mean	delta_std	y_only_mean	y_only_std	concat_mean	concat_std	x_only_mean	diff_delta_y	diff_delta_concat	max_mcnemar_p	max_mcnemar_sig_0_001
bert-base-uncased	0.9075	0.0117	0.8632	0.0167	0.8969	0.0185	0.1667	0.0443	0.0106	0.0000	1.0000
distilgpt2	0.8572	0.0125	0.7265	0.0094	0.7604	0.0107	0.1667	0.1307	0.0968	0.0000	1.0000
distilroberta-base	0.8810	0.0053	0.8306	0.0056	0.8417	0.0060	0.1667	0.0504	0.0394	0.0000	1.0000
gpt2	0.8332	0.0076	0.7496	0.0152	0.7765	0.0093	0.1667	0.0836	0.0567	0.0000	1.0000
roberta-base	0.8817	0.0090	0.8391	0.0072	0.8666	0.0089	0.1667	0.0426	0.0151	0.0000	1.0000

Syntax representation ablation: y_only solves the split

model	classifier	concat	delta	x_only	y_only
bert-base-uncased	linear_svc	1.0000	1.0000	0.1667	1.0000
bert-base-uncased	logreg	0.9904	0.9910	0.1667	0.9861
distilgpt2	linear_svc	1.0000	1.0000	0.1667	1.0000
distilgpt2	logreg	0.9976	1.0000	0.1667	0.9957
distilroberta-base	linear_svc	1.0000	1.0000	0.1667	1.0000
distilroberta-base	logreg	1.0000	1.0000	0.1667	1.0000
gpt2	linear_svc	1.0000	1.0000	0.1667	1.0000
gpt2	logreg	0.9979	0.9924	0.1667	0.9876
roberta-base	linear_svc	1.0000	1.0000	0.1667	1.0000
roberta-base	logreg	1.0000	1.0000	0.1667	1.0000

Layerwise/pooling syntax sanity check: top rows

classifier	accuracy	model	layer	pooling	representation	n_base	n_train	n_test
linear_svc	1.0000	bert-base-uncased	0.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	0.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	2.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	1.0000	cls	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	1.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	1.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	2.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	4.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	4.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	3.0000	cls	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	3.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	3.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	2.0000	cls	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	2.0000	cls	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	12.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	7.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	6.0000	last_token	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	6.0000	cls	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	11.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	11.0000	mean	delta	200.0000	2400.0000	4800.0000

DeBERTa-v3-small modern spot-check

model	classifier	concat	delta	x_only	y_only
microsoft/deberta-v3-small	linear_svc	0.8225	0.8708	0.1667	0.8042
microsoft/deberta-v3-small	logreg	0.8283	0.7958	0.1667	0.7700

BERT-large and DeBERTa-v3-base spot-checks

model	classifier	concat	delta	x_only	y_only
bert-large-uncased	linear_svc	0.8508	0.9025	0.1667	0.8375
bert-large-uncased	logreg	0.7600	0.8542	0.1667	0.7500
microsoft/deberta-v3-base	linear_svc	0.7758	0.8117	0.1667	0.7467
microsoft/deberta-v3-base	logreg	0.6942	0.7517	0.1667	0.7258

UPAT hard-holdout representation ablation

model	feature	acc	f1	baseline
bert-base-uncased	x_only	0.2000	0.0667	0.2000
bert-base-uncased	y_only	0.8250	0.8266	0.2000
bert-base-uncased	delta	0.7250	0.7253	0.2000
bert-base-uncased	concat_xy	0.8000	0.7974	0.2000
distilroberta-base	x_only	0.2000	0.0667	0.2000
distilroberta-base	y_only	0.6500	0.6105	0.2000
distilroberta-base	delta	0.7250	0.6530	0.2000
distilroberta-base	concat_xy	0.6250	0.5703	0.2000
roberta-base	x_only	0.2000	0.0667	0.2000
roberta-base	y_only	0.7250	0.7210	0.2000
roberta-base	delta	0.6750	0.6333	0.2000
roberta-base	concat_xy	0.7250	0.7185	0.2000
gpt2	x_only	0.2000	0.0667	0.2000
gpt2	y_only	0.4750	0.4358	0.2000
gpt2	delta	0.4250	0.3481	0.2000
gpt2	concat_xy	0.4750	0.4338	0.2000
distilgpt2	x_only	0.2000	0.0959	0.2000
distilgpt2	y_only	0.6000	0.5441	0.2000
distilgpt2	delta	0.6500	0.5953	0.2000
distilgpt2	concat_xy	0.5750	0.5169	0.2000

UPAT delta vs y_only McNemar tests

model	compare	delta_acc	y_only_acc	diff	b_delta_correct_y_wrong	c_delta_wrong_y_correct	p_value
bert-base-uncased	delta_vs_y_only	0.7250	0.8250	-0.1000	2.0000	6.0000	0.2891
distilroberta-base	delta_vs_y_only	0.7250	0.6500	0.0750	4.0000	1.0000	0.3750
roberta-base	delta_vs_y_only	0.6750	0.7250	-0.0500	7.0000	9.0000	0.8036
gpt2	delta_vs_y_only	0.4250	0.4750	-0.0500	7.0000	9.0000	0.8036
distilgpt2	delta_vs_y_only	0.6500	0.6000	0.0500	5.0000	3.0000	0.7266

UPAT-large Procrustes N=1000 null controls

train_model	test_model	null_type	n_null	observed_raw_f1	observed_aligned_f1	observed_gain_f1	null_mean_f1	null_std_f1	null_max_f1	null_mean_accomp	empirical_null_gain_obs	observed_minus_null_mean
bert-base-uncased	roberta-base	random_pairing	1000.0000	0.2462	0.6498	0.4036	0.1384	0.0659	0.4351	0.1995	0.0010	0.5115
bert-base-uncased	roberta-base	random_labels	1000.0000	0.2462	0.6498	0.4036	0.1717	0.0802	0.4751	0.2009	0.0010	0.4781
bert-base-uncased	roberta-base	random_orthogonal	1000.0000	0.2462	0.6498	0.4036	0.1172	0.0484	0.3105	0.2041	0.0010	0.5326
bert-base-uncased	distilroberta-base	random_pairing	1000.0000	0.2317	0.6133	0.3816	0.1352	0.0664	0.4506	0.2009	0.0010	0.4781
bert-base-uncased	distilroberta-base	random_labels	1000.0000	0.2317	0.6133	0.3816	0.1655	0.0800	0.5105	0.2005	0.0010	0.4478
bert-base-uncased	distilroberta-base	random_orthogonal	1000.0000	0.2317	0.6133	0.3816	0.1141	0.0474	0.2913	0.2041	0.0010	0.4991
bert-base-uncased	transformers/all-MiniLM-L6-v2	random_pairing	1000.0000	0.1657	0.7538	0.5881	0.1513	0.0718	0.4153	0.1990	0.0010	0.6025
bert-base-uncased	transformers/all-MiniLM-L6-v2	random_labels	1000.0000	0.1657	0.7538	0.5881	0.1746	0.0762	0.4602	0.1989	0.0010	0.5792
bert-base-uncased	transformers/all-MiniLM-L6-v2	random_orthogonal	1000.0000	0.1657	0.7538	0.5881	0.1288	0.0554	0.3847	0.2012	0.0010	0.6250
bert-base-uncased	transformers/all-MiniLM-L6-v2	random_pairing	1000.0000	0.1517	0.8570	0.7053	0.1569	0.0805	0.5049	0.2026	0.0010	0.7001
bert-base-uncased	transformers/all-MiniLM-L6-v2	random_labels	1000.0000	0.1517	0.8570	0.7053	0.1780	0.0831	0.5112	0.1986	0.0010	0.6790
bert-base-uncased	transformers/all-MiniLM-L6-v2	random_orthogonal	1000.0000	0.1517	0.8570	0.7053	0.1243	0.0519	0.3036	0.2006	0.0010	0.7327
bert-base-uncased	microsoft/deberta-v3-base	random_pairing	1000.0000	0.1264	0.6845	0.5581	0.1543	0.0607	0.4212	0.1992	0.0010	0.5303
bert-base-uncased	microsoft/deberta-v3-base	random_labels	1000.0000	0.1264	0.6845	0.5581	0.1746	0.0691	0.4399	0.1967	0.0010	0.5099
bert-base-uncased	microsoft/deberta-v3-base	random_orthogonal	1000.0000	0.1264	0.6845	0.5581	0.1168	0.0434	0.2979	0.1995	0.0010	0.5677
roberta-base	bert-base-uncased	random_pairing	1000.0000	0.1995	0.6790	0.4795	0.1477	0.0618	0.4227	0.1996	0.0010	0.5313
roberta-base	bert-base-uncased	random_labels	1000.0000	0.1995	0.6790	0.4795	0.1792	0.0714	0.5296	0.1965	0.0010	0.4998
roberta-base	bert-base-uncased	random_orthogonal	1000.0000	0.1995	0.6790	0.4795	0.0969	0.0293	0.2374	0.2007	0.0010	0.5821
roberta-base	distilroberta-base	random_pairing	1000.0000	0.4374	0.6891	0.2517	0.1324	0.0646	0.3632	0.2006	0.0010	0.5566
roberta-base	distilroberta-base	random_labels	1000.0000	0.4374	0.6891	0.2517	0.1793	0.0879	0.5944	0.1954	0.0010	0.5097
roberta-base	distilroberta-base	random_orthogonal	1000.0000	0.4374	0.6891	0.2517	0.0848	0.0296	0.2680	0.1956	0.0010	0.6042
roberta-base	transformers/all-MiniLM-L6-v2	random_pairing	1000.0000	0.1512	0.7685	0.6174	0.1387	0.0698	0.4063	0.1987	0.0010	0.6298
roberta-base	transformers/all-MiniLM-L6-v2	random_labels	1000.0000	0.1512	0.7685	0.6174	0.1710	0.0835	0.5507	0.1944	0.0010	0.5975
roberta-base	transformers/all-MiniLM-L6-v2	random_orthogonal	1000.0000	0.1512	0.7685	0.6174	0.1026	0.0453	0.2628	0.1994	0.0010	0.6659
roberta-base	transformers/all-MiniLM-L6-v2	random_pairing	1000.0000	0.0931	0.7804	0.6873	0.1438	0.0773	0.4447	0.1968	0.0010	0.6367
roberta-base	transformers/all-MiniLM-L6-v2	random_labels	1000.0000	0.0931	0.7804	0.6873	0.1726	0.0870	0.5321	0.1938	0.0010	0.6078
roberta-base	transformers/all-MiniLM-L6-v2	random_orthogonal	1000.0000	0.0931	0.7804	0.6873	0.0952	0.0429	0.3000	0.1994	0.0010	0.6852
roberta-base	microsoft/deberta-v3-base	random_pairing	1000.0000	0.4772	0.6277	0.1505	0.1535	0.0609	0.4175	0.2009	0.0010	0.4742
roberta-base	microsoft/deberta-v3-base	random_labels	1000.0000	0.4772	0.6277	0.1505	0.1836	0.0747	0.5324	0.1977	0.0010	0.4440
roberta-base	microsoft/deberta-v3-base	random_orthogonal	1000.0000	0.4772	0.6277	0.1505	0.0933	0.0305	0.2532	0.2005	0.0010	0.5343

UPAT-large held-out alignment-size curve

alignment_size	mean_f1	std_f1	mean_acc	mean_full_anchor_f1	mean_raw_f1	mean_gap_to_full_anchor	mean_gain_over_raw	n
25.0000	0.4520	0.1040	0.4986	0.6847	0.2415	-0.2326	0.2105	300.0000
50.0000	0.5732	0.0943	0.6072	0.6847	0.2415	-0.1114	0.3317	300.0000
100.0000	0.6294	0.0744	0.6602	0.6847	0.2415	-0.0552	0.3879	300.0000
250.0000	0.6537	0.0733	0.6841	0.6847	0.2415	-0.0310	0.4122	300.0000
500.0000	0.6591	0.0720	0.6908	0.6847	0.2415	-0.0256	0.4176	300.0000
1000.0000	0.6619	0.0753	0.6934	0.6847	0.2415	-0.0227	0.4204	300.0000

UPAT-large held-out alignment by direction: largest anchor size

	train_model	test_model	alignment_size	mean_f1	std_f1	mean_acc	observed_full_anchor_f	observed_raw_f1	mean_gap_to_full_anchor	mean_gain_over_raw	n
m	icrosoft/deberta-base	transformers/all-mpne	1000.0000	0.7444	0.0189	0.7522	0.8316	0.2290	-0.0872	0.5154	10.0000
m	icrosoft/deberta-v3-base	bert-base-uncased	1000.0000	0.6922	0.0037	0.7334	0.7614	0.1028	-0.0692	0.5894	10.0000
	distilroberta-base	bert-base-uncased	1000.0000	0.5468	0.0096	0.6262	0.6137	0.3369	-0.0669	0.2100	10.0000
m	icrosoft/deberta-v3-base	roberta-base	1000.0000	0.6143	0.0083	0.6674	0.6762	0.4724	-0.0619	0.1419	10.0000
ce	transformers/all-Mini	icrosoft/deberta-v3-base	1000.0000	0.6589	0.0055	0.6594	0.7127	0.1444	-0.0538	0.5145	10.0000
ce	transformers/all-mpne	bert-base-uncased	1000.0000	0.5566	0.0112	0.5744	0.6090	0.1959	-0.0524	0.3606	10.0000
	distilroberta-base	transformers/all-Mini	1000.0000	0.6919	0.0091	0.7456	0.7384	0.0995	-0.0465	0.5924	10.0000
m	icrosoft/deberta-v3-base	distilroberta-base	1000.0000	0.5391	0.0037	0.5890	0.5849	0.3539	-0.0458	0.1852	10.0000
	roberta-base	transformers/all-Mini	1000.0000	0.7312	0.0088	0.7508	0.7685	0.1512	-0.0373	0.5800	10.0000
	distilroberta-base	transformers/all-mpne	1000.0000	0.7212	0.0109	0.7370	0.7550	0.1983	-0.0338	0.5229	10.0000
ce	transformers/all-Mini	transformers/all-mpne	1000.0000	0.7279	0.0043	0.7474	0.7583	0.2265	-0.0304	0.5014	10.0000
	roberta-base	bert-base-uncased	1000.0000	0.6508	0.0100	0.6760	0.6790	0.1995	-0.0282	0.4513	10.0000
ce	transformers/all-Mini	distilroberta-base	1000.0000	0.5601	0.0108	0.6252	0.5877	0.1320	-0.0276	0.4281	10.0000
ce	transformers/all-mpne	icrosoft/deberta-v3-base	1000.0000	0.6619	0.0110	0.6648	0.6894	0.1960	-0.0275	0.4659	10.0000
m	icrosoft/deberta-v3-base	transformers/all-Mini	1000.0000	0.7465	0.0088	0.7558	0.7730	0.2228	-0.0266	0.5237	10.0000
	bert-base-uncased	icrosoft/deberta-v3-base	1000.0000	0.6610	0.0064	0.6780	0.6845	0.1264	-0.0236	0.5345	10.0000
	roberta-base	transformers/all-mpne	1000.0000	0.7575	0.0084	0.7592	0.7804	0.0931	-0.0230	0.6643	10.0000
	roberta-base	distilroberta-base	1000.0000	0.6702	0.0050	0.7172	0.6891	0.4374	-0.0189	0.2328	10.0000
	roberta-base	icrosoft/deberta-v3-base	1000.0000	0.6110	0.0072	0.6128	0.6277	0.4772	-0.0167	0.1338	10.0000
	distilroberta-base	roberta-base	1000.0000	0.6178	0.0084	0.6872	0.6339	0.4776	-0.0161	0.1402	10.0000
	bert-base-uncased	distilroberta-base	1000.0000	0.5979	0.0088	0.6578	0.6133	0.2317	-0.0154	0.3662	10.0000
	bert-base-uncased	transformers/all-mpne	1000.0000	0.8421	0.0114	0.8430	0.8570	0.1517	-0.0149	0.6904	10.0000
ce	transformers/all-Mini	bert-base-uncased	1000.0000	0.6688	0.0092	0.6758	0.6807	0.3491	-0.0119	0.3197	10.0000
	distilroberta-base	icrosoft/deberta-v3-base	1000.0000	0.6004	0.0091	0.6152	0.6094	0.1740	-0.0090	0.4264	10.0000
ce	transformers/all-Mini	roberta-base	1000.0000	0.6285	0.0076	0.6758	0.6086	0.1254	0.0199	0.5031	10.0000
	bert-base-uncased	roberta-base	1000.0000	0.6699	0.0072	0.7318	0.6498	0.2462	0.0201	0.4237	10.0000
ce	transformers/all-mpne	distilroberta-base	1000.0000	0.5536	0.0072	0.6002	0.5322	0.2739	0.0214	0.2797	10.0000
ce	transformers/all-mpne	transformers/all-Mini	1000.0000	0.6953	0.0068	0.7314	0.6681	0.3469	0.0272	0.3484	10.0000
ce	transformers/all-mpne	roberta-base	1000.0000	0.6437	0.0083	0.7064	0.6122	0.3084	0.0315	0.3353	10.0000
	bert-base-uncased	transformers/all-Mini	1000.0000	0.7965	0.0087	0.8048	0.7538	0.1657	0.0427	0.6308	10.0000

UPAT RISE-aware prototype comparison

comparison	method	mean_target_cosine	std_target_cosine	mean_retrieval_top1_acc	mean_retrieval_label_acc	mean_retrieval_label_f1	mean_delta_classifier_f1	n
cross_model_full_anchor	mdv_raw	0.5769	0.1822	0.3157	0.4667	0.4114	0.6847	30.0000
cross_model_full_anchor	mdv_unit	0.6136	0.1583	0.3485	0.4632	0.4077	0.6847	30.0000
cross_model_full_anchor	rise_style	0.5783	0.1614	0.3778	0.5037	0.4455	0.6847	30.0000
within_model	mdv_raw	0.9224	0.0733	0.3603	0.4953	0.4567	0.7156	6.0000
within_model	mdv_unit	0.9218	0.0741	0.3677	0.5027	0.4635	0.7156	6.0000
within_model	rise_style	0.9230	0.0722	0.3740	0.5173	0.4773	0.7156	6.0000

UPAT hybrid RISE-Procrustes transfer

comparison	prototype_method	feature	mean_acc	mean_f1	std_f1	mean_gain_vs_observed_aligned	n
cross_model_full_anchor	mdv_raw	delta_only	0.7176	0.6847	0.0794	0.0000	30.0000
cross_model_full_anchor	mdv_unit	delta_only	0.7176	0.6847	0.0794	0.0000	30.0000
cross_model_full_anchor	rise_style	delta_only	0.7176	0.6847	0.0794	0.0000	30.0000
cross_model_full_anchor	mdv_raw	mdv_raw_hybrid_delta_scores	0.5037	0.4308	0.1396	-0.2538	30.0000
cross_model_full_anchor	mdv_raw	mdv_raw_prototype_scores	0.4854	0.4136	0.1514	-0.2711	30.0000
cross_model_full_anchor	rise_style	rise_style_prototype_scores	0.4553	0.3756	0.2040	-0.3091	30.0000
cross_model_full_anchor	mdv_unit	mdv_unit_prototype_scores	0.4458	0.3695	0.1965	-0.3152	30.0000
cross_model_full_anchor	rise_style	rise_style_hybrid_delta_scores	0.3801	0.2844	0.1742	-0.4003	30.0000
cross_model_full_anchor	mdv_unit	mdv_unit_hybrid_delta_scores	0.3711	0.2765	0.1669	-0.4082	30.0000
within_model	mdv_raw	delta_only	0.7393	0.7156	0.0338		6.0000
within_model	mdv_unit	delta_only	0.7393	0.7156	0.0338		6.0000
within_model	rise_style	delta_only	0.7393	0.7156	0.0338		6.0000
within_model	mdv_raw	mdv_raw_prototype_scores	0.6700	0.6333	0.0781		6.0000
within_model	mdv_raw	mdv_raw_hybrid_delta_scores	0.6687	0.6208	0.0765		6.0000
within_model	rise_style	rise_style_prototype_scores	0.6217	0.5887	0.0765		6.0000
within_model	mdv_unit	mdv_unit_prototype_scores	0.6220	0.5885	0.0825		6.0000
within_model	rise_style	rise_style_hybrid_delta_scores	0.6053	0.5560	0.0822		6.0000
within_model	mdv_unit	mdv_unit_hybrid_delta_scores	0.6033	0.5546	0.0861		6.0000

UPAT spherical delta steering

comparison	method	mean_target_cosine	std_target_cosine	mean_retrieval_top1_acc	mean_retrieval_label_acc	mean_retrieval_label_f1	n
cross_model_full_anchor	linear_delta	0.6136	0.1583	0.3485	0.4632	0.4077	30.0000
cross_model_full_anchor	linear_then_residual	0.6129	0.1578	0.3486	0.4639	0.4075	30.0000
cross_model_full_anchor	spherical_delta	0.6048	0.1538	0.3581	0.4697	0.4127	30.0000
cross_model_full_anchor	rise_then_ours_residual	0.5998	0.1550	0.3669	0.4861	0.4285	30.0000
cross_model_full_anchor	spherical_then_residual	0.5974	0.1465	0.3709	0.4905	0.4326	30.0000
cross_model_full_anchor	hybrid_average	0.5969	0.1587	0.3717	0.4913	0.4339	30.0000
cross_model_full_anchor	rise_only	0.5783	0.1614	0.3778	0.5037	0.4455	30.0000
within_model	rise_then_ours_residual	0.9234	0.0719	0.3740	0.5160	0.4756	6.0000
within_model	hybrid_average	0.9231	0.0724	0.3717	0.5107	0.4707	6.0000
within_model	rise_only	0.9230	0.0722	0.3740	0.5173	0.4773	6.0000
within_model	linear_then_residual	0.9218	0.0741	0.3700	0.5060	0.4663	6.0000
within_model	linear_delta	0.9218	0.0741	0.3677	0.5027	0.4635	6.0000
within_model	spherical_delta	0.9217	0.0742	0.3677	0.5023	0.4629	6.0000
within_model	spherical_then_residual	0.9215	0.0744	0.3717	0.5077	0.4679	6.0000

Full-semantic pooling ablation

model	pooling	classifier	concat	delta	x_only	y_only
bert-base-uncased	cls	linear_svc	0.8269	0.8775	0.1667	0.7397
bert-base-uncased	cls	logreg	0.7031	0.7200	0.1667	0.6781
bert-base-uncased	last_token	linear_svc	0.7564	0.8514	0.1667	0.7508
bert-base-uncased	last_token	logreg	0.6792	0.7103	0.1667	0.6847
bert-base-uncased	mean	linear_svc	0.8850	0.8958	0.1667	0.8806
bert-base-uncased	mean	logreg	0.8725	0.8844	0.1667	0.8603
gpt2	cls	linear_svc	0.3683	0.3867	0.1667	0.3683
gpt2	cls	logreg	0.3422	0.3203	0.1667	0.3683
gpt2	last_token	linear_svc	0.7469	0.7564	0.1667	0.7200
gpt2	last_token	logreg	0.6453	0.6683	0.1667	0.6253
gpt2	mean	linear_svc	0.7822	0.8450	0.1667	0.7539
gpt2	mean	logreg	0.7803	0.7492	0.1667	0.7633
roberta-base	cls	linear_svc	0.8236	0.8736	0.1667	0.7911
roberta-base	cls	logreg	0.7944	0.8514	0.1667	0.7669
roberta-base	last_token	linear_svc	0.7656	0.7650	0.1667	0.7042
roberta-base	last_token	logreg	0.7056	0.6756	0.1667	0.6739
roberta-base	mean	linear_svc	0.8631	0.8786	0.1667	0.8431
roberta-base	mean	logreg	0.8150	0.8092	0.1667	0.8017

Confusion analysis: negation vs non-negation recall

source	model	classifier	group	mean_recall	mean_error_rate	n_classes
full_semantic	bert-base-uncased	linear_svc	negation	1.0000	0.0000	1.0000
full_semantic	bert-base-uncased	linear_svc	non_negation	0.8725	0.1275	5.0000
full_semantic	bert-base-uncased	logreg	negation	0.9862	0.0138	1.0000
full_semantic	bert-base-uncased	logreg	non_negation	0.8817	0.1182	5.0000
full_semantic	distilgpt2	linear_svc	negation	1.0000	0.0000	1.0000
full_semantic	distilgpt2	linear_svc	non_negation	0.8020	0.1980	5.0000
full_semantic	distilgpt2	logreg	negation	0.9725	0.0275	1.0000
full_semantic	distilgpt2	logreg	non_negation	0.7295	0.2705	5.0000
full_semantic	distilroberta-base	linear_svc	negation	1.0000	0.0000	1.0000
full_semantic	distilroberta-base	linear_svc	non_negation	0.8470	0.1530	5.0000
full_semantic	distilroberta-base	logreg	negation	1.0000	0.0000	1.0000
full_semantic	distilroberta-base	logreg	non_negation	0.8252	0.1747	5.0000
full_semantic	gpt2	linear_svc	negation	0.9600	0.0400	1.0000
full_semantic	gpt2	linear_svc	non_negation	0.7973	0.2027	5.0000
full_semantic	gpt2	logreg	negation	0.7312	0.2687	1.0000
full_semantic	gpt2	logreg	non_negation	0.7410	0.2590	5.0000
full_semantic	roberta-base	linear_svc	negation	0.9988	0.0012	1.0000
full_semantic	roberta-base	linear_svc	non_negation	0.8570	0.1430	5.0000
full_semantic	roberta-base	logreg	negation	0.9300	0.0700	1.0000
full_semantic	roberta-base	logreg	non_negation	0.7760	0.2240	5.0000

Third-order signed permutation summary with bootstrap CI

model	triple	mean_relative_jacobi_norm	relative_jacobi_norm_ci_95_low	relative_jacobi_norm_ci_95_high	mean_jacobi_to_null_mean_ratio	jacobi_to_null_mean_ratio_ci_95_low	jacobi_to_null_mean_ratio_ci_95_high	mean_percentile_smaller_is	n
bert-base-uncased	NMT	0.3048	0.3005	0.3090	0.7750	0.7669	0.7829	0.3522	100.0000
bert-base-uncased	NQM	0.2695	0.2660	0.2727	0.8628	0.8533	0.8720	0.3850	100.0000
bert-base-uncased	NQT	0.3719	0.3674	0.3763	1.1170	1.1082	1.1259	0.7005	100.0000
bert-base-uncased	QMT	0.2760	0.2712	0.2807	0.6833	0.6773	0.6892	0.1018	100.0000
distilroberta-base	NMT	0.3522	0.3485	0.3558	1.1338	1.1276	1.1404	0.7066	100.0000
distilroberta-base	NQM	0.2847	0.2809	0.2882	1.0821	1.0735	1.0914	0.7777	100.0000
distilroberta-base	NQT	0.2777	0.2752	0.2802	1.0803	1.0765	1.0841	0.6274	100.0000
distilroberta-base	QMT	0.1849	0.1832	0.1866	0.6311	0.6267	0.6356	0.0997	100.0000
roberta-base	NMT	0.4315	0.4255	0.4371	1.2532	1.2423	1.2637	0.8057	100.0000
roberta-base	NQM	0.3122	0.3086	0.3158	0.9799	0.9743	0.9853	0.6475	100.0000
roberta-base	NQT	0.3452	0.3413	0.3490	1.0805	1.0739	1.0871	0.7059	100.0000
roberta-base	QMT	0.2190	0.2160	0.2222	0.6229	0.6168	0.6287	0.1013	100.0000

Decoder signed permutation replication

model	triple	mean_jacobi_norm	mean_relative_jacobi_norm	null_relative_jacobi_norm	jacobi_to_null_mean_ratio	mean_percentile_smaller	n	relative_jacobi_norm	relative_jacobi_norm	jacobi_to_null_mean_ratio	jacobi_to_null_mean_ratio_c95
distilgpt2	NMT	10.3687	0.2276	0.2387	0.9507	0.5781	100.0000	0.2171	0.2388	0.9284	0.9722
distilgpt2	NQM	9.5684	0.2144	0.2540	0.8464	0.4061	100.0000	0.2056	0.2239	0.8249	0.8692
distilgpt2	NQT	15.8990	0.3385	0.2403	1.4202	0.8621	100.0000	0.3241	0.3537	1.3820	1.4555
distilgpt2	QMT	9.8263	0.2492	0.3179	0.7708	0.3207	100.0000	0.2320	0.2668	0.7446	0.7972
gpt2	NMT	31.1167	0.2894	0.2137	1.3348	0.7670	100.0000	0.2736	0.3065	1.2981	1.3723
gpt2	NQM	24.9467	0.2799	0.2315	1.1947	0.7302	100.0000	0.2642	0.2955	1.1675	1.2221
gpt2	NQT	16.4593	0.1562	0.1777	0.8666	0.4331	100.0000	0.1466	0.1658	0.8356	0.9004
gpt2	QMT	14.3721	0.1705	0.3167	0.5393	0.1852	100.0000	0.1621	0.1797	0.5192	0.5611

Signed permutation multiple-testing correction

model	triple	n	mean_ratio_to_nullboot	bootstrap_p_ratio_less_than	direction	bonferroni_p	bh_fdr_p	passes_bonferroni_0_05	passes_bh_fdr_0_05
bert-base-uncased	NMT	100.0000	0.7750	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
bert-base-uncased	NQM	100.0000	0.8628	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
bert-base-uncased	QMT	100.0000	0.6833	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
distilgpt2	NQM	100.0000	0.8464	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
distilgpt2	QMT	100.0000	0.7708	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
distilroberta-base	QMT	100.0000	0.6311	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
gpt2	NQT	100.0000	0.8666	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
gpt2	QMT	100.0000	0.5393	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
roberta-base	NQM	100.0000	0.9799	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
roberta-base	QMT	100.0000	0.6229	0.0000	below_null	0.0010	0.0001	1.0000	1.0000
distilgpt2	NMT	100.0000	0.9507	0.0001	below_null	0.0020	0.0002	1.0000	1.0000
bert-base-uncased	NQT	100.0000	1.1170	1.0000	above_null	1.0000	1.0000	0.0000	0.0000
distilgpt2	NQT	100.0000	1.4202	1.0000	above_null	1.0000	1.0000	0.0000	0.0000
distilroberta-base	NMT	100.0000	1.1338	1.0000	above_null	1.0000	1.0000	0.0000	0.0000
distilroberta-base	NQM	100.0000	1.0821	1.0000	above_null	1.0000	1.0000	0.0000	0.0000
distilroberta-base	NQT	100.0000	1.0803	1.0000	above_null	1.0000	1.0000	0.0000	0.0000
gpt2	NMT	100.0000	1.3348	1.0000	above_null	1.0000	1.0000	0.0000	0.0000
gpt2	NQM	100.0000	1.1947	1.0000	above_null	1.0000	1.0000	0.0000	0.0000
roberta-base	NMT	100.0000	1.2532	1.0000	above_null	1.0000	1.0000	0.0000	0.0000
roberta-base	NQT	100.0000	1.0805	1.0000	above_null	1.0000	1.0000	0.0000	0.0000

Decoder pairwise composition summary

model	pair	mean_commutator_norm	std_commutator_norm	mean_relative_commutator_norm	std_relative_commutator_norm	mean_cosine	std_cosine	mean_noncommutativity	std_noncommutativity	n
distilgpt2	MT_vs_TM	10.5809	1.8042	0.8545	0.2156	0.7111	0.1725	0.2889	0.1725	80.0000
distilgpt2	NM_vs_MN	3.7641	0.9414	0.3098	0.0893	0.9583	0.0283	0.0417	0.0283	80.0000
distilgpt2	NQ_vs_QN	5.7897	0.6043	0.4841	0.1058	0.8844	0.0472	0.1156	0.0472	80.0000
distilgpt2	NT_vs_TN	11.1703	1.8386	0.8664	0.1749	0.7046	0.1705	0.2954	0.1705	80.0000
distilgpt2	QM_vs_MQ	7.7162	0.9341	0.6474	0.1543	0.8039	0.0973	0.1961	0.0973	80.0000
distilgpt2	QT_vs_TQ	2.7566	0.6003	0.3249	0.1427	0.9571	0.0470	0.0429	0.0470	80.0000
gpt2	MT_vs_TM	11.3260	1.9708	0.3072	0.0694	0.9707	0.0080	0.0293	0.0080	80.0000
gpt2	NM_vs_MN	5.2897	1.1579	0.1894	0.0387	0.9901	0.0035	0.0099	0.0035	80.0000
gpt2	NQ_vs_QN	11.1700	1.5640	0.3855	0.0863	0.9337	0.0283	0.0663	0.0283	80.0000
gpt2	NT_vs_TN	23.9958	3.0734	0.7100	0.1186	0.9667	0.0305	0.0333	0.0305	80.0000
gpt2	QM_vs_MQ	13.3556	2.9359	0.5125	0.1313	0.8888	0.0563	0.1112	0.0563	80.0000
gpt2	QT_vs_TQ	3.9191	0.8893	0.2190	0.0846	0.9761	0.0196	0.0239	0.0196	80.0000

Semantic equivalence control

model	label	mean_noncommutativity	std_noncommutativity	mean_relative_commutator_std	std_relative_commutator_norm	mean_cosine	std_cosine	n
bert-base-uncased	equivalent	0.1130	0.0409	0.4818	0.0809	0.8870	0.0409	400.0000
bert-base-uncased	non_equivalent	0.3291	0.1134	0.8065	0.1326	0.6709	0.1134	400.0000
distilroberta-base	equivalent	0.1295	0.0483	0.5406	0.1277	0.8705	0.0483	400.0000
distilroberta-base	non_equivalent	0.2319	0.0381	0.6868	0.0560	0.7681	0.0381	400.0000
roberta-base	equivalent	0.1543	0.0630	0.5831	0.1304	0.8457	0.0630	400.0000
roberta-base	non_equivalent	0.2784	0.0439	0.7465	0.0597	0.7216	0.0439	400.0000

Semantic equivalence effect sizes

model	equiv_mean	non_equiv_mean	diff	cohens_d	equiv_std	non_equiv_std
bert-base-uncased	0.1130	0.3291	0.2161	2.5344	0.0409	0.1134
distilroberta-base	0.1295	0.2319	0.1024	2.3536	0.0483	0.0381
roberta-base	0.1543	0.2784	0.1241	2.2854	0.0630	0.0439

Composition summary: strongest noncommutativity rows

model	pair	mean_commutator_norm	std_commutator_norm	mean_relative_commutator_norm	std_relative_commutator_norm	mean_cosine	std_cosine	mean_noncommutativity	std_noncommutativity	n
bert-base-uncased	NT_vs_TN	5.1565	0.5832	0.9908	0.1066	0.5102	0.1009	0.4898	0.1009	80.0000
roberta-base	NQ_vs_QN	1.6422	0.0726	0.7846	0.0364	0.6928	0.0279	0.3072	0.0279	80.0000
bert-base-uncased	MT_vs_TM	4.2726	0.3075	0.7817	0.0954	0.6938	0.0727	0.3062	0.0727	80.0000
roberta-base	QM_vs_MQ	1.5649	0.0863	0.7714	0.0541	0.7032	0.0410	0.2968	0.0410	80.0000
distilroberta-base	MT_vs_TM	1.3504	0.0606	0.7683	0.0387	0.7124	0.0302	0.2876	0.0302	80.0000
bert-base-uncased	QM_vs_MQ	4.4793	0.2156	0.7341	0.0487	0.7340	0.0347	0.2660	0.0347	80.0000
roberta-base	MT_vs_TM	1.3956	0.0770	0.7215	0.0560	0.7411	0.0394	0.2589	0.0394	80.0000
roberta-base	NT_vs_TN	1.4185	0.1071	0.7172	0.0599	0.7429	0.0428	0.2571	0.0428	80.0000
bert-base-uncased	NQ_vs_QN	4.0903	0.2939	0.6970	0.0493	0.7598	0.0347	0.2402	0.0347	80.0000
distilroberta-base	NQ_vs_QN	1.4541	0.0644	0.6731	0.0384	0.7735	0.0257	0.2265	0.0257	80.0000
distilroberta-base	NT_vs_TN	1.3975	0.0725	0.6675	0.0342	0.7783	0.0232	0.2217	0.0232	80.0000
distilroberta-base	QM_vs_MQ	1.3672	0.0842	0.6613	0.0460	0.7926	0.0287	0.2074	0.0287	80.0000
bert-base-uncased	QT_vs_TQ	3.1001	0.1729	0.5463	0.0440	0.8513	0.0231	0.1487	0.0231	80.0000
roberta-base	QT_vs_TQ	0.7784	0.0701	0.3860	0.0342	0.9269	0.0128	0.0731	0.0128	80.0000
distilroberta-base	QT_vs_TQ	0.6580	0.0281	0.3422	0.0173	0.9415	0.0059	0.0585	0.0059	80.0000
bert-base-uncased	NM_vs_MN	1.2527	0.2368	0.2305	0.0538	0.9728	0.0125	0.0272	0.0125	80.0000
roberta-base	NM_vs_MN	0.3865	0.0403	0.2032	0.0248	0.9794	0.0050	0.0206	0.0050	80.0000
distilroberta-base	NM_vs_MN	0.3128	0.0212	0.1721	0.0201	0.9852	0.0036	0.0148	0.0036	80.0000

Grammar-generated pairwise composition summary

model	pair	mean_relative_commutator_norm	std_relative_commutator_norm	mean_noncommutativity	std_noncommutativity	mean_cosine	n
bert-base-uncased	MT_vs_TM	0.5195	0.2368	0.1566	0.1215	0.8434	120.0000
bert-base-uncased	NM_vs_MN	0.3424	0.1513	0.0682	0.0582	0.9318	120.0000
bert-base-uncased	NQ_vs_QN	0.6145	0.0455	0.1861	0.0283	0.8139	120.0000
bert-base-uncased	NT_vs_TN	0.5681	0.1053	0.1580	0.0580	0.8420	120.0000
bert-base-uncased	QM_vs_MQ	0.5511	0.0451	0.1418	0.0228	0.8582	120.0000
bert-base-uncased	QT_vs_TQ	0.6036	0.0624	0.1630	0.0340	0.8370	120.0000
distilroberta-base	MT_vs_TM	0.4693	0.2291	0.1146	0.0762	0.8854	120.0000
distilroberta-base	NM_vs_MN	0.4265	0.1677	0.1017	0.0615	0.8983	120.0000
distilroberta-base	NQ_vs_QN	0.6999	0.0344	0.2450	0.0241	0.7550	120.0000
distilroberta-base	NT_vs_TN	0.5709	0.0326	0.1422	0.0177	0.8578	120.0000
distilroberta-base	QM_vs_MQ	0.6748	0.0396	0.2019	0.0256	0.7981	120.0000
distilroberta-base	QT_vs_TQ	0.8299	0.0516	0.2481	0.0334	0.7519	120.0000
roberta-base	MT_vs_TM	0.4418	0.2110	0.1023	0.0728	0.8977	120.0000
roberta-base	NM_vs_MN	0.4525	0.1685	0.1131	0.0709	0.8869	120.0000
roberta-base	NQ_vs_QN	0.7745	0.0323	0.2994	0.0249	0.7006	120.0000
roberta-base	NT_vs_TN	0.5540	0.0788	0.1418	0.0417	0.8582	120.0000
roberta-base	QM_vs_MQ	0.7713	0.0446	0.2711	0.0309	0.7289	120.0000
roberta-base	QT_vs_TQ	0.7780	0.0518	0.2380	0.0338	0.7620	120.0000

Grammar composition endpoint controls

model	feature	accuracy	macro_f1	n_train	n_test	chance
bert-base-uncased	source_only	0.1667	0.0476	360.0000	360.0000	0.1667
bert-base-uncased	ab_endpoint_only	1.0000	1.0000	360.0000	360.0000	0.1667
bert-base-uncased	ba_endpoint_only	0.9972	0.9972	360.0000	360.0000	0.1667
bert-base-uncased	ab_delta_only	1.0000	1.0000	360.0000	360.0000	0.1667
bert-base-uncased	ba_delta_only	1.0000	1.0000	360.0000	360.0000	0.1667
bert-base-uncased	commutator_delta	1.0000	1.0000	360.0000	360.0000	0.1667
bert-base-uncased	endpoint_concat	1.0000	1.0000	360.0000	360.0000	0.1667
distilroberta-base	source_only	0.1667	0.0476	360.0000	360.0000	0.1667
distilroberta-base	ab_endpoint_only	1.0000	1.0000	360.0000	360.0000	0.1667
distilroberta-base	ba_endpoint_only	1.0000	1.0000	360.0000	360.0000	0.1667
distilroberta-base	ab_delta_only	1.0000	1.0000	360.0000	360.0000	0.1667
distilroberta-base	ba_delta_only	1.0000	1.0000	360.0000	360.0000	0.1667
distilroberta-base	commutator_delta	1.0000	1.0000	360.0000	360.0000	0.1667
distilroberta-base	endpoint_concat	1.0000	1.0000	360.0000	360.0000	0.1667
roberta-base	source_only	0.1667	0.0476	360.0000	360.0000	0.1667
roberta-base	ab_endpoint_only	1.0000	1.0000	360.0000	360.0000	0.1667
roberta-base	ba_endpoint_only	1.0000	1.0000	360.0000	360.0000	0.1667
roberta-base	ab_delta_only	1.0000	1.0000	360.0000	360.0000	0.1667
roberta-base	ba_delta_only	1.0000	1.0000	360.0000	360.0000	0.1667
roberta-base	commutator_delta	1.0000	1.0000	360.0000	360.0000	0.1667
roberta-base	endpoint_concat	1.0000	1.0000	360.0000	360.0000	0.1667

Grammar composition commutator nulls: same-pair shuffle

model	pair	null_type	observed_mean_relative_comm	null_mean	null_std	null_p05	null_p95	observed_percentile_smaller_is_p	smaller_or_equal_n	larger_or_equal_n	n_null
bert-base-uncased	MT_vs_TM	random_pairing_same_	0.5195	0.9584	0.0123	0.9392	0.9779	0.0000	0.0010	1.0000	1000.0000
bert-base-uncased	NM_vs_MN	random_pairing_same_	0.3424	0.8659	0.0115	0.8454	0.8845	0.0000	0.0010	1.0000	1000.0000
bert-base-uncased	NQ_vs_QN	random_pairing_same_	0.6145	0.8750	0.0068	0.8633	0.8859	0.0000	0.0010	1.0000	1000.0000
bert-base-uncased	NT_vs_TN	random_pairing_same_	0.5681	0.9147	0.0087	0.9003	0.9285	0.0000	0.0010	1.0000	1000.0000
bert-base-uncased	QM_vs_MQ	random_pairing_same_	0.5511	0.8511	0.0071	0.8395	0.8631	0.0000	0.0010	1.0000	1000.0000
bert-base-uncased	QT_vs_TQ	random_pairing_same_	0.6036	0.8616	0.0063	0.8515	0.8713	0.0000	0.0010	1.0000	1000.0000
distilroberta-base	MT_vs_TM	random_pairing_same_	0.4693	0.7682	0.0085	0.7529	0.7816	0.0000	0.0010	1.0000	1000.0000
distilroberta-base	NM_vs_MN	random_pairing_same_	0.4265	0.7067	0.0070	0.6949	0.7181	0.0000	0.0010	1.0000	1000.0000
distilroberta-base	NQ_vs_QN	random_pairing_same_	0.6999	0.7803	0.0022	0.7767	0.7837	0.0000	0.0010	1.0000	1000.0000
distilroberta-base	NT_vs_TN	random_pairing_same_	0.5709	0.6946	0.0034	0.6888	0.7002	0.0000	0.0010	1.0000	1000.0000
distilroberta-base	QM_vs_MQ	random_pairing_same_	0.6748	0.8040	0.0035	0.7983	0.8097	0.0000	0.0010	1.0000	1000.0000
distilroberta-base	QT_vs_TQ	random_pairing_same_	0.8299	0.8844	0.0016	0.8817	0.8870	0.0000	0.0010	1.0000	1000.0000
roberta-base	MT_vs_TM	random_pairing_same_	0.4418	0.8063	0.0090	0.7914	0.8201	0.0000	0.0010	1.0000	1000.0000
roberta-base	NM_vs_MN	random_pairing_same_	0.4525	0.7931	0.0074	0.7809	0.8051	0.0000	0.0010	1.0000	1000.0000
roberta-base	NQ_vs_QN	random_pairing_same_	0.7745	0.8844	0.0029	0.8792	0.8890	0.0000	0.0010	1.0000	1000.0000
roberta-base	NT_vs_TN	random_pairing_same_	0.5540	0.7664	0.0057	0.7573	0.7751	0.0000	0.0010	1.0000	1000.0000
roberta-base	QM_vs_MQ	random_pairing_same_	0.7713	0.9301	0.0042	0.9229	0.9369	0.0000	0.0010	1.0000	1000.0000
roberta-base	QT_vs_TQ	random_pairing_same_	0.7780	0.8776	0.0030	0.8726	0.8824	0.0000	0.0010	1.0000	1000.0000

Multilingual max audit: global triple summary

triple	mean_ratio_to_null_mean	std_ratio_to_null_mean	min_ratio_to_null_mean	max_ratio_to_null_mean	below_null_cells	n_cells	frac_below_null_cells
NMT	0.7716	0.0486	0.6616	0.8329	35.0000	35.0000	1.0000
NQM	0.5798	0.0722	0.4182	0.7179	35.0000	35.0000	1.0000
NQT	0.7014	0.0727	0.4999	0.8366	35.0000	35.0000	1.0000
QMT	0.6203	0.0850	0.4832	0.7559	35.0000	35.0000	1.0000

Multilingual max audit: triple summary by model

model	triple	mean_ratio_to_null_mean	std_ratio_to_null_mean	min_ratio_to_null_mean	max_ratio_to_null_mean	below_null_languages	n_languages
bge-m3	NMT	0.7901	0.0433	0.7029	0.8329	7.0000	7.0000
bge-m3	NQM	0.6020	0.0534	0.5003	0.6542	7.0000	7.0000
bge-m3	NQT	0.7266	0.0864	0.5633	0.8366	7.0000	7.0000
bge-m3	QMT	0.6547	0.0754	0.5258	0.7267	7.0000	7.0000
mBERT	NMT	0.7616	0.0480	0.7124	0.8329	7.0000	7.0000
mBERT	NQM	0.5364	0.0520	0.4750	0.6083	7.0000	7.0000
mBERT	NQT	0.6799	0.0235	0.6583	0.7189	7.0000	7.0000
mBERT	QMT	0.6421	0.0703	0.5282	0.7382	7.0000	7.0000
e5-large	NMT	0.7787	0.0508	0.6983	0.8329	7.0000	7.0000
e5-large	NQM	0.5978	0.0782	0.4817	0.6731	7.0000	7.0000
e5-large	NQT	0.7187	0.0690	0.5862	0.7815	7.0000	7.0000
e5-large	QMT	0.6405	0.0904	0.4868	0.7559	7.0000	7.0000
LaBSE	NMT	0.7440	0.0515	0.6616	0.8329	7.0000	7.0000
LaBSE	NQM	0.5621	0.0366	0.4988	0.6014	7.0000	7.0000
LaBSE	NQT	0.6636	0.0448	0.6141	0.7201	7.0000	7.0000
LaBSE	QMT	0.5344	0.0284	0.4832	0.5678	7.0000	7.0000
multi-mpnet	NMT	0.7833	0.0485	0.7062	0.8329	7.0000	7.0000
multi-mpnet	NQM	0.6009	0.1116	0.4182	0.7179	7.0000	7.0000
multi-mpnet	NQT	0.7181	0.1074	0.4999	0.8077	7.0000	7.0000
multi-mpnet	QMT	0.6298	0.1012	0.4888	0.7431	7.0000	7.0000

Multilingual max audit: pairwise commutator summary

pair	mean_relative_commutator_norm	std_relative_commutator_norm	min_relative_commutator_norm	max_relative_commutator_norm	n_cells
MT_vs_TM	0.2249	0.1182	0.0000	0.4824	35.0000
NM_vs_MN	0.2029	0.1247	0.0000	0.4424	35.0000
NQ_vs_QN	0.7935	0.1724	0.5208	1.2479	35.0000
NT_vs_TN	0.7575	0.2649	0.2758	1.2584	35.0000
QM_vs_MQ	0.6127	0.1481	0.3322	0.9865	35.0000
QT_vs_TQ	0.6738	0.1308	0.4500	0.9765	35.0000

Multilingual max audit: held-out language endpoint controls

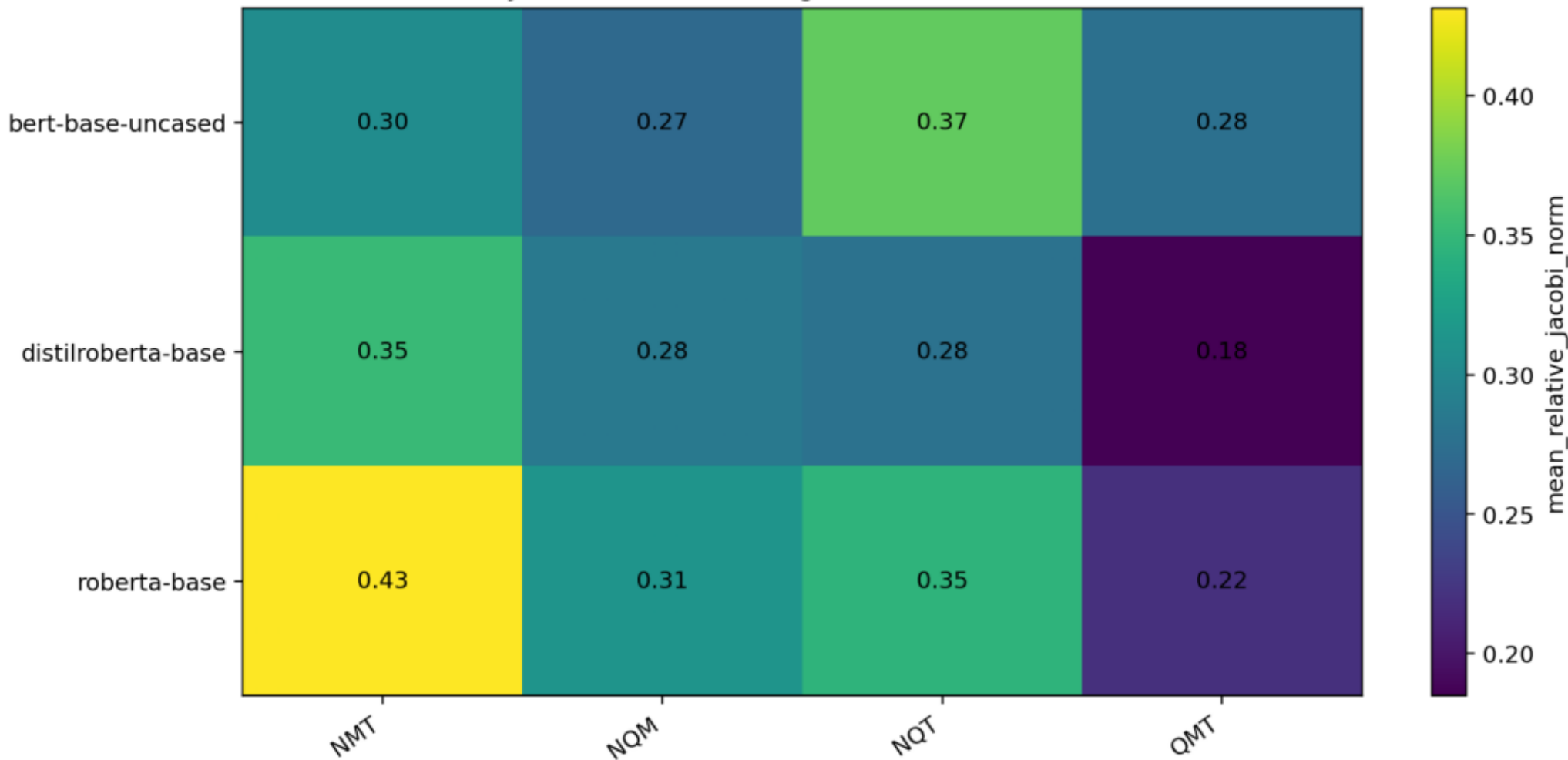
model	feature	mean_macro_f1	min_macro_f1	n
bge-m3	ab_delta_only	0.8349	0.3837	7.0000
bge-m3	ab_endpoint_only	0.7979	0.3341	7.0000
bge-m3	ba_delta_only	0.8011	0.1655	7.0000
bge-m3	ba_endpoint_only	0.8033	0.2182	7.0000
bge-m3	commutator_delta	0.6630	0.1576	7.0000
bge-m3	source_only	0.0476	0.0476	7.0000
mBERT	ab_delta_only	0.6791	0.3377	7.0000
mBERT	ab_endpoint_only	0.6070	0.2436	7.0000
mBERT	ba_delta_only	0.6706	0.3292	7.0000
mBERT	ba_endpoint_only	0.5734	0.1596	7.0000
mBERT	commutator_delta	0.4970	0.0529	7.0000
mBERT	source_only	0.0476	0.0476	7.0000
e5-large	ab_delta_only	0.8352	0.3752	7.0000
e5-large	ab_endpoint_only	0.7844	0.3592	7.0000
e5-large	ba_delta_only	0.8081	0.1708	7.0000
e5-large	ba_endpoint_only	0.7818	0.2224	7.0000
e5-large	commutator_delta	0.6697	0.0459	7.0000
e5-large	source_only	0.0476	0.0476	7.0000
LaBSE	ab_delta_only	0.8666	0.3554	7.0000
LaBSE	ab_endpoint_only	0.8554	0.3729	7.0000
LaBSE	ba_delta_only	0.8248	0.2846	7.0000
LaBSE	ba_endpoint_only	0.8514	0.4066	7.0000
LaBSE	commutator_delta	0.7577	0.2208	7.0000
LaBSE	source_only	0.0476	0.0476	7.0000
multi-mpnet	ab_delta_only	0.7456	0.2418	7.0000
multi-mpnet	ab_endpoint_only	0.7094	0.0476	7.0000
multi-mpnet	ba_delta_only	0.6671	0.1863	7.0000
multi-mpnet	ba_endpoint_only	0.6805	0.1604	7.0000
multi-mpnet	commutator_delta	0.6736	0.0877	7.0000
multi-mpnet	source_only	0.0476	0.0476	7.0000

Multilingual max audit: cross-language centroid consistency

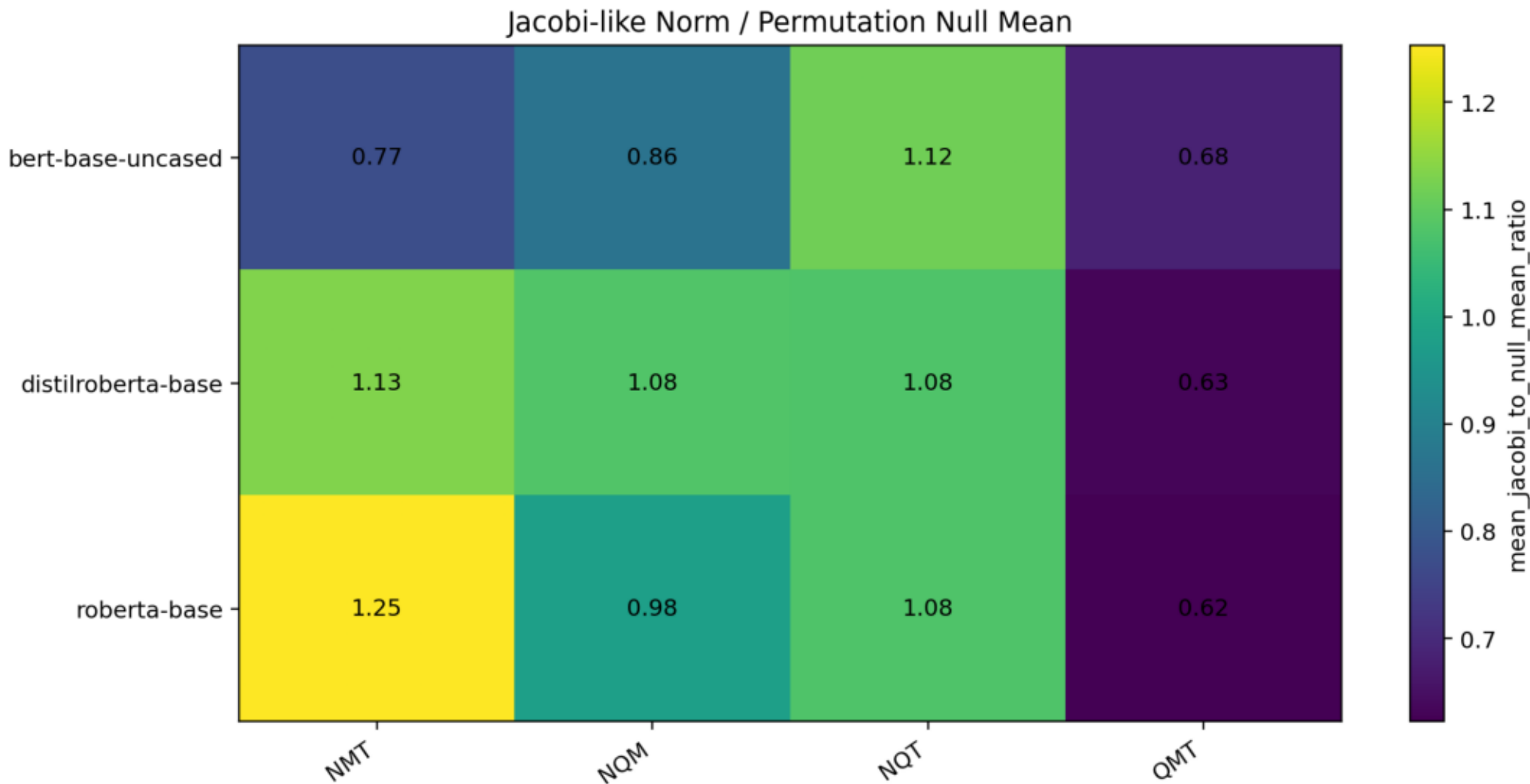
kind	mean_centroid_cosine	std_centroid_cosine	min_centroid_cosine	max_centroid_cosine	n
pair_commutator	0.3118	0.3315	-0.4805	0.9517	630.0000
triple_signed	0.3216	0.3394	-0.3325	0.9475	420.0000

Signed permutation relative norm

Jacobi-like Alternating Sum Relative Norm

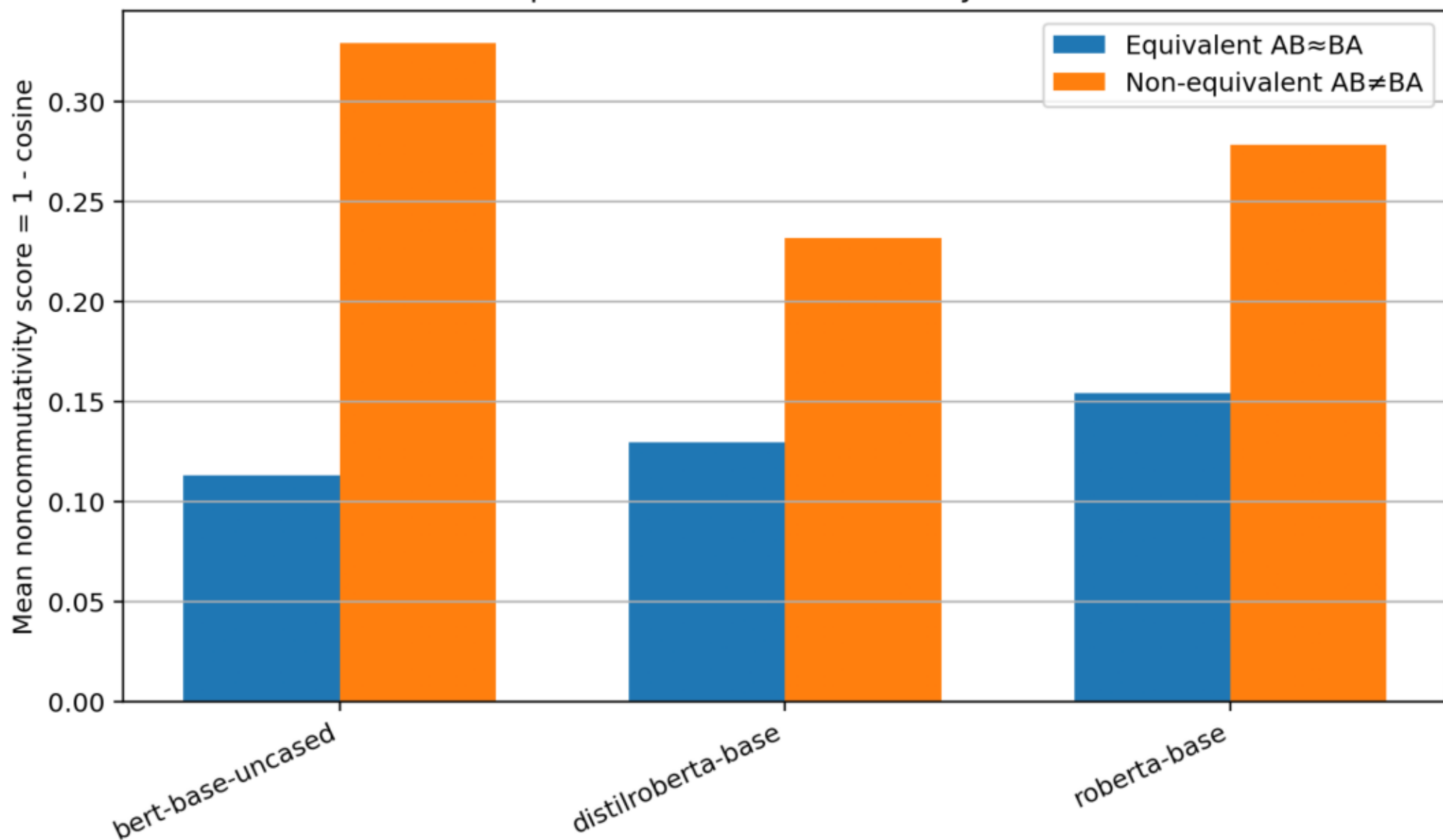


Signed permutation norm versus permutation-null

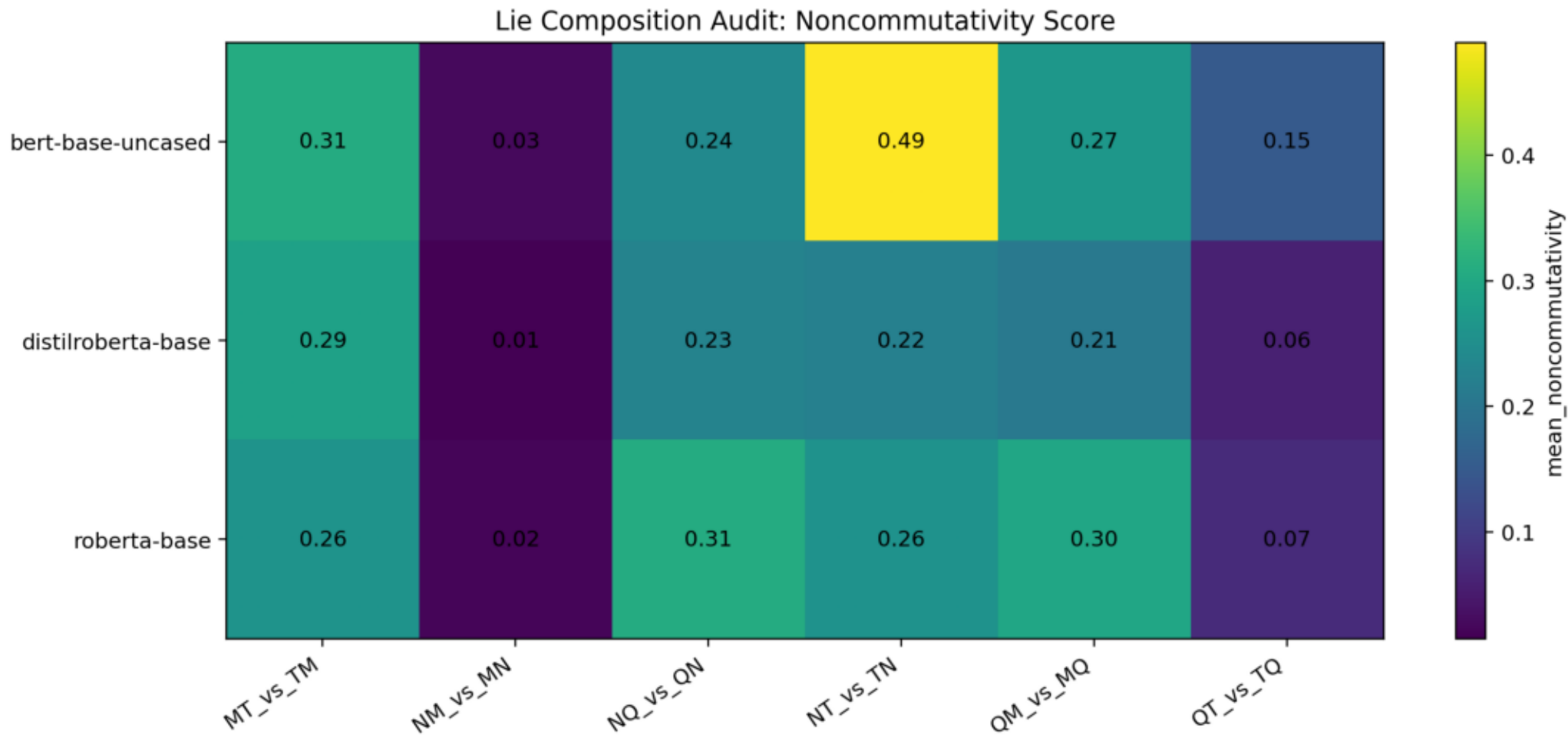


Semantic equivalent vs non-equivalent control

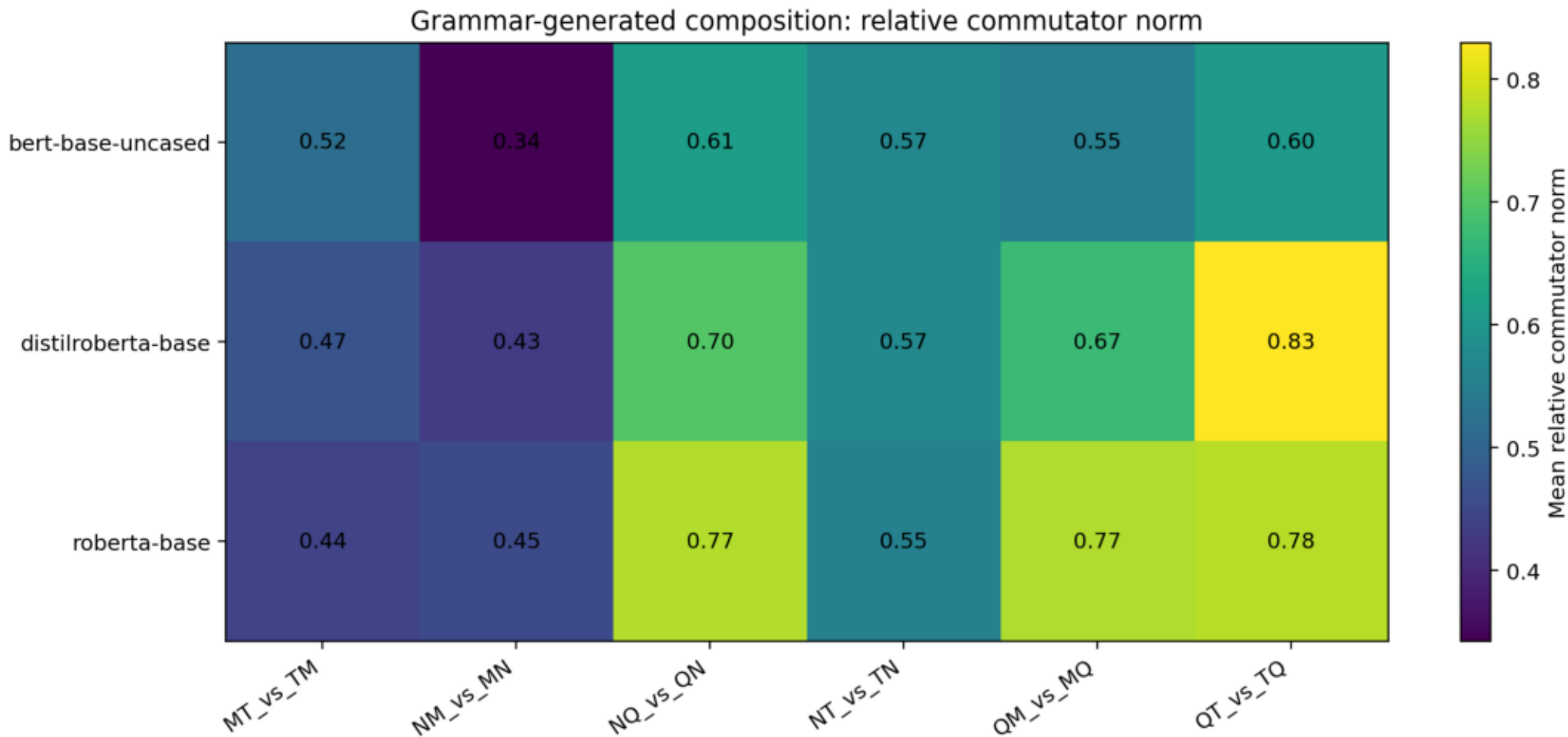
Semantic Equivalence Control for Lie-Style Commutators



Composition noncommutativity heatmap

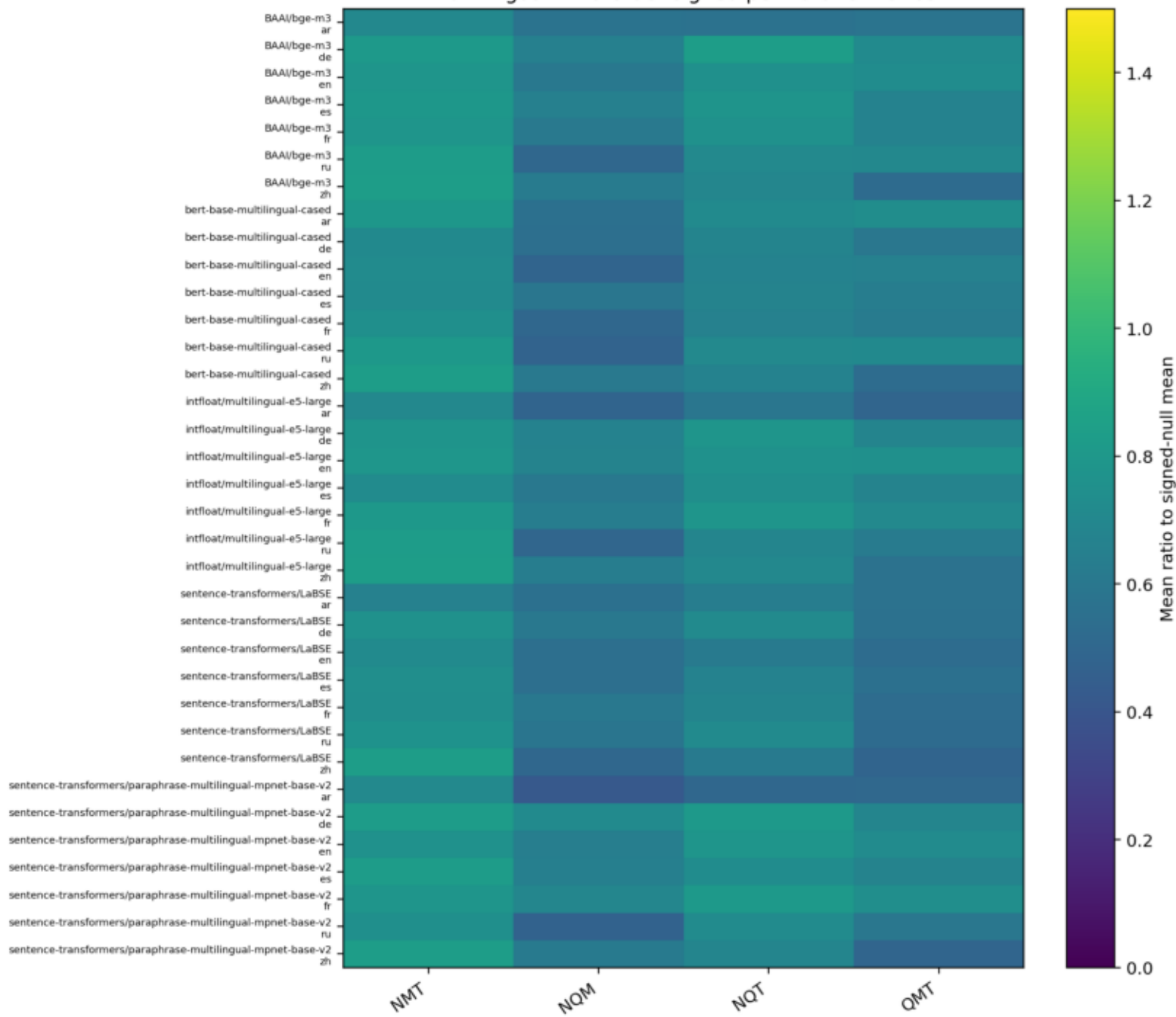


Grammar composition relative commutator heatmap



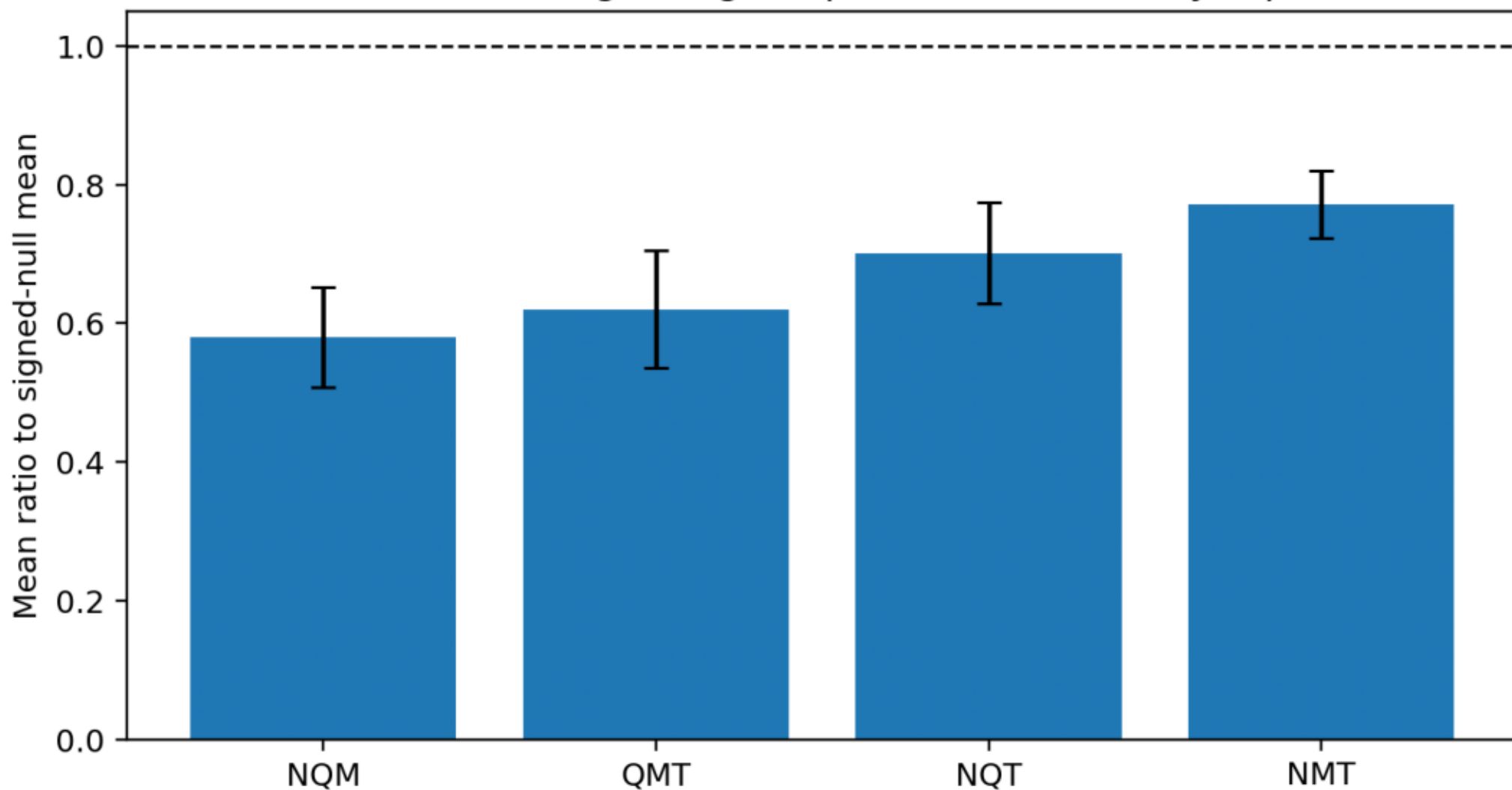
Multilingual signed-permutation ratios

Multilingual third-order signed permutation ratios

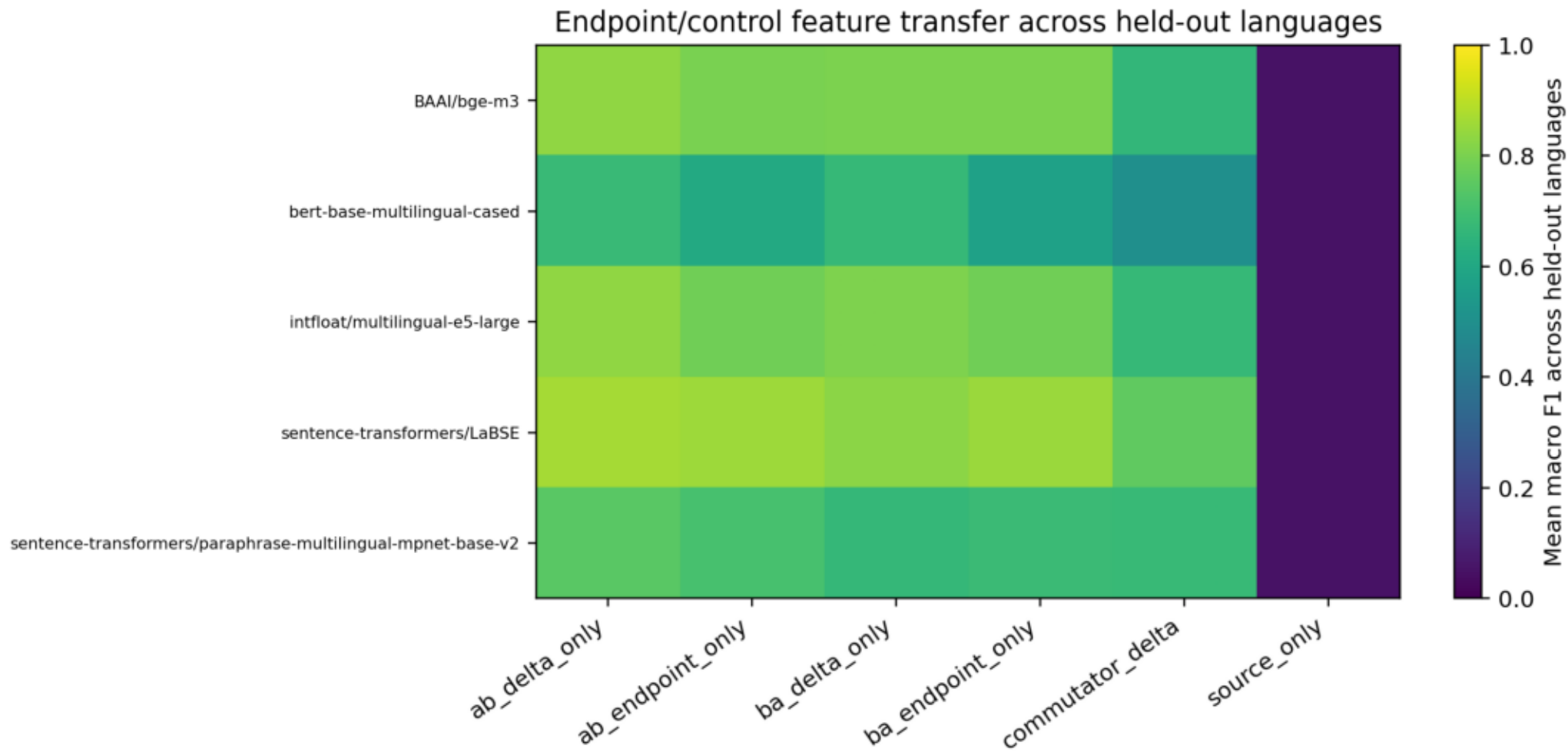


Multilingual global triple ratios

Global multilingual signed-permutation ratio by triple

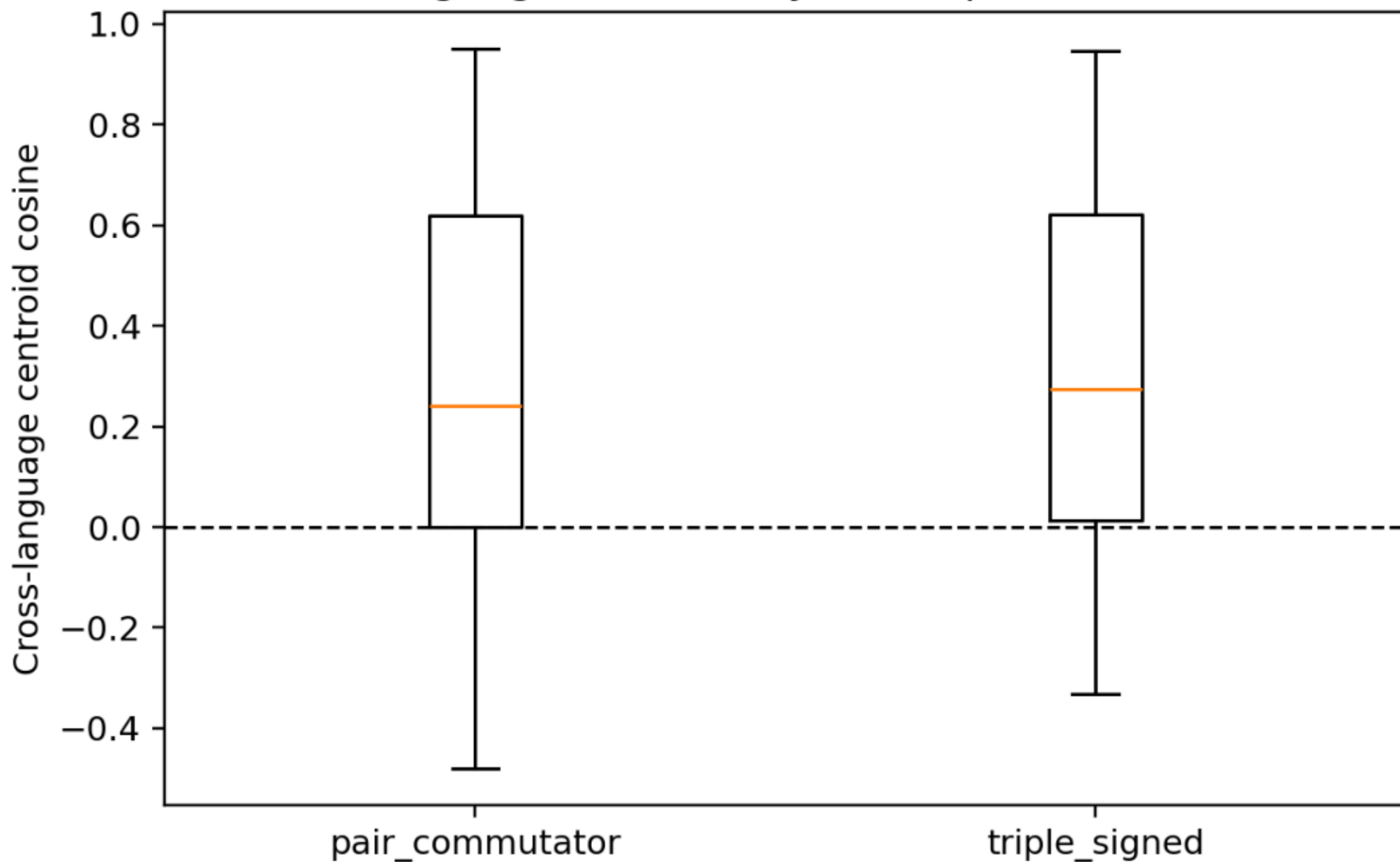


Multilingual endpoint controls



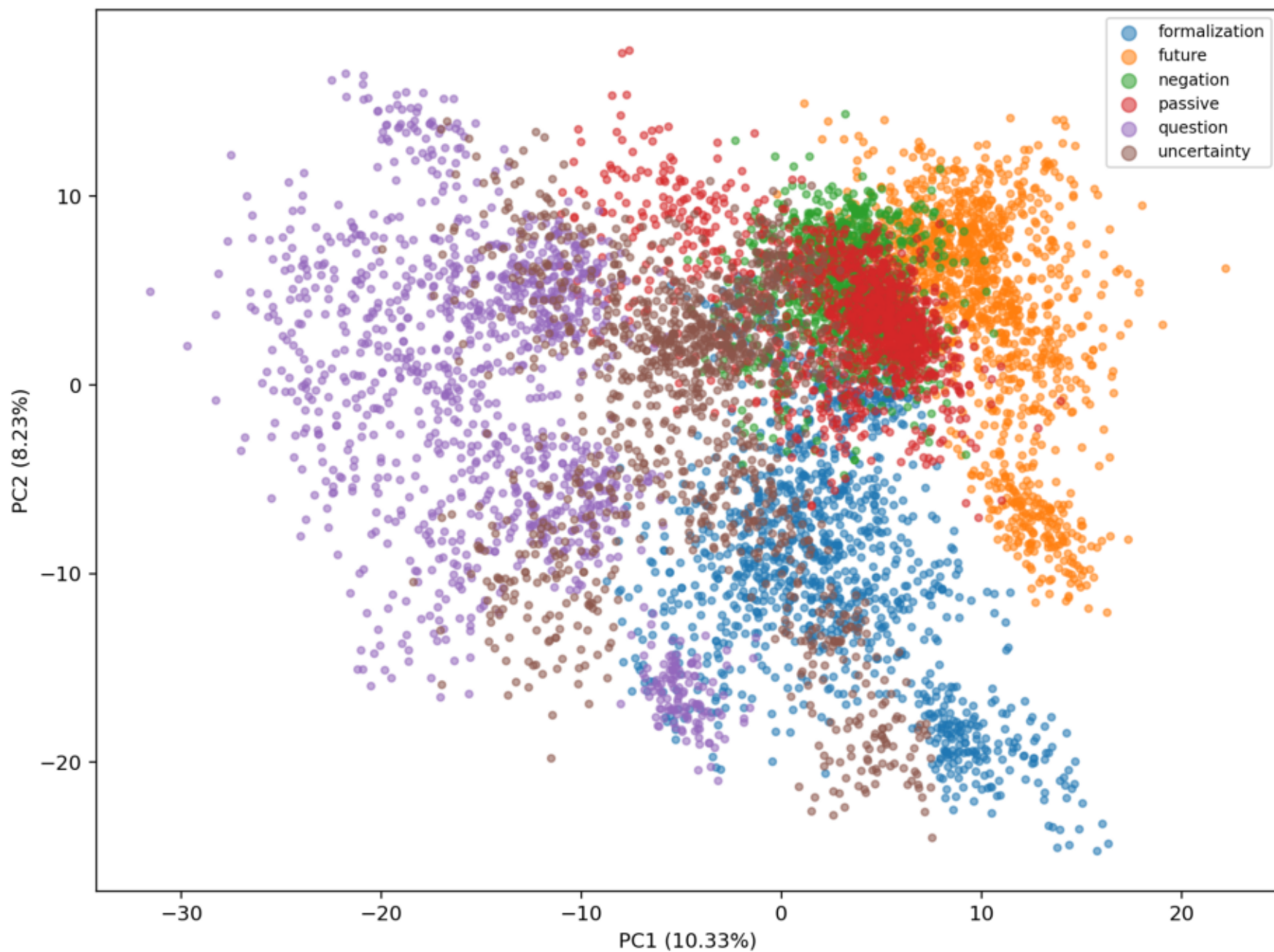
Multilingual cross-language centroid consistency

Cross-language consistency of composition centroids



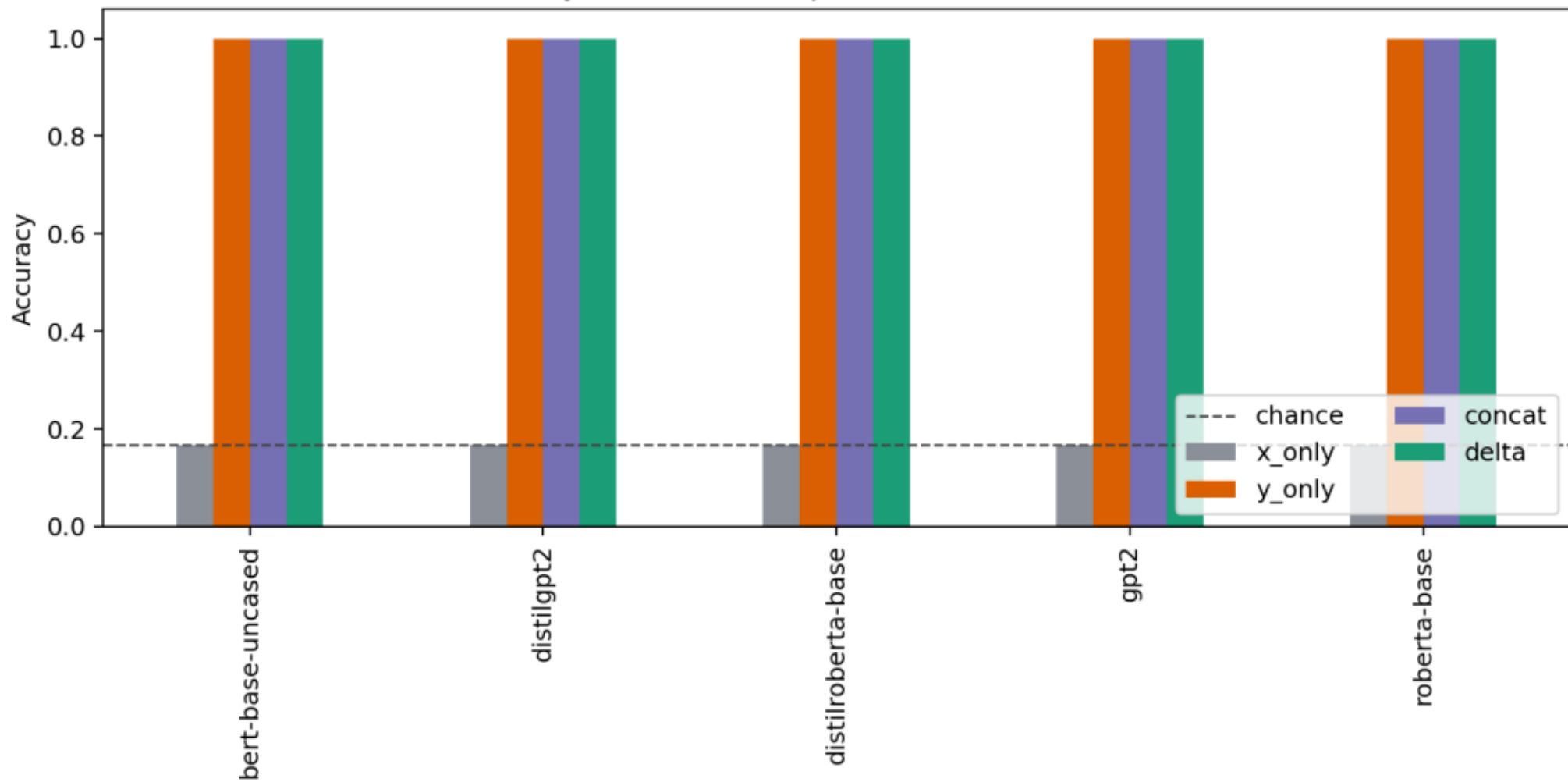
Original PCA class geometry

PCA of Delta Vectors: bert-base-uncased



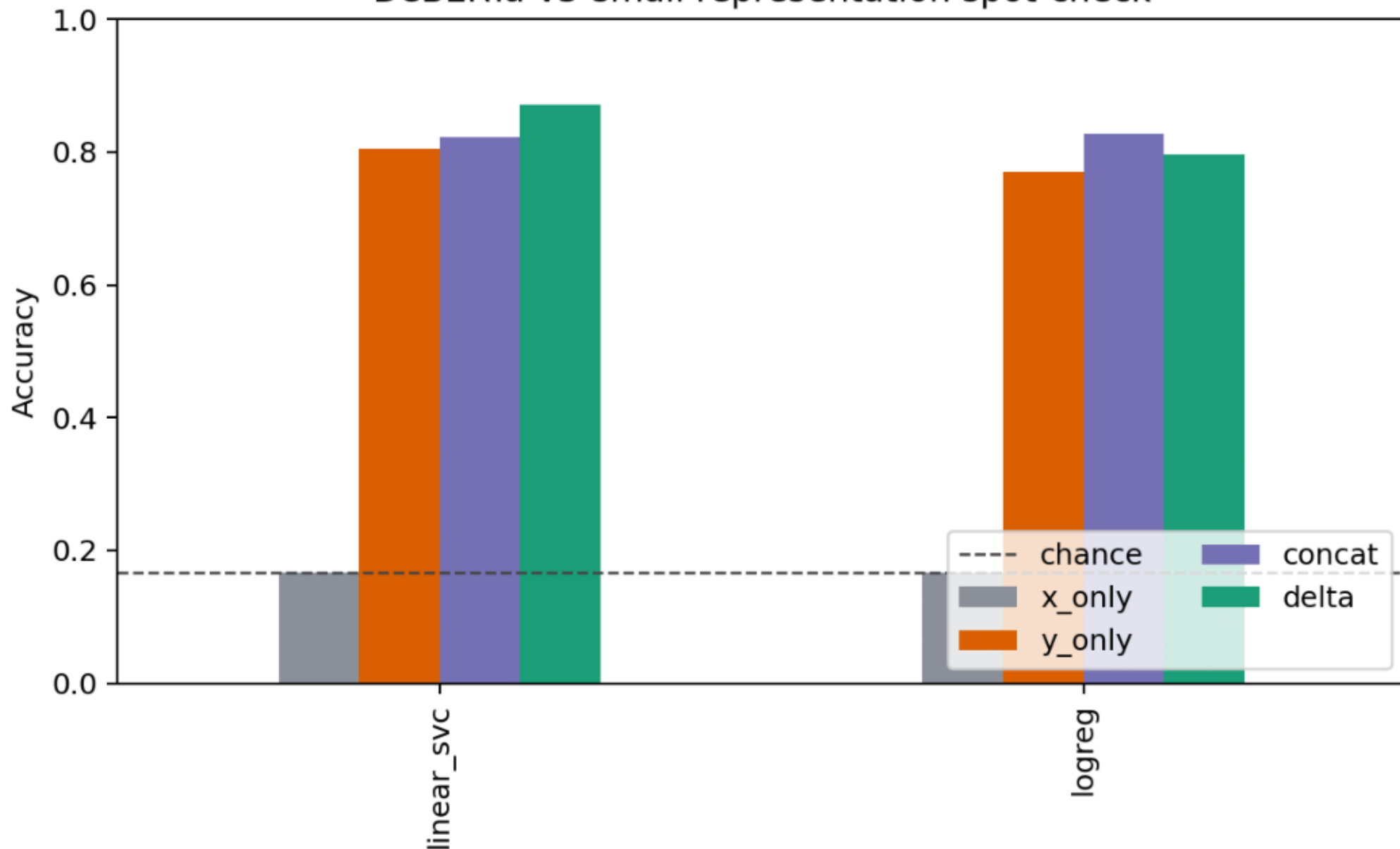
Syntax representation ablation

Syntax holdout representation ablation

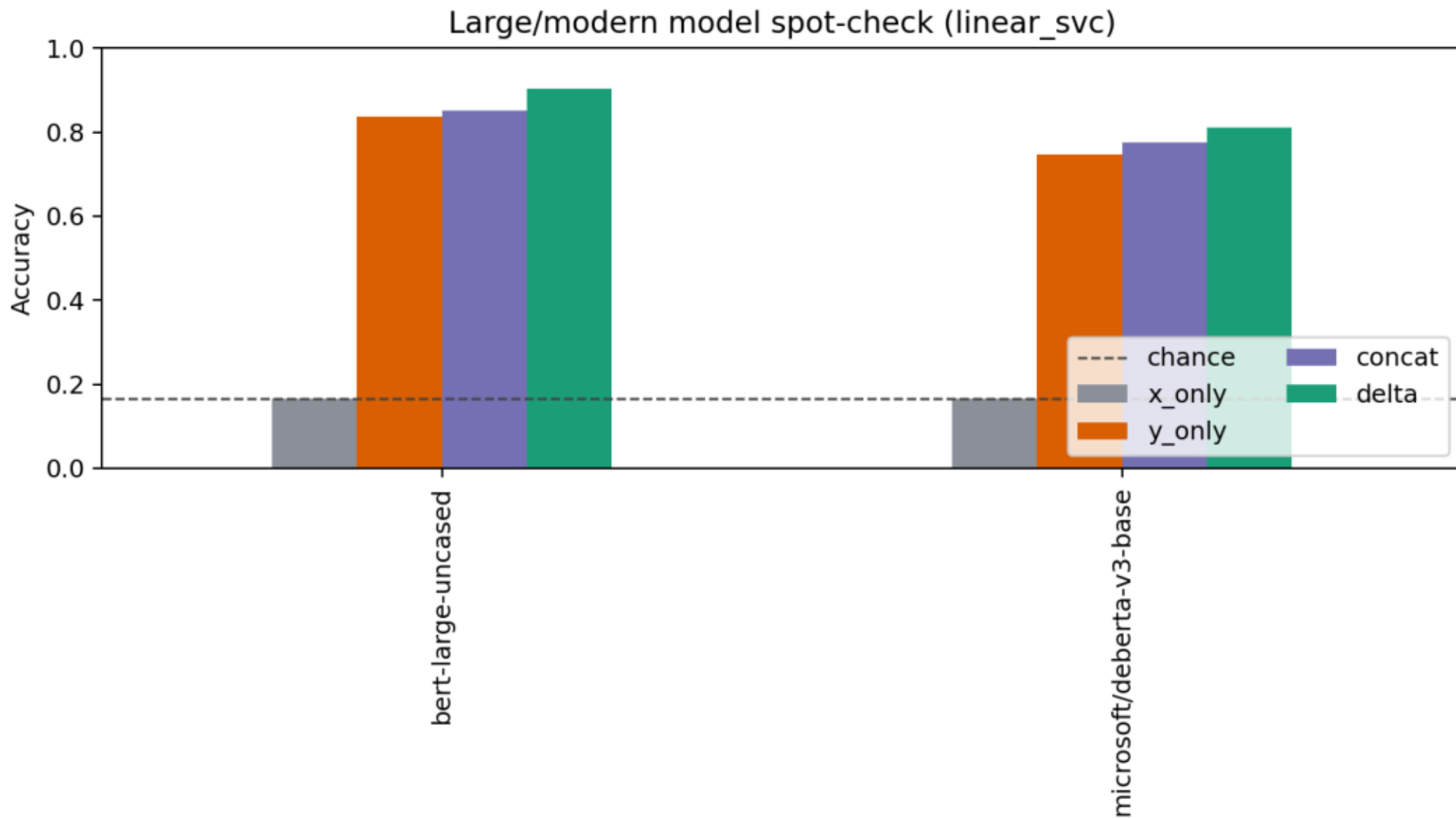


DeBERTa-v3-small spot-check

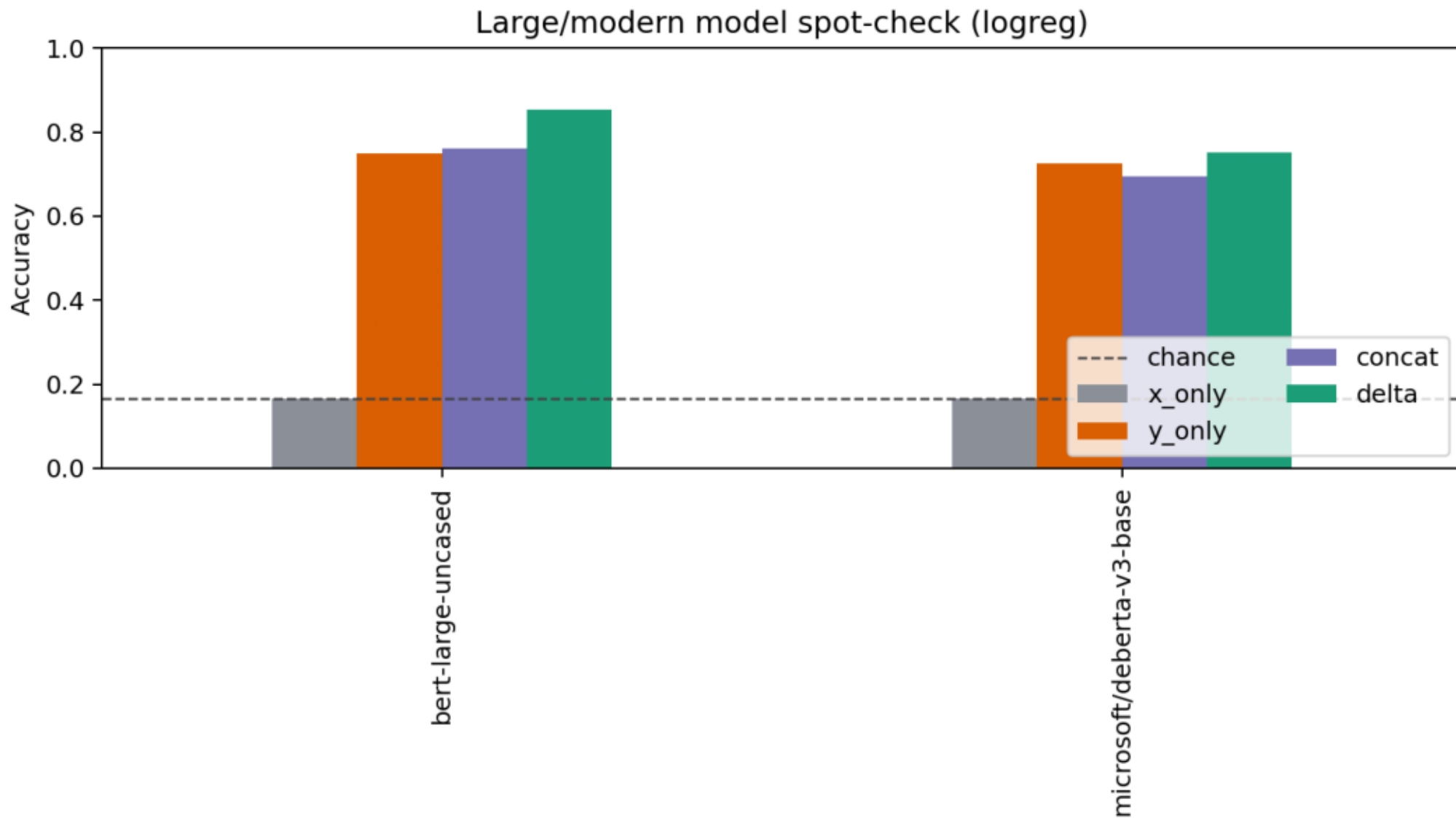
DeBERTa-v3-small representation spot-check



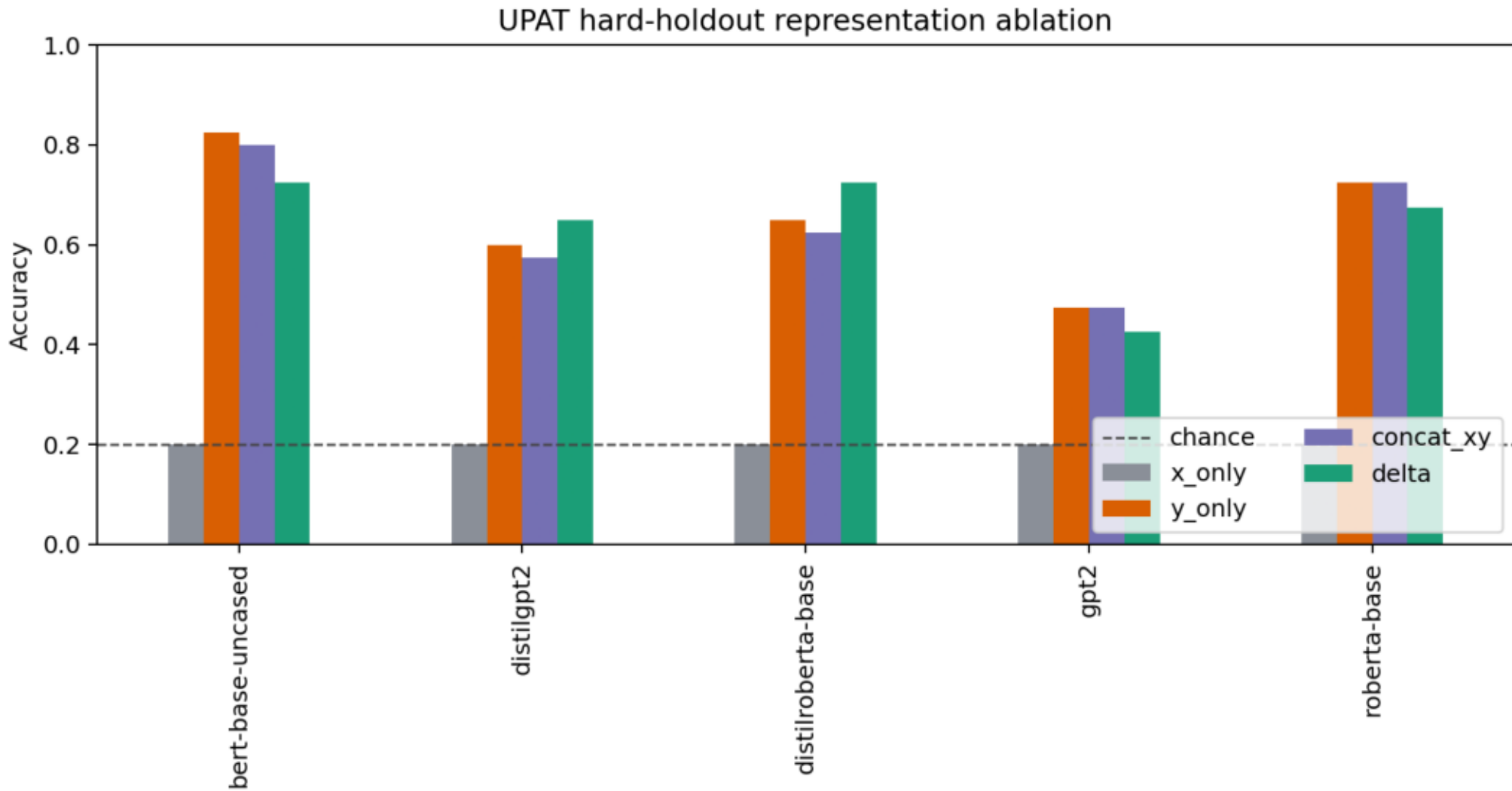
Large/modern spot-check: Linear SVC



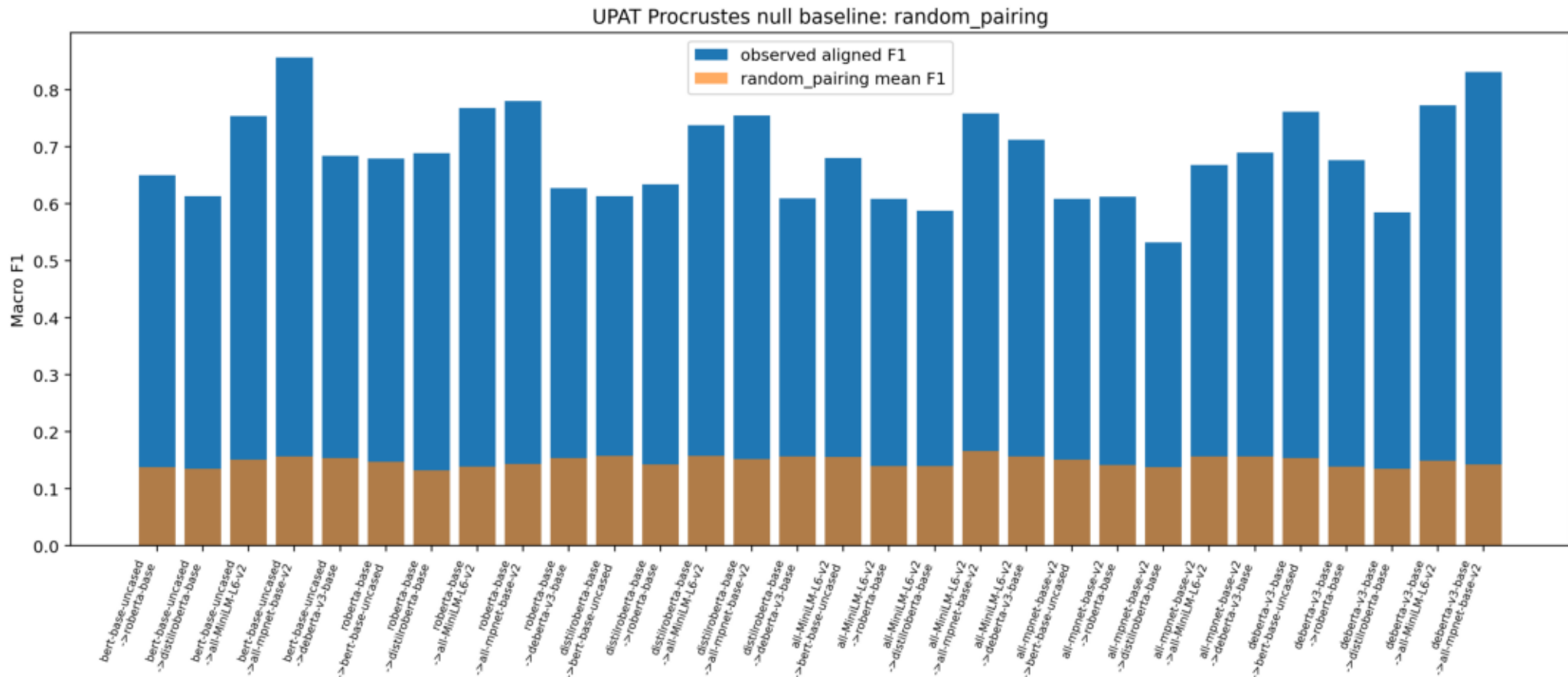
Large/modern spot-check: logistic regression



UPAT hard-holdout ablation

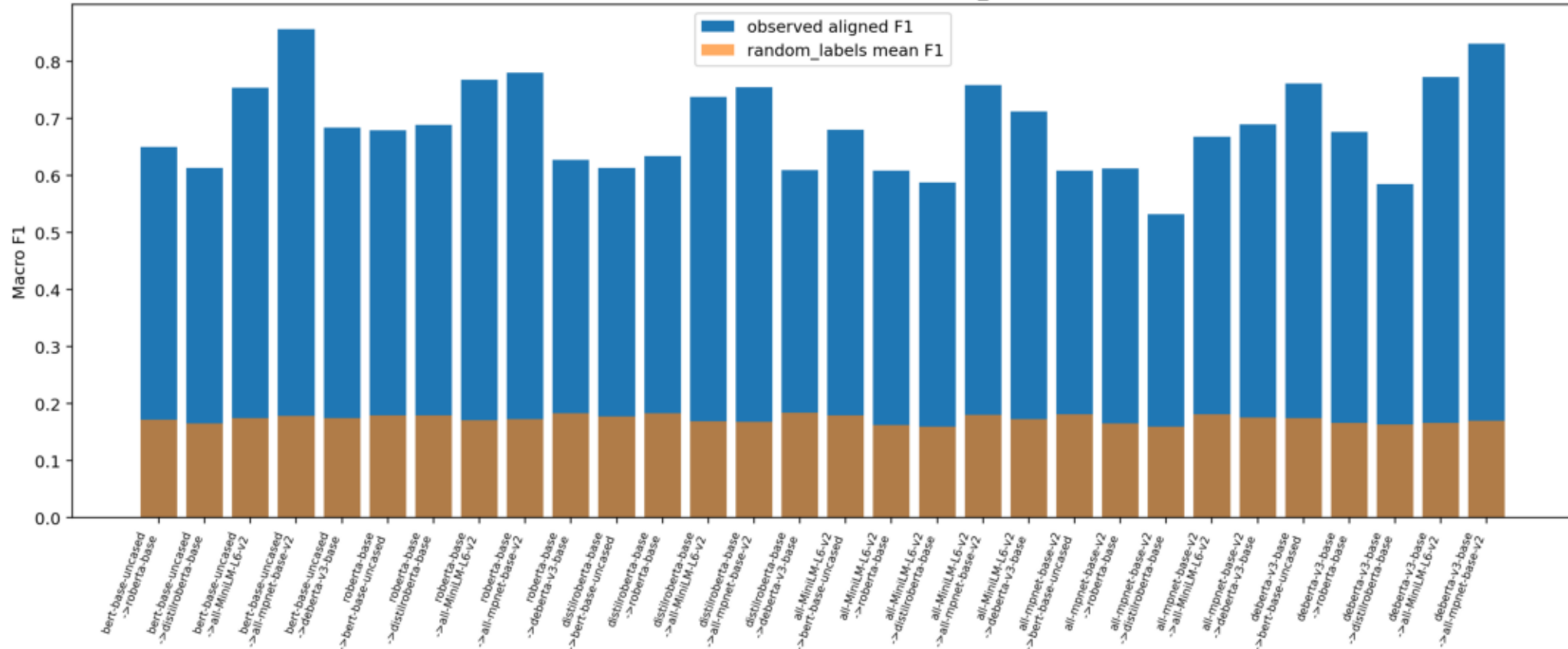


UPAT-large Procrustes random-pairing null



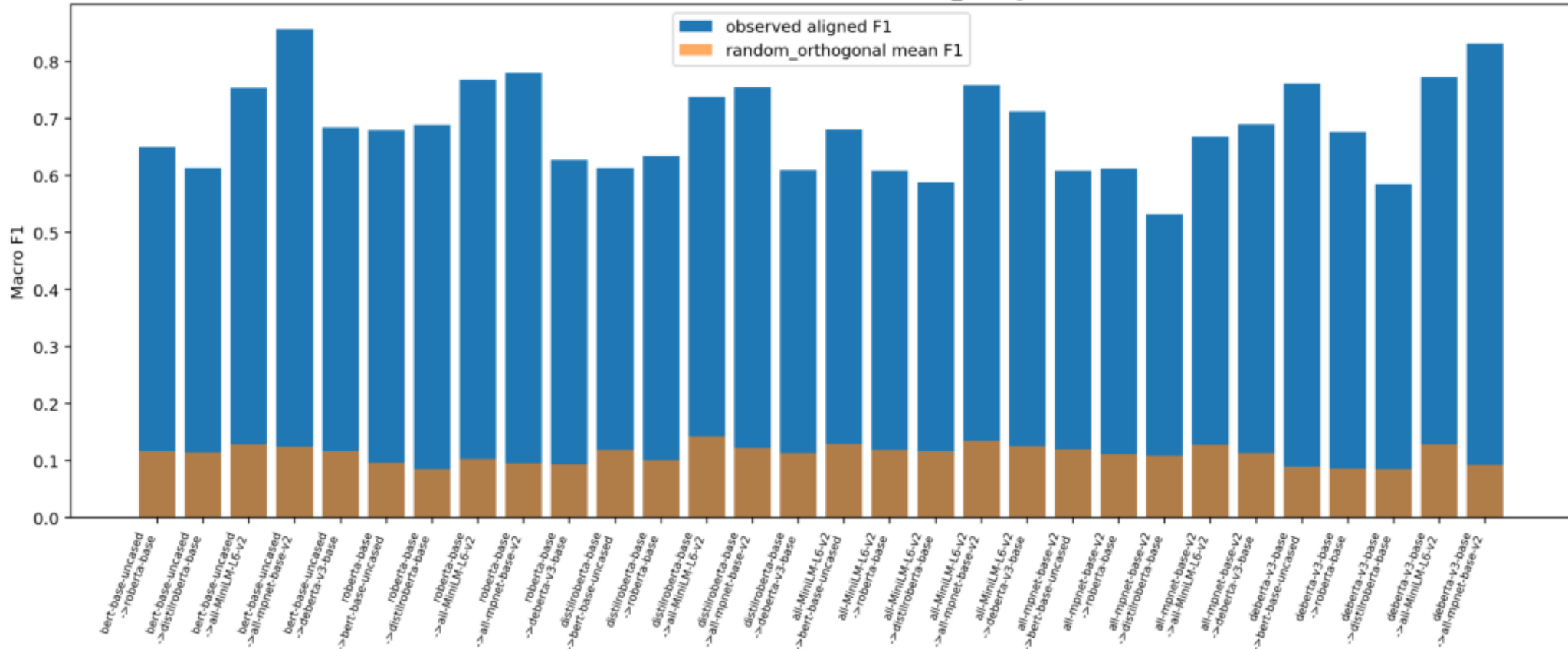
UPAT-large Procrustes random-label null

UPAT Procrustes null baseline: random_labels

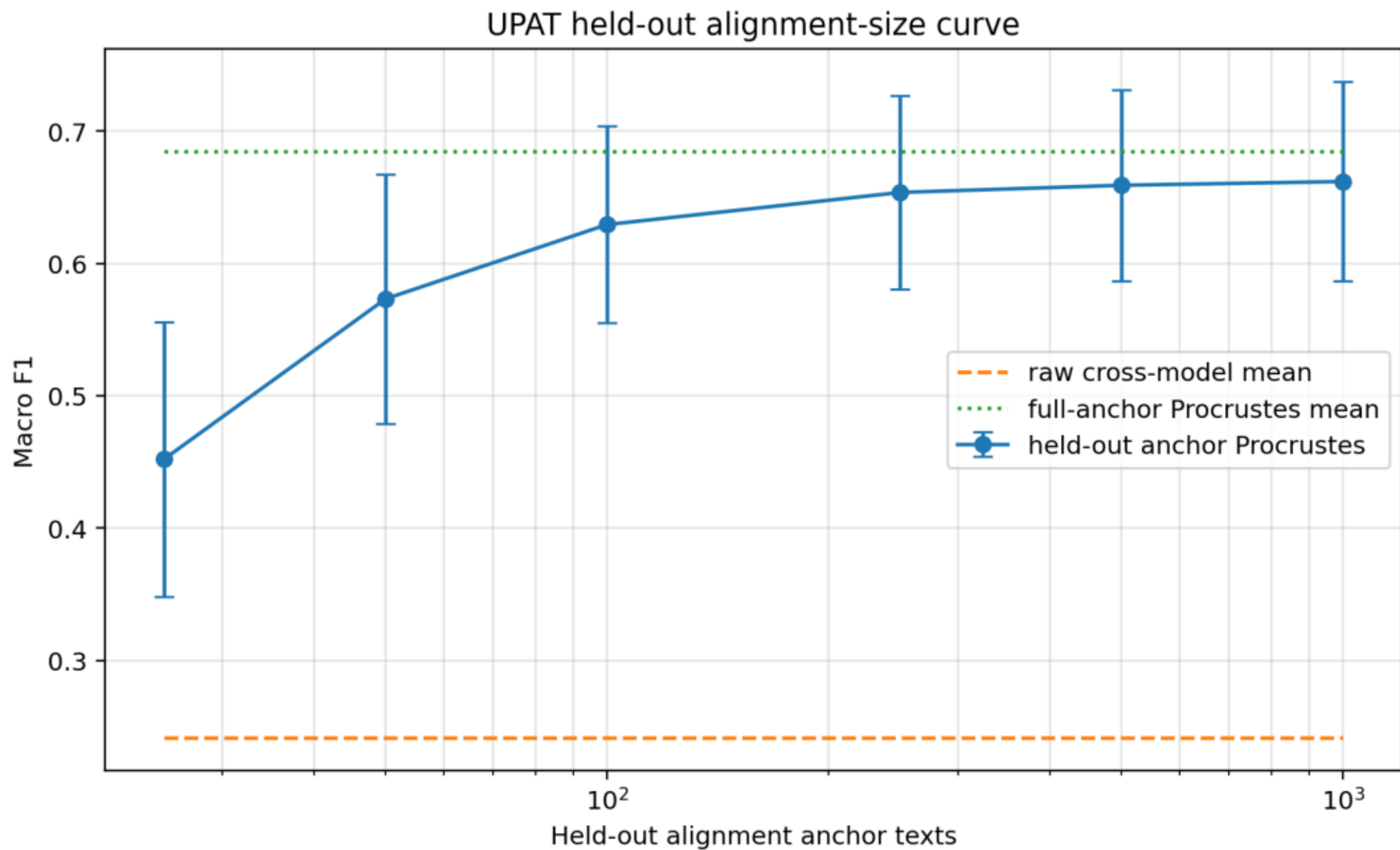


UPAT-large Procrustes random-orthogonal null

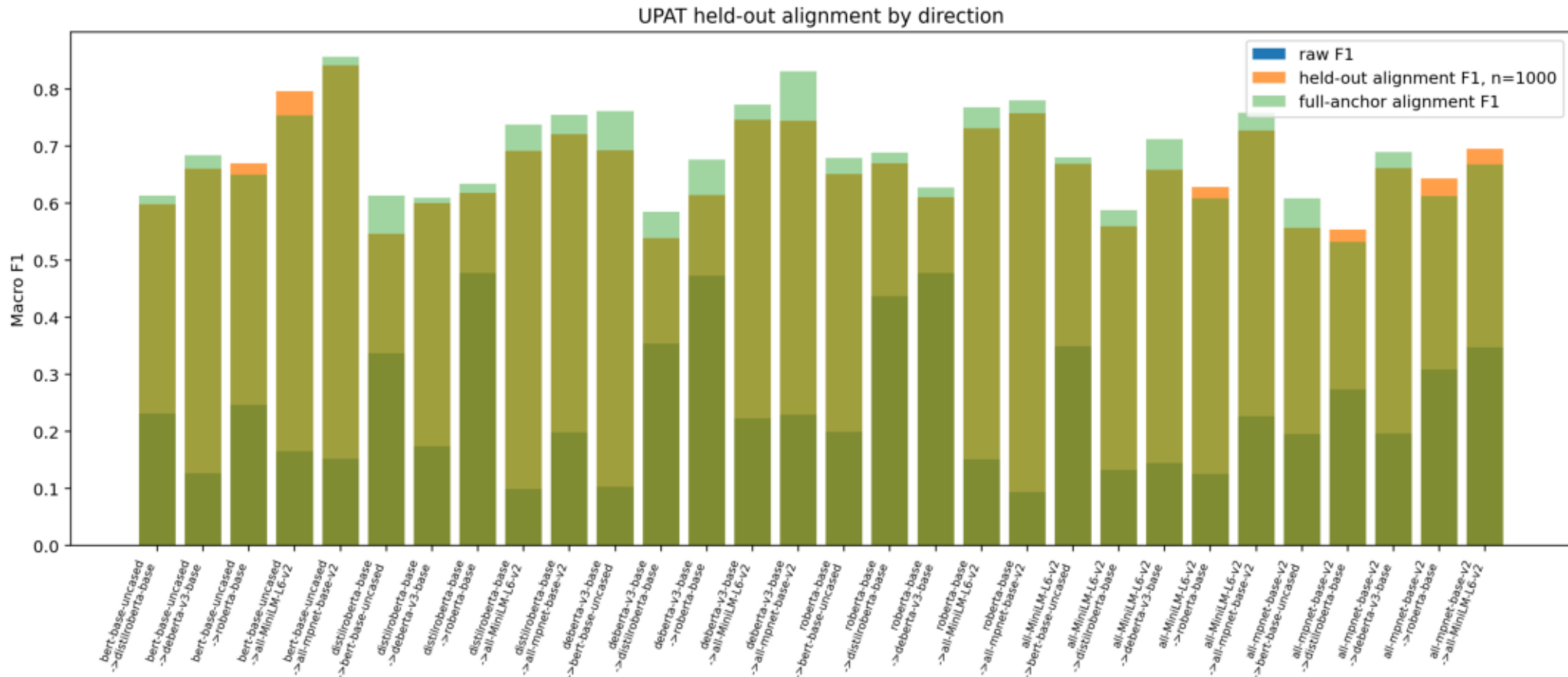
UPAT Procrustes null baseline: random_orthogonal



UPAT-large held-out alignment-size curve

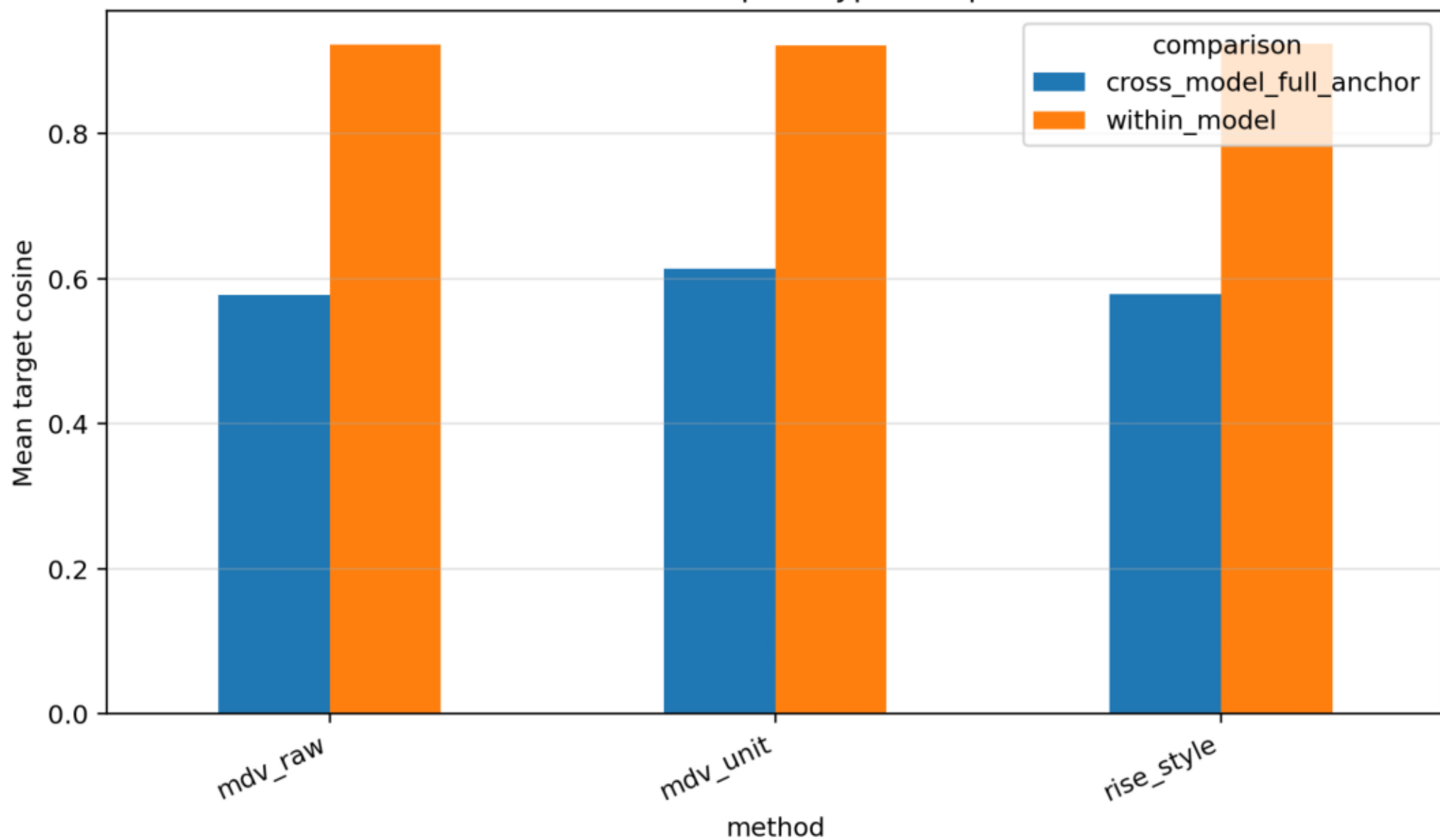


UPAT-large held-out alignment by direction



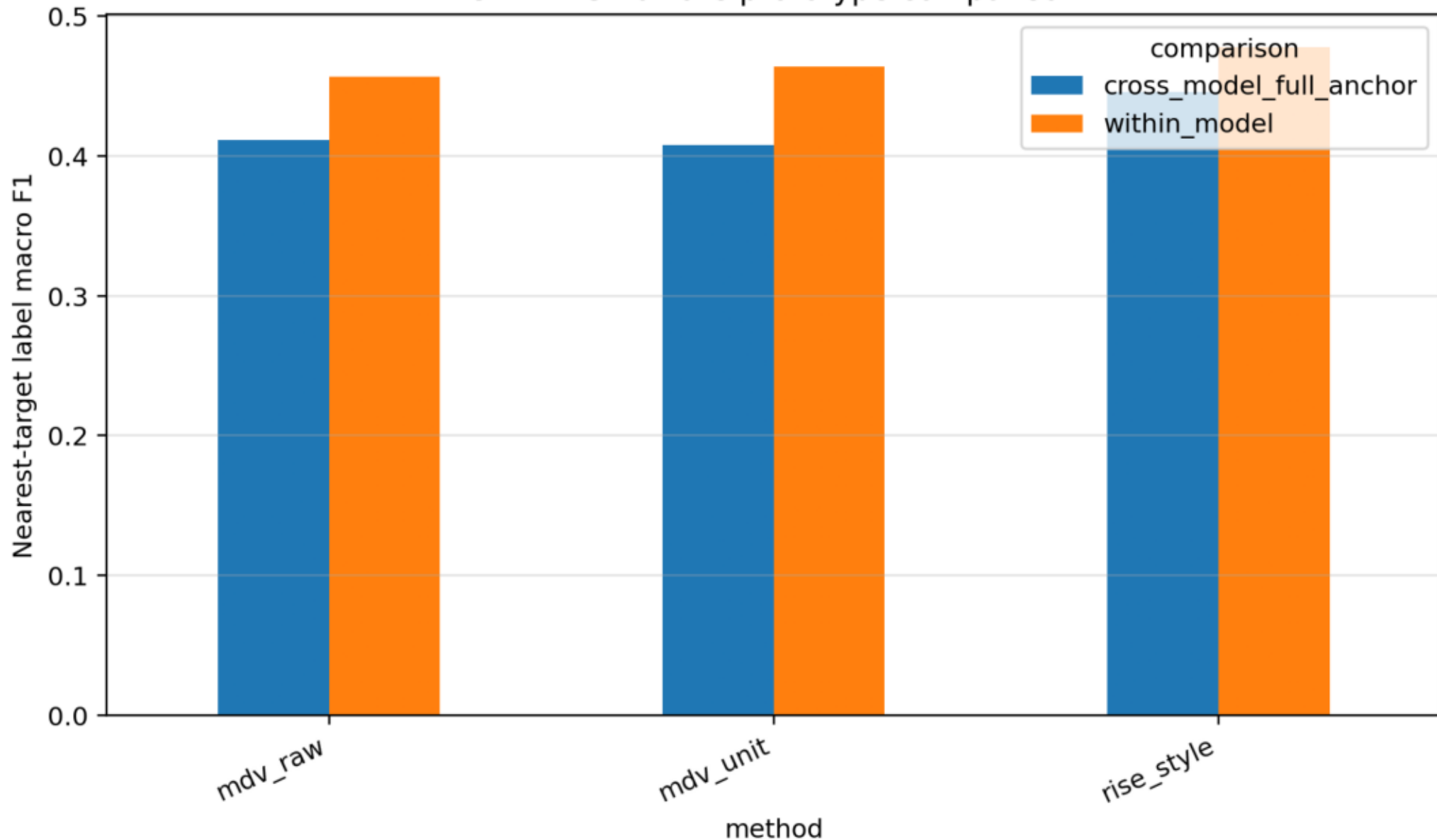
UPAT RISE-aware target prediction cosine

UPAT RISE-aware prototype comparison

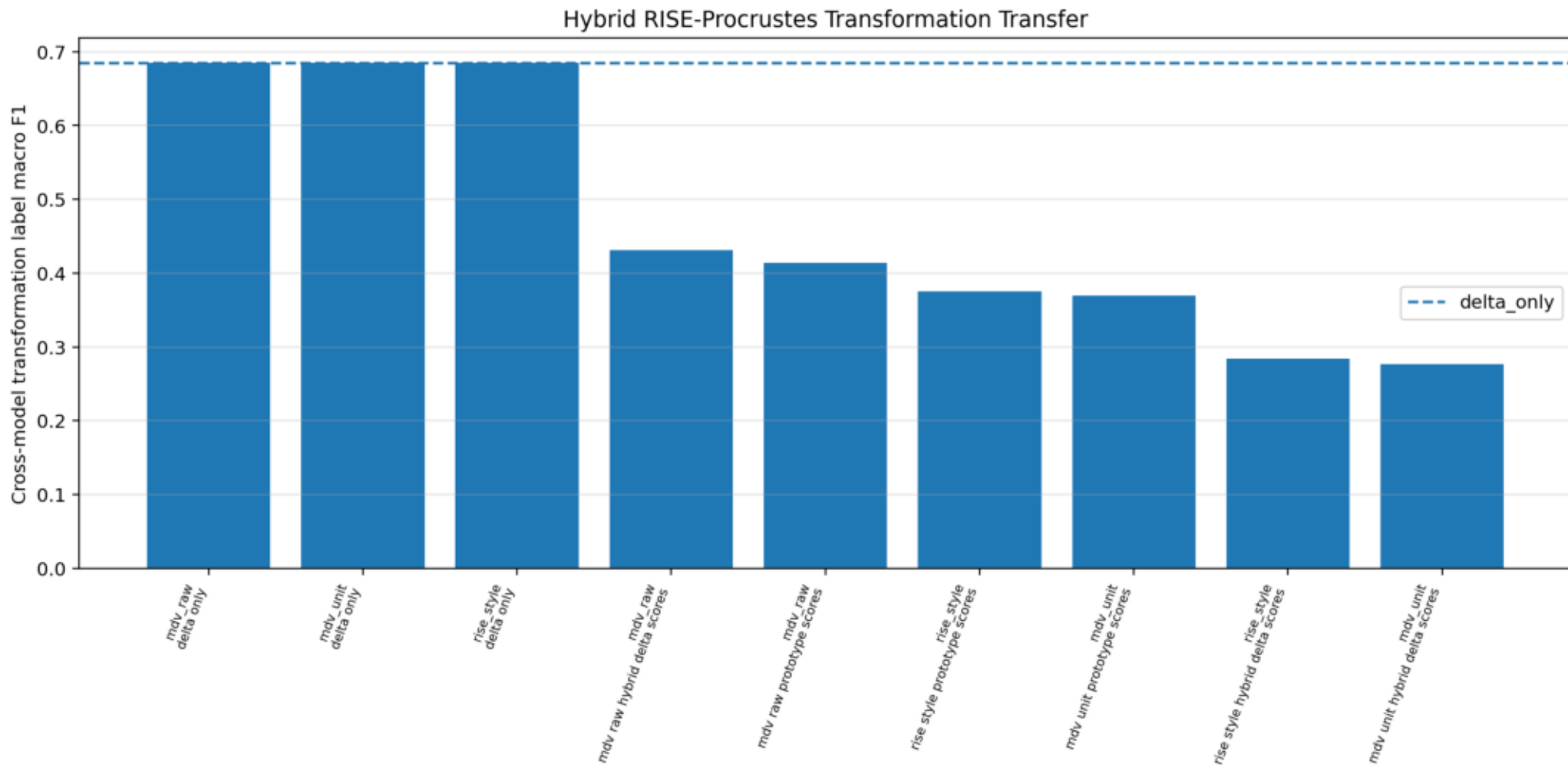


UPAT RISE-aware nearest-target class retrieval

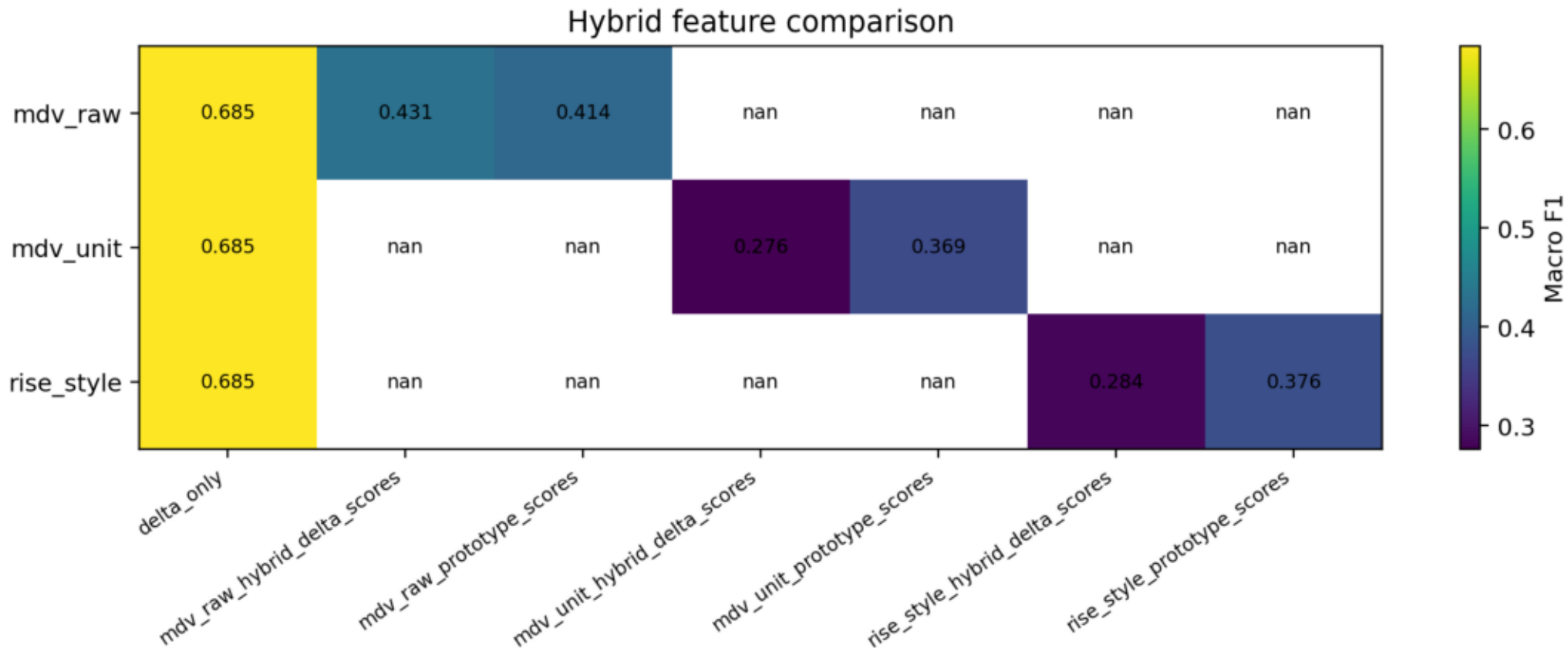
UPAT RISE-aware prototype comparison



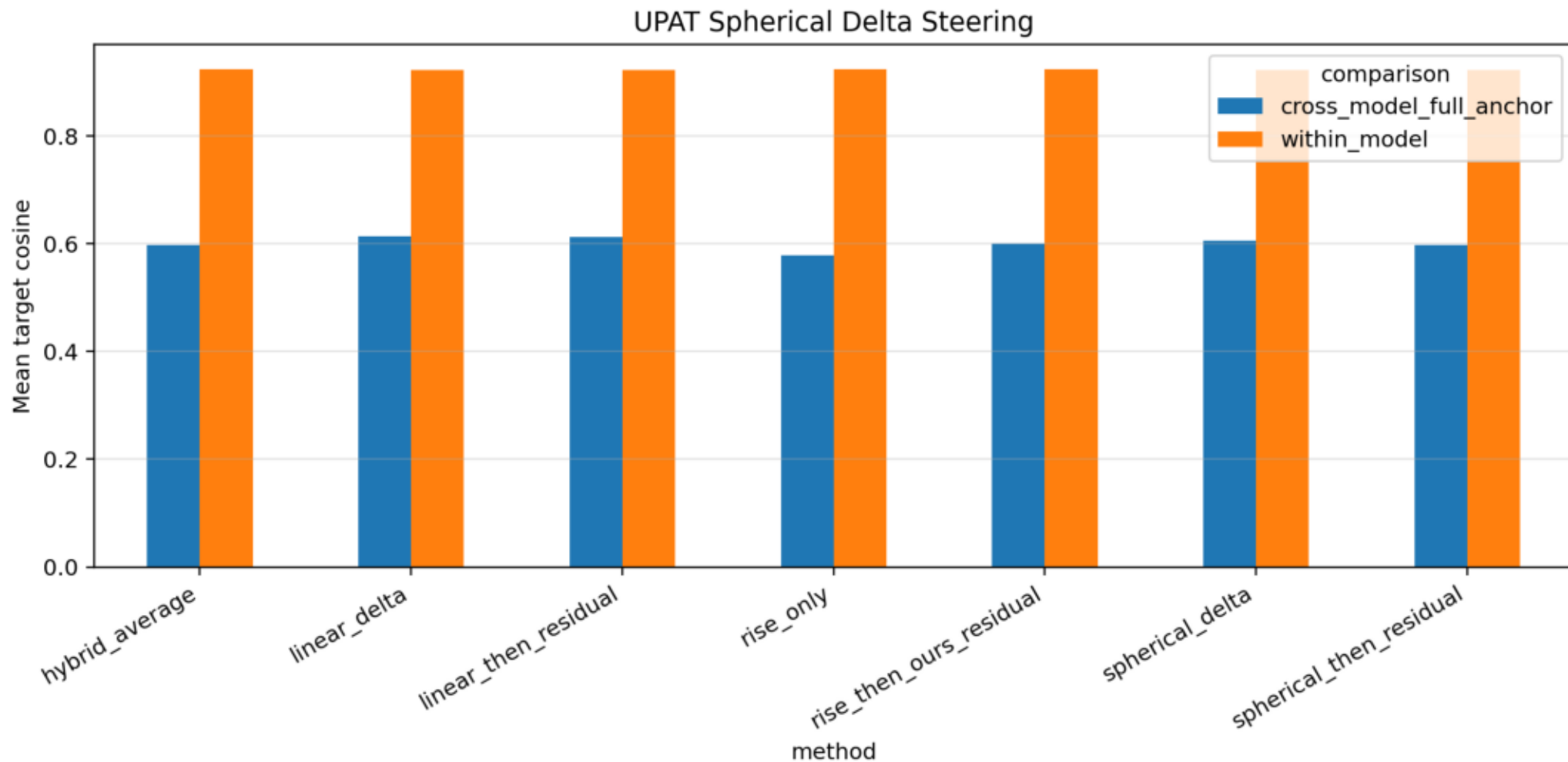
UPAT hybrid RISE-Procrustes label F1



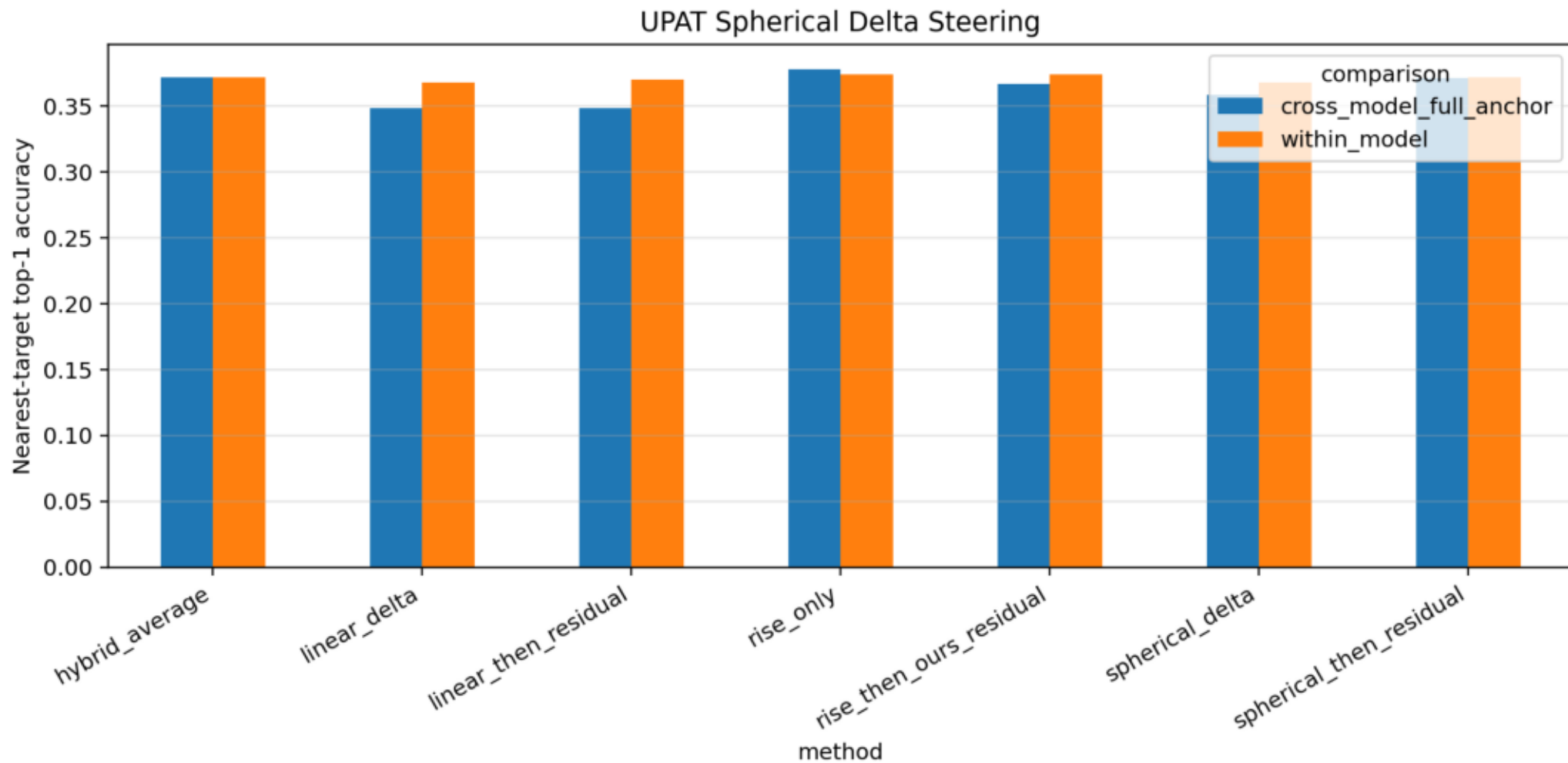
UPAT hybrid RISE-Procrustes heatmap



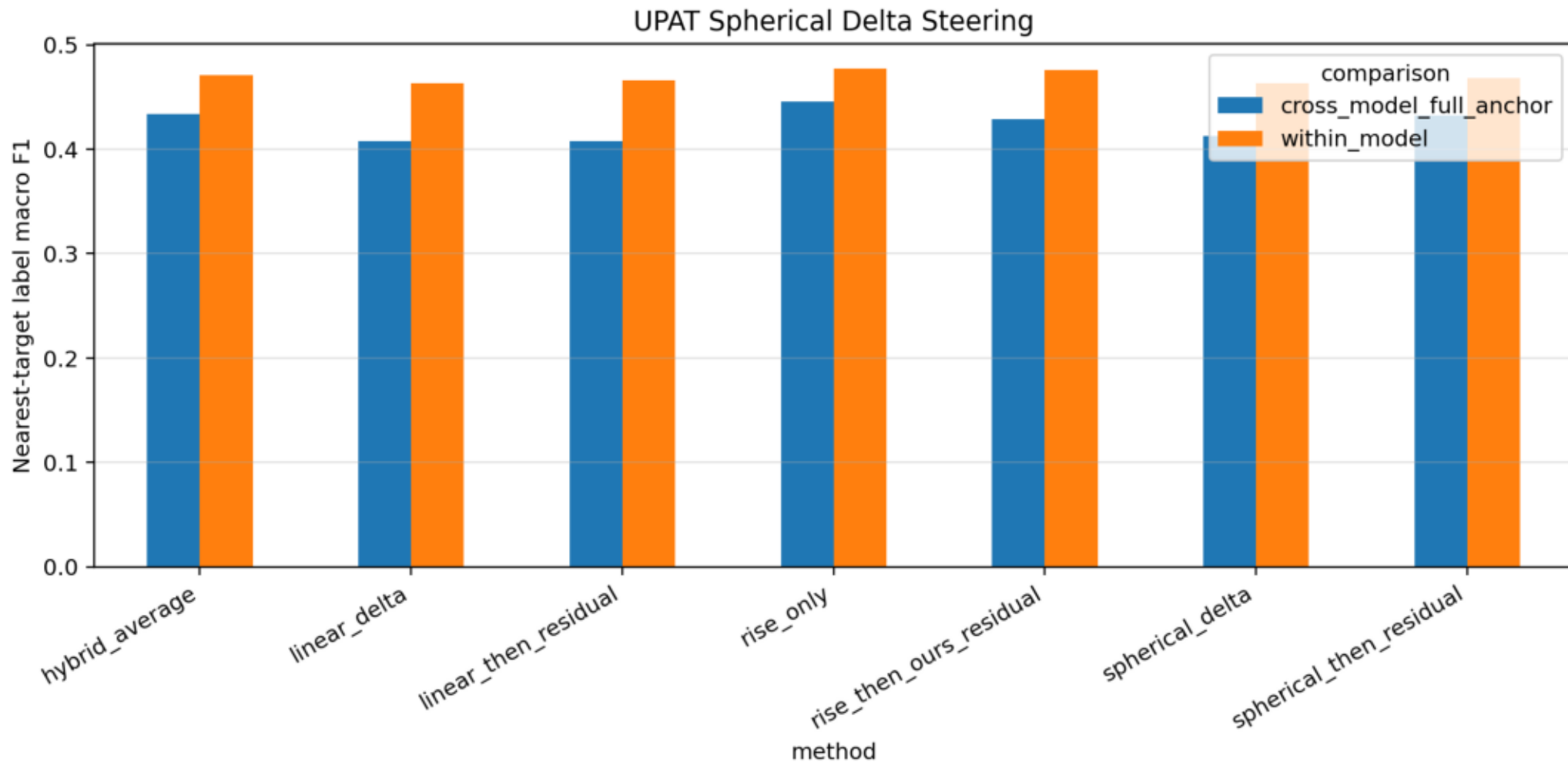
UPAT spherical delta target cosine



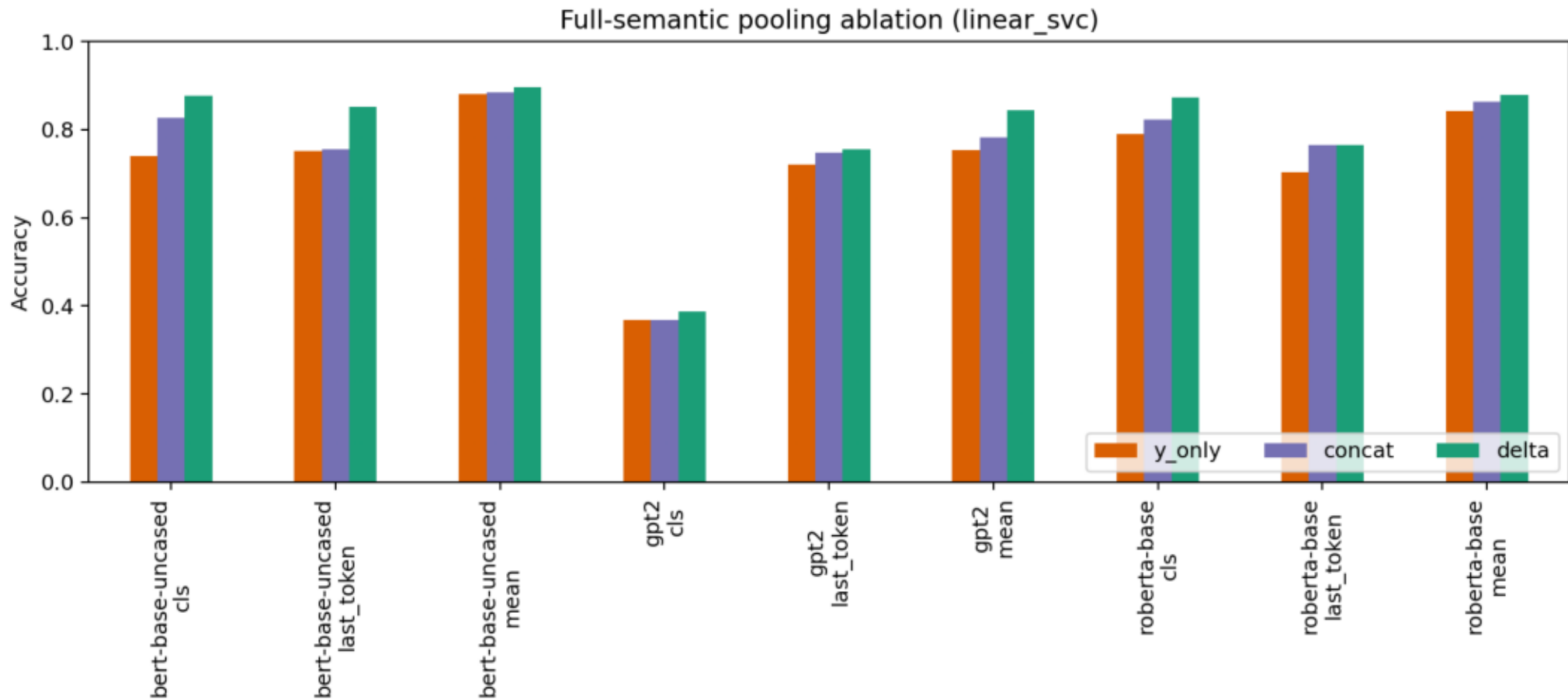
UPAT spherical delta retrieval top-1



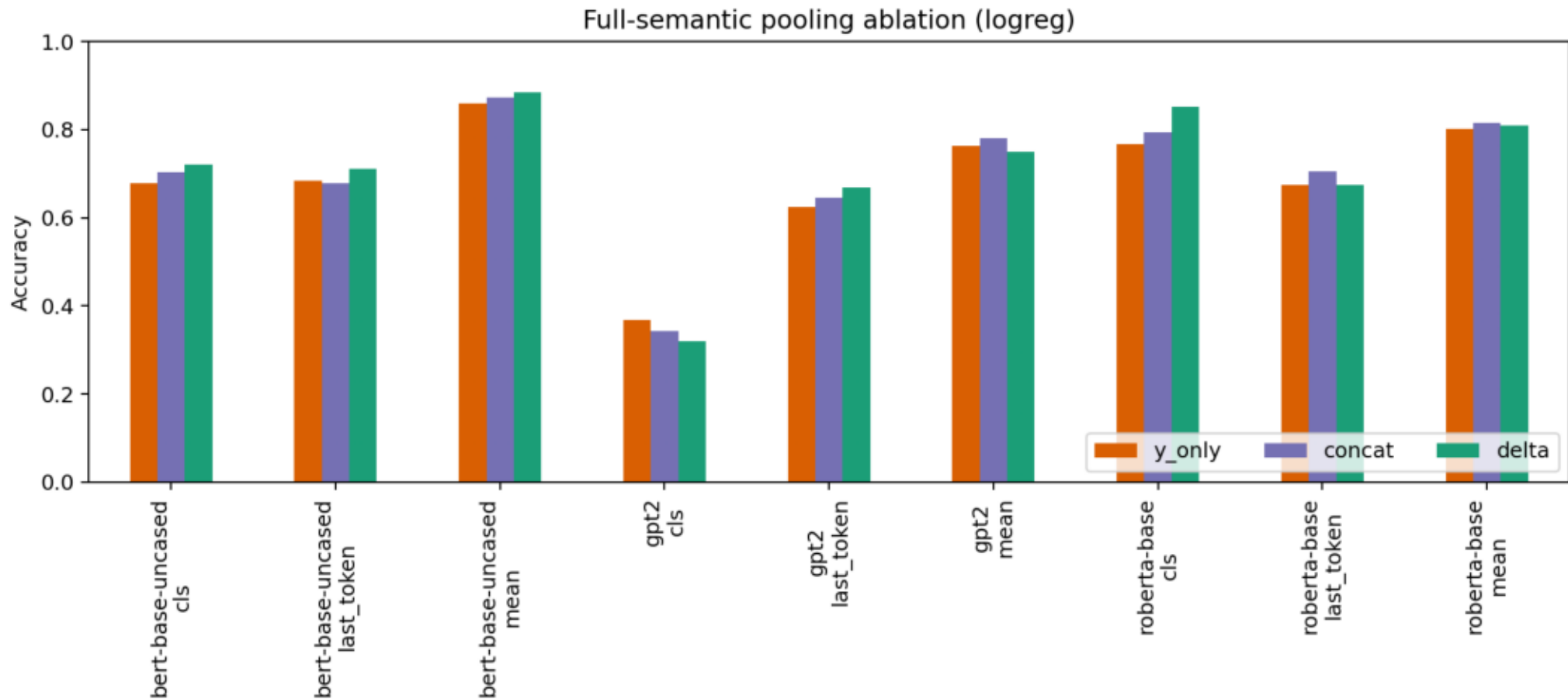
UPAT spherical delta retrieval label F1



Full-semantic pooling ablation: Linear SVC

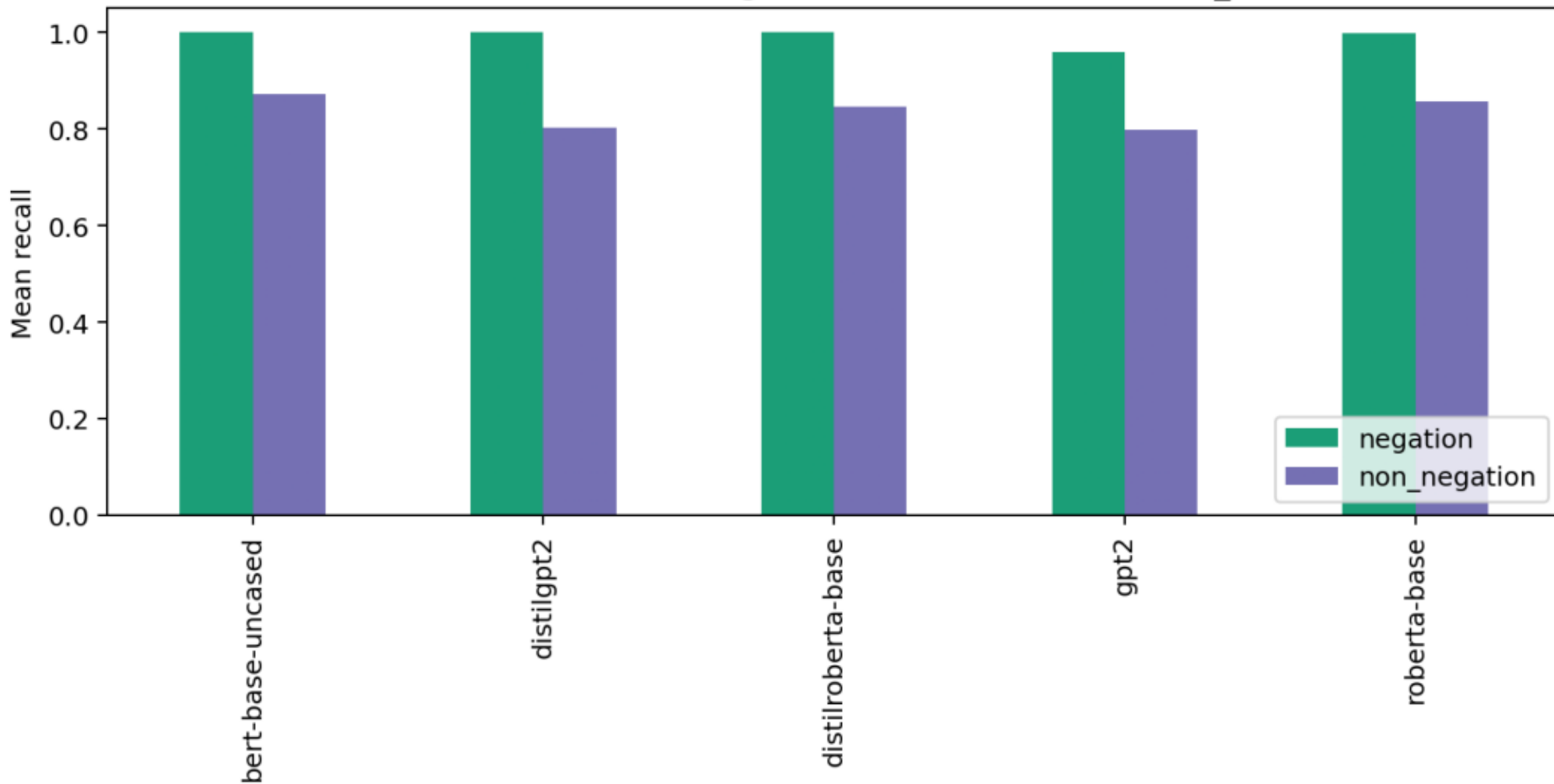


Full-semantic pooling ablation: logistic regression



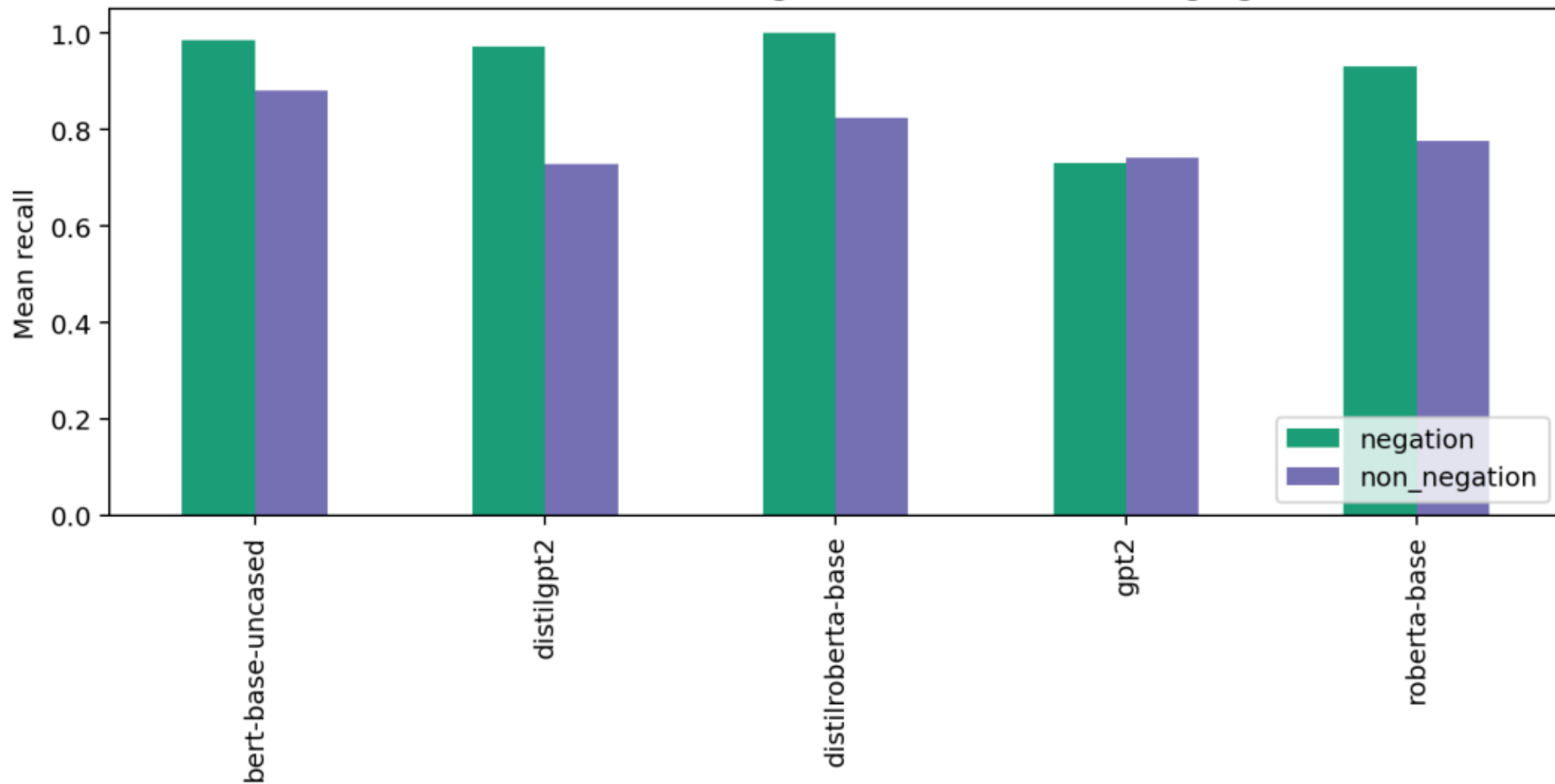
Confusion analysis: negation recall Linear SVC

Full-semantic recall: negation vs other classes (linear_svc)



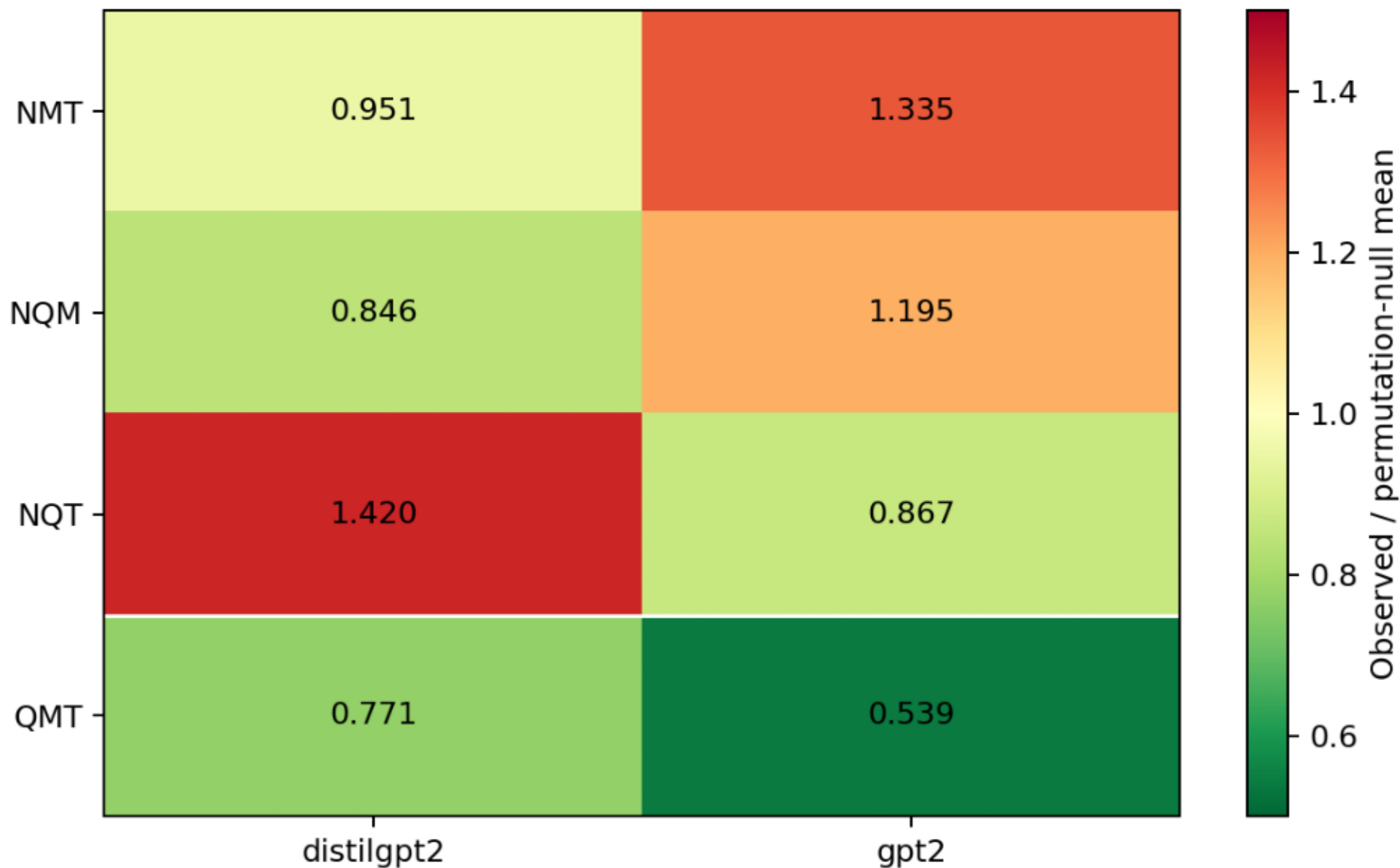
Confusion analysis: negation recall logistic regression

Full-semantic recall: negation vs other classes (logreg)

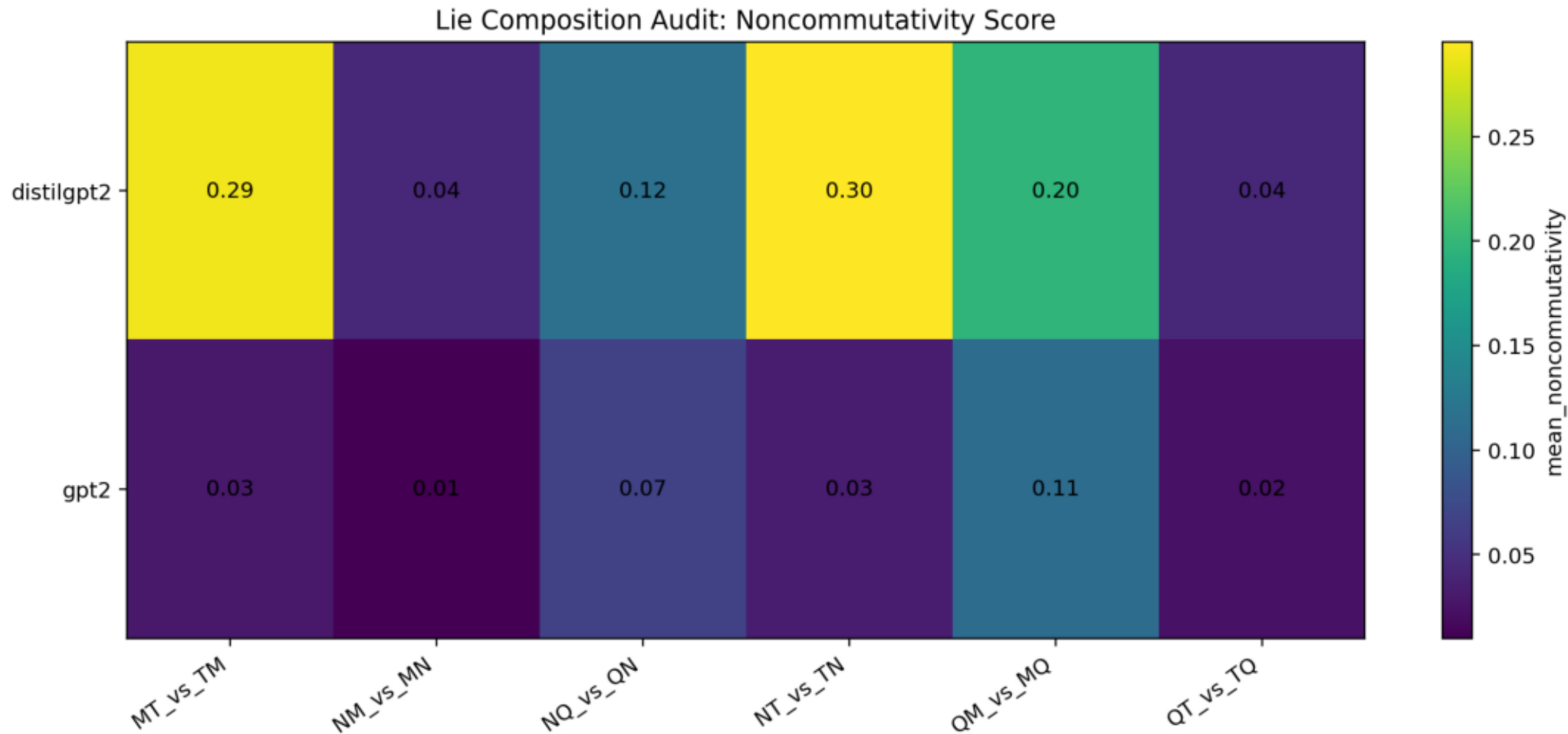


Decoder signed permutation replication

Decoder signed permutation ratio to null



Decoder pairwise composition



Audit notes for external verifier

Antisymmetry is not reported as evidence because the implementation defines [B,A] as the negative order of [A,B]. It is a tautological implementation check only.

A literal nested-commutator Jacobi expression over the same six endpoint vectors cancels algebraically. The reported third-order diagnostic is the non-tautological signed permutation composition sum $ABC+BCA+CAB-ACB-CBA-BAC$.

Duplicate endpoint templates were detected and removed before finalizing today's result. The final dataset has 400 Jacobi rows and zero duplicate endpoint sets.

New Track 1 result: syntax $y_{\text{only}}=1.0$ confirms target/surface leakage for the syntax split. New Track 1 spot-check: DeBERTa-v3-small supports delta superiority for Linear SVC but not for logistic regression. New Track 2 result: GPT-2 and DistilGPT-2 both keep QMT below permutation null, while negation triples remain mixed.

2026-06-13 update: after cache cleanup, BERT-large and DeBERTa-v3-base were both run successfully. Delta is best for both Linear SVC and logistic regression on both models.

Second 2026-06-13 update: UPAT is now explicitly reported as a hard-holdout boundary condition. Full-semantic pooling ablation, confusion/negation analysis, decoder pairwise composition, and signed-permutation multiple-testing correction are included.

2026-06-14 update: UPAT-large Procrustes null controls are scaled to $N=1000$ and now include random-label, random-pairing, and random-orthogonal baselines. The observed aligned F1 exceeds every null repeat across all non-identity directions.

Second 2026-06-14 update: held-out alignment-size controls are now added. Procrustes maps fitted on auxiliary anchor texts disjoint from classifier train/test texts recover most of the full-anchor transfer effect.

Third 2026-06-14 update: RISE is now the central related-work anchor. Track 3 is reframed as RISE-aware stress testing rather than first-discovery cross-model geometry. A RISE/MDV-style UPAT comparison is now a required gate.

Fourth 2026-06-14 update: first-pass RISE-aware UPAT comparison is now added. MDV and simplified RISE-style prototype methods predict targets well within-model, but cross-model target prediction remains harder than aligned delta-classifier transfer on the class-discrimination metric.

Fifth 2026-06-14 update: a non-leaky Hybrid RISE-Procrustes transfer test is now added. It scores every pair against all class prototypes, then tests prototype-score and delta+prototype-score features for cross-model transformation-label F1. The hybrid does not improve over aligned delta_only, suggesting that target-reconstructive prototype geometry and class-discriminative delta geometry are not automatically complementary under this metric.

Sixth 2026-06-14 update: movement-level spherical delta steering is now added. Linear centroid steering is best for cross-model target cosine, while RISE-style prediction is best for nearest-target label F1. Uncalibrated spherical delta movement slightly improves label F1 over linear deltas but lowers target cosine.

Seventh 2026-06-14 update: grammar-generated pairwise composition controls are now added for Track 2. Observed relative commutator norms remain below shuffled/norm-matched nulls, but endpoint-only and delta-only controls classify pair labels almost perfectly, so this is not yet endpoint-independent algebraic evidence.

2026-06-23 update: the multilingual max audit completed successfully on seven languages and five multilingual encoders: paraphrase-multilingual-mpnet-base-v2, LaBSE, multilingual-e5-large, BGE-M3, and mBERT. All four third-order triples are below signed-null in every model-language cell; the global mean ratios are NQM=0.5798, QMT=0.6203, NQT=0.7014, and NMT=0.7716. This strengthens the existence of a controlled signed-permutation signal while weakening any universal claim that QMT is uniquely special.

2026-06-23 caution: source-only held-out-language controls are chance-like, but endpoint/delta/commutator controls remain high. Cross-language centroid cosine is moderate and high-variance. The next Track 2 gate is endpoint-balanced multilingual generation plus third-order target-only controls, not another unqualified scale-up.

Remaining blockers: confidence intervals and anchor-domain diversity for Track 3, stronger/faithful RISE comparison if feasible, resolve or expand UPAT hard-holdout, representation-ablation tables for every non-syntax holdout split, endpoint-balanced grammar generation and third-order target-only controls for Track 2, composition layer/pooling ablations, and final bibliography formatting.

Round-3 Reviewer Response

Reviewer Response Round 3

Date: 2026-06-13

Postscript, 2026-06-23:

The later multilingual max audit revised the Track 2 narrative. The `QMT`-only claim below is historical and scoped to the original English encoder/decoder table. In the 7-language, 5-multilingual-encoder audit, all four tested triples are below signed-null in every model-language cell, with `NQM` and `QMT` strongest. Current drafts should use the multilingual framing rather than promote `QMT` as globally unique.

This note records the current response to the latest review of the research package.

Fixed in the Current Drafts

Antisymmetry

The Track 2 draft no longer presents antisymmetry as a discovery. It is now described as a tautological implementation check:

```
```text
[A,B] = delta_AB - delta_BA
[B,A] = delta_BA - delta_AB
```
```

Therefore `[A,B] = -[B,A]` follows for arbitrary vectors and does not provide evidence about transformer representations.

UPAT and y_only Confounding

The Track 1 draft now explicitly reports UPAT as a hard-holdout boundary result. UPAT sometimes favors `y\_only` over `delta`, and no UPAT `delta` vs `y\_only` McNemar comparison is significant. The claim is narrowed:

> Delta can add useful relational information, but endpoint-only features remain a serious confounder and can dominate under smaller, harder holdouts.

Layer-0 Syntax Result

The layer-0 syntax result is now framed as a red flag. Perfect separability at the embedding layer

indicates token/form cues rather than deep semantic geometry.

Synthetic Lie Templates

The Track 2 draft now explicitly states that the current Lie-style composition dataset is synthetic and contains stable lexical markers. The result is a controlled diagnostic, not a general natural-language operator algebra.

Mixed Signed-Permutation Results

The Track 2 draft now explains that several triples are below null in some models, while others are above null. `QMT` is promoted only because it is the only tested triple that is below null across all five models after table-level correction.

Still Required Before Submission

1. Add a Procrustes alignment null baseline for UPAT:
 - random-label or random-pairing alignment
 - report null distribution for F1 gain
2. Increase shuffle controls from 100 permutations to at least 1000 before reporting precise p-values.
3. Add null baselines for pairwise commutator norms `||[A,B]||`.
4. Replace hand-written Lie templates with grammar-generated paraphrase templates.
5. Run endpoint-only controls for composition endpoints.
6. Convert large/modern Track 1 spot-checks into multiseed runs if Track 1 is promoted to submission.

Current Submission Readiness

The package is now stronger as a research record, but it should not be submitted as a main-track paper yet. The best current framing is:

- Track 1: a representation-geometry study with clear endpoint confounders and a hard-holdout boundary.
- Track 2: a diagnostic study of local signed-permutation coherence, with `QMT` as the only cross-model stable triple and negation as a composition instability.

Updated Research Roadmap

Research Program: From Linguistic Transformation Vectors to Composition Diagnostics

Core question

Can linguistic transformations be represented not only as separable classes in transformer embedding spaces, but as reusable geometric operations with meaningful composition structure?

The project currently has two paper tracks.

Strategic narrative

The strongest long-term story is broader than either current draft:

> Transformers may encode linguistic operators as geometric objects in a low-dimensional, partially universal transformation subspace, and this subspace may be transferable across architectures and languages.

Current results are not enough to claim this as a complete theory yet. The Procrustes transfer numbers now survive large random-label, random-pairing, random-orthogonal, and held-out-anchor alignment controls, but RISE already provides a stronger published neighboring result for spherical/geodesic semantic-syntactic transformations across languages and embedding models. The roadmap below therefore treats cross-model transfer as a stress-test and comparison track, not as an unqualified novelty claim.

Closest prior-work anchor:

- Freenor and Alvarez 2026, RISE, already demonstrates rotor-based discourse-level semantic-syntactic transformation geometry across languages and embedding models. Our work must be positioned as endpoint-controlled delta diagnostics, Procrustes/null stress testing, and ordered-composition diagnostics rather than as the first evidence for geometric linguistic transformations.

Track 1: Geometric transformation vectors

Working title:

****Geometric Transformation Vectors in Transformer Embedding Spaces****

Central claim:

Transformer embedding spaces contain displacement directions that encode linguistic transformations. These directions are recoverable across multiple models and holdout regimes.

Current evidence:

- Delta vectors classify transformations well across BERT, RoBERTa, DistilRoBERTa, GPT-2, and DistilGPT-2.
- Syntax holdouts reach `1.0` accuracy across tested models, but the new representation ablation shows `y\_only=1.0` as well. This is now interpreted as endpoint/surface leakage, not as deep generalization.
- Full semantic holdouts remain above chance, with accuracies around `0.82-0.89`.
- Entity and variant holdouts remain strong, especially for BERT-family models.
- Modern/larger spot-checks now support the main delta advantage:
 - BERT-large Linear SVC: `delta=0.903`, `concat=0.851`, `y\_only=0.838`
 - BERT-large logistic regression: `delta=0.854`, `concat=0.760`, `y\_only=0.750`
 - DeBERTa-v3-base Linear SVC: `delta=0.812`, `concat=0.776`, `y\_only=0.747`
 - DeBERTa-v3-base logistic regression: `delta=0.752`, `concat=0.694`, `y\_only=0.726`
- UPAT hard-holdout is a boundary result: `delta` is worse than `y\_only` for BERT, RoBERTa, and GPT-2, and no UPAT delta-vs-y McNemar test is significant.
- Full-semantic pooling ablation now supports mean pooling as a defensible main choice, while showing that the decoder effect is partly pooling-dependent.

Scientific status:

This is the more mature paper. It can be written as a representation-geometry result with careful controls against template memorization.

Main risk:

Some signal may come from target-sentence artifacts rather than pure transformation geometry. The concrete high-risk case is now confirmed: `syntax=1.0` reproduces under `y\_only`, so it must be interpreted as a target/surface artifact unless future controls overturn that. UPAT also shows that `delta > y\_only` is not universal under small hard-holdout regimes. The paper needs strong source-only, target-only, delta-only, and paraphrase controls.

Track 2: Signed permutation coherence for linguistic operators

Working title:

Third-Order Signed Permutation Coherence for Linguistic Transformations in Transformer Embedding Spaces

Central claim:

Some linguistic transformations behave like locally noncommuting operations in embedding space. Certain triples show third-order signed permutation cancellation stronger than permutation-null

baselines, but the evidence supports local composition structure rather than a global Lie algebra.

Current evidence:

- Composition order matters: AB and BA often differ substantially.
- Semantic equivalence controls show lower noncommutativity for equivalent pairs than for non-equivalent pairs.
- Antisymmetry is not treated as evidence. It is a tautological implementation check because $[A,B] = \delta_{AB} - \delta_{BA}$ implies $[B,A] = -[A,B]$.
- The non-tautological third-order signed permutation diagnostic is:

text

$$S(A,B,C) = ABC + BCA + CAB - ACB - CBA - BAC$$

...

- The QMT triple shows robust cancellation across tested models:
 - BERT: ratio to permutation null 0.683 , CI $[0.677, 0.689]$
 - DistilRoBERTa: ratio 0.631 , CI $[0.627, 0.636]$
 - RoBERTa: ratio 0.623 , CI $[0.617, 0.629]$
- Decoder replication is now partially complete:
 - GPT-2 QMT : ratio 0.539 , CI $[0.519, 0.561]$
 - DistilGPT-2 QMT : ratio 0.771 , CI $[0.745, 0.797]$
- Negation-containing triples remain mixed and model-dependent.
- Decoder pairwise composition summaries are now added, not only decoder third-order summaries.
- In the original English encoder/decoder table, multiple-testing correction over 4 triples x 5 models supports the narrowed claim that QMT is the only below-null triple passing across all five tested models.
- A grammar-generated pairwise composition control now tests 720 generated N,Q,M,T rows across BERT, DistilRoBERTa, and RoBERTa.
- In that grammar-control probe, observed relative commutator norms remain below all $N=1000$ same-pair, any-pair, and norm-matched null means at the current $p=0.000999$ resolution floor.
- But endpoint-only and delta-only pair-label controls are essentially perfect ($\text{macro F1} \approx 1.0$), so this is not endpoint-independent algebraic evidence yet.
- A 2026-06-23 multilingual max audit now extends the signed-permutation probe to 7 languages and 5 multilingual encoders:
 - languages: English, Spanish, French, German, Russian, Chinese, Arabic
 - models: paraphrase-multilingual-mpnet-base-v2, LaBSE, multilingual-e5-large, BGE-M3, and mBERT
 - all four tested triples are below signed-null in all $35/35$ model-language cells
 - global mean ratios to signed-null: $NQM=0.580$, $QMT=0.620$, $NQT=0.701$, $NMT=0.772$
 - source-only held-out-language controls are chance-like ($\text{macro F1}=0.0476$), but endpoint/delta/commutator controls remain high, so endpoint artifacts are not solved
 - cross-language centroid consistency is moderate and high-variance ($\text{mean cosine} \approx 0.32$)

Scientific status:

This is promising but more fragile. It should be framed as a null-controlled diagnostic, not as proof of a Lie algebra or as a formal Jacobi identity.

Main risk:

Hand-written templates may induce or suppress cancellation. GPT-2/DistilGPT-2 support `QMT` below-null cancellation, and the grammar-generated pairwise control shows commutator coherence below nulls. The multilingual max audit strengthens the existence of a signed-permutation signal but also revises the story: `QMT` is not uniquely strongest once we move to multilingual templates and multilingual encoders. Endpoint controls remain too strong. The next experiments need endpoint-balanced multilingual generation, target-only third-order composition controls, and a focused explanation of why different template regimes change the relative ordering of `NQM` and `QMT`.

Track 3: Cross-model transformation transfer

Working title:

****Stress-Testing Cross-Model Transformation Transfer in Transformer Embedding Spaces****

Central hypothesis:

Transformation geometry may be partially model-independent after low-dimensional alignment. A classifier trained on transformation deltas in one model may transfer to another model after Procrustes alignment.

Promising exploratory evidence:

- BERT to `all-mpnet-base-v2`: raw F1 around `0.15`, aligned F1 around `0.857`.
- BERT to `all-MiniLM-L6-v2`: raw F1 around `0.17`, aligned F1 around `0.754`.
- UPAT-large cross-model results show large aligned-transfer gains across several architectures.
- A scaled null audit (`N=1000` repeats per direction and null type) found that every non-identity cross-model direction stayed above random-label, random-pairing, and random-orthogonal null baselines.
- Mean observed aligned F1 was `0.684651`.
- Mean null F1 was `0.173017` for random-label, `0.148036` for random-pairing, and `0.112103` for random-orthogonal.
- No null repeat reached observed aligned F1 in any of the `30 x 3` direction/null tests, so all empirical p-values are at the `N=1000` resolution floor: $1/(1000+1)=0.000999$.
- A held-out alignment-size curve now fits Procrustes maps on `1200` auxiliary anchor texts that are disjoint from the classifier train/test texts.
- Held-out anchor mean F1 rises from `0.452046` at `25` anchors to `0.661928` at `1000` anchors, compared with raw cross-model mean F1 `0.241524` and full-anchor Procrustes mean F1 `0.684651`.
- At `1000` held-out anchors, no direction falls below its raw cross-model baseline.

- A first RISE-aware UPAT comparison now evaluates ``mdv_raw``, ``mdv_unit``, and a simplified spherical ``rise_style`` prototype baseline. Within-model target prediction cosine is high (``rise_style=0.923008``), while cross-model target prediction is harder (``rise_style`` mean cosine ``0.578347``, nearest-target label F1 ``0.445518``) than delta-classifier transfer in aligned spaces (``0.684651`` mean F1).
- A non-leaky Hybrid RISE-Procrustes transfer test now scores each pair against every MDV/RISE-style class prototype and trains transformation-label classifiers on ``delta_only``, prototype scores, and ``delta + prototype scores``. The hybrid does not improve cross-model label F1: ``delta_only`` remains ``0.684651``, while the best hybrid/prototype variant is ``mdv_raw_hybrid_delta_scores`` at ``0.430839``.
- A movement-level spherical delta steering test now compares linear centroid steering, tangent/exp-map steering, RISE-style prediction, residual orderings, and hybrid averaging. Cross-model target cosine is best for ``linear_delta`` (``0.613559``), while retrieval label F1 is best for ``rise_only`` (``0.445518``); ``spherical_delta`` slightly improves label F1 over ``linear_delta`` (``0.412695`` vs ``0.407674``) but lowers target cosine (``0.604795`` vs ``0.613559``).

Current status:

This is now a serious candidate for a complementary paper, but not as a "first universal geometry" claim. RISE is stronger on cross-lingual/cross-model geometric transformation modeling. Our distinct angle is stress testing: null-controlled Procrustes transfer, held-out anchors, endpoint controls, and comparison against simpler delta/MDV/RISE-style baselines.

Required gates before promotion:

1. Add bootstrap confidence intervals and direction-family summaries for the held-out alignment and RISE-aware comparison curves.
2. Stress-test anchor-domain diversity, e.g. anchors from a different template family or natural paraphrase pool.
3. Improve the RISE-aware comparison:
 - verify the simplified ``rise_style`` baseline against the published RISE implementation if feasible
 - add confidence intervals
 - clarify target-prediction versus class-discrimination metrics
4. Extend movement-level composition with calibration:
 - learn per-class step sizes for spherical delta steering on train only
 - add bootstrap confidence intervals over directions and examples
 - compare residual orderings against a more faithful published RISE implementation if feasible
5. Reverse-direction transfer summary by model family, e.g. small model to large model and large model to small model.
6. Stronger architectures if feasible, such as Llama/Mistral-class embedding spaces or high-quality sentence encoders.
7. Package the result as a standalone Track 3 draft only after the RISE/MDV comparison and confidence-interval checks.

Working title:

From Transformation Geometry to Controllable Linguistic Editing

Central hypothesis:

If transformation directions are real, they should not only classify transformations; they should also steer generation or hidden states toward those transformations.

Minimal experiment:

1. Use GPT-2 as the first testbed.
2. Compute a centroid delta for a transformation such as negation or question formation.
3. Inject the vector into the residual stream during generation.
4. Measure whether outputs systematically acquire the target transformation.
5. Compare against random-vector, norm-matched, and wrong-class controls.

Scientific payoff:

This would move the project from descriptive geometry to causal intervention. A positive result would connect the work to representation steering and controllable generation.

Track 5: Cross-lingual transformation geometry

Working title:

Language-Invariant Geometry of Linguistic Transformations

Central hypothesis:

If transformation geometry is universal rather than English-template-specific, aligned multilingual models should show comparable transformation subspaces across languages.

Minimal experiment:

1. Use mBERT or XLM-R.
2. Build parallel transformation pairs in English and one or more non-English languages.
3. Compare delta geometry within each language.
4. Align language-specific transformation spaces with Procrustes.
5. Test whether classifiers or centroids transfer across languages.

The first large multilingual Track 2 audit is complete for 7 languages and 5 multilingual encoders. It is not yet a clean cross-lingual universality result, because the templates are synthetic and endpoint controls remain strong. Still, it is the strongest current evidence that the signed-permutation diagnostic is not purely an English-only artifact.

Scientific payoff:

This is the cleanest answer to the criticism that the current effects may be English grammar or template artifacts.

Track 6: Dimensionality of transformation manifolds

Working title:

****Effective Dimensionality of Linguistic Transformation Subspaces****

Central hypothesis:

Each transformation may occupy a low-dimensional subspace rather than a single direction or the full embedding space.

Measurements:

- PCA spectra of per-class delta matrices.
- Participation ratio of singular values.
- Classification accuracy as a function of retained dimensions.
- Sample-complexity curves for learning each transformation.

Scientific payoff:

This would quantify the complexity of each linguistic operator and explain why capacity curves keep improving with more examples.

Iterative logic

The research direction is:

1. Show transformations are encoded as recoverable displacement vectors.
2. Test whether those directions compose nontrivially.
3. Separate semantic order effects from wording artifacts.
4. Test local third-order signed permutation structure with null baselines.
5. Expand from hand-written probes to systematic grammar-generated probes.
6. Decide which claims are strong enough for publication.

Common methodology standards

Every promoted claim should have at least one control:

- holdout split
- source-only and target-only baseline
- permutation/null baseline
- semantic equivalence control
- bootstrap confidence interval
- cross-model replication
- template de-duplication check

Negative results remain part of the research record.

Next research plan

Short term

1. Close methodology blockers required for any submission:

- remove antisymmetry from the evidence narrative; already done in the draft, keep it that way
- keep Procrustes null baselines in the evidence packet; `N=1000` random-label/random-pairing/random-orthogonal controls are now complete
- increase shuffle/permutation controls to at least 1000, ideally 5000 for final numbers
- add commutator norm null baselines

2. Resolve UPAT:

- either expand UPAT and match train sizes against the main dataset
- or keep it explicitly as a hard negative control and narrow Track 1 claims

3. Extend UPAT alignment controls beyond the completed `N=1000` null audit and held-out alignment curve:

- bootstrap confidence intervals
- direction-family summary
- anchor-domain diversity check

4. Test cross-model transformation transfer as a RISE-aware stress-test paper:

- reverse-direction transfer
- alignment-size curve
- random-label/null alignment controls
- RISE/MDV-style prototype baseline

5. Treat the syntax holdout as resolved for the current draft: `y\_only=1.0` and layer-0 `1.0` mean the `syntax=1.0` result is a target/surface artifact unless a future redesigned split proves otherwise.

6. Convert large/modern Track 1 spot-checks into multiseed runs if Track 1 is promoted to submission.

7. Build a grammar-driven template generator for `N,Q,M,T` that produces many paraphrases without

duplicate endpoints.

8. Add automated dataset validation:

- no duplicate endpoint strings within a composition tuple
- balanced subjects/actions
- controlled lexical overlap

9. Add commutator null baselines for ``[[[A,B]]]` using random or label-shuffled operations with matched norms.

10. Re-run composition and signed-permutation diagnostics on generated templates.

11. Add target-only and endpoint-only baselines for third-order multilingual signed-permutation endpoints.

12. Build an endpoint-balanced multilingual generator and re-run the 7-language audit.

13. Explain the ``NQM`` versus ``QMT`` reversal between the English/decoder table and the multilingual max audit.

14. Regenerate a dated PDF packet after every major run.

Medium term

1. Run the GPT-2 steering-vector experiment for one transformation, starting with negation or question formation.

2. Run the cross-lingual mBERT/XLM-R transformation-transfer experiment.

3. Measure effective dimensionality of transformation subspaces:

- participation ratio
- PCA retention curves
- accuracy versus retained dimensions

4. Run layerwise and pooling ablations before expanding the operator set. If final-layer mean pooling is not the best representation, the expanded operator experiments should use the validated representation instead.

5. Add more linguistic operators:

- passive voice
- modality strength
- evidentiality
- aspect
- conditionality
- quantifier changes

6. Add paraphrase robustness with semantically equivalent endpoint variants.

Paper milestones

1. Track 1 is arXiv-ready only when this checklist is complete:

- syntax holdout breakdown is in the draft and interpreted as endpoint leakage
- McNemar tests are reported in the text
- multiseed standard deviations are reported
- at least one prior-work baseline or comparison is written up, such as task vectors or function

vectors

- at least one model outside the original five is included as a spot-check; currently satisfied

by BERT-large and DeBERTa-v3-base

- UPAT hard-holdout is either resolved experimentally or explicitly reported as a negative

boundary condition

2. Keep Track 2 as a diagnostics paper until grammar-generated templates and endpoint-only controls succeed.

3. Promote Track 3 only if it becomes clearly complementary to RISE: null-controlled Procrustes transfer plus held-out anchors plus an explicit RISE/MDV comparison. The main narrative should be stress-testing cross-model transfer, not claiming first discovery of universal transformation geometry.

4. If steering works, write Track 4 as an intervention/controllable-generation paper.

5. If cross-lingual transfer works, it becomes the strongest version of the universality claim.

6. If Track 2 survives grammar-generated controls, write it as a separate diagnostics paper rather than merging it into Track 1.

7. If Track 2 weakens under controls, keep it as a negative/diagnostic section in a broader research note.

RISE Related-Work Positioning

Related Work Positioning: RISE and Our Tracks

This note records how the project should position itself after reading:

Freenor and Alvarez, **"Mapping Semantic & Syntactic Relationships with Geometric Rotation"**, ICLR 2026.

What RISE Already Covers

RISE is the closest and strongest neighboring work. It treats discourse-level semantic-syntactic transformations as geometric operations in sentence-embedding space.

Key points from the paper:

- Method: Rotor-Invariant Shift Estimation (RISE), a spherical/Riemannian rotor method that maps neutral-to-transformed sentence pairs through tangent-space prototypes.
- Transformations: negation, conditionality, and politeness.
- Scope: seven languages and three multilingual embedding models (`text-embedding-3-large`, `bge-m3`, `mBERT`).
- Baselines: mean difference vectors and Procrustes alignment.
- Evaluation: cosine similarity between predicted transformed embeddings and target transformed embeddings on held-out sets, plus cross-language and cross-model transfer.
- Main implication for us: we cannot claim novelty for the broad statement that discourse-level semantic-syntactic transformations have cross-lingual/cross-model geometric structure.

What Our Work Should Not Claim

We should not write any of the following as novelty claims:

- "First evidence that linguistic transformations are geometric operations in sentence embeddings."
- "First cross-model transformation geometry result."
- "Universal transformation subspaces across embedding models" without qualification.
- "Procrustes alignment establishes universality" without comparing to RISE-like spherical methods.

Remaining Distinct Contributions

Our current repository still has several distinct contributions if framed narrowly.

Track 1: Endpoint-Controlled Delta Diagnostics

RISE focuses on predicting transformed embeddings via rotor/MDV-style operations. Track 1 asks a

different diagnostic question:

> Does `embedding(y) - embedding(x)` add class-discriminative transformation information beyond `x\_only`, `y\_only`, and `concat` baselines?

Distinct value:

- explicit endpoint leakage controls
- multiseed delta/y_only/concat/x_only ablations
- McNemar tests
- hard negative/boundary reporting via UPAT
- syntax `1.0` reinterpreted as endpoint/surface leakage rather than a success

This makes Track 1 a conservative control paper, not a stronger version of RISE.

Track 2: Ordered Composition Diagnostics

RISE reports that its transformations behave commutatively in the tangent-space framing. Track 2 asks almost the opposite question:

> Do ordered linguistic transformations show noncommutative or signed-permutation composition structure in embedding endpoints?

Distinct value:

- pairwise order diagnostics (`AB` versus `BA`)
- semantic-equivalence controls
- third-order signed-permutation diagnostic
- negation as a composition instability rather than simply a robust one-step transformation

This is the cleanest conceptual separation from RISE. It should be framed as a diagnostic of local composition/order effects, not as a steering method.

Track 3: Cross-Model Transfer Stress Tests

Track 3 is closest to RISE and therefore needs the most careful positioning.

Current distinct value:

- direct classifier transfer over transformation deltas
- `N=1000` random-label, random-pairing, and random-orthogonal Procrustes nulls
- held-out alignment-size curve using auxiliary anchor texts disjoint from classifier train/test texts
- explicit reporting of raw, full-anchor, and held-out-anchor transfer gaps

Now addressed as a first-pass comparison:

Track 3 now includes a first-pass RISE-inspired comparison on UPAT:

- MDV/prototype prediction baseline on UPAT
- spherical normalization / tangent-space variant if feasible
- comparison of classifier-transfer F1 versus target-embedding prediction similarity

Current interpretation:

- Prototype methods answer a target-embedding prediction question.
- Procrustes delta-classifier transfer answers a class-discriminative transfer question.
- Within-model prototype target cosine is high.
- Cross-model prototype prediction is harder and model-pair dependent.
- The simplified `rise\_style` baseline improves nearest-target label retrieval over MDV in the current UPAT setup, but it is not a full reproduction of RISE.
- A non-leaky hybrid test that combines aligned `delta\_only` features with all-class MDV/RISE-style prototype scores does not improve cross-model transformation-label F1. The current interpretation is that target-reconstructive geometry and class-discriminative geometry are not automatically complementary under this classifier-transfer metric.
- A movement-level spherical delta steering test gives a more nuanced split: linear centroid steering is best for cross-model target cosine, while RISE-style prediction is best for nearest-target label F1. Spherical/tangent steering does not improve target cosine in the current uncalibrated form, but it slightly improves label F1 over linear deltas.

Track 3 remains a useful stress-test package, but it still should not be promoted as a main novelty paper against RISE until confidence intervals, anchor-domain robustness, and a more faithful RISE implementation are added.

Recommended Framing

Use this framing in future drafts:

> RISE shows that some discourse-level semantic-syntactic transformations can be modeled as spherical/geodesic operations across languages and embedding models. Our work is complementary: we stress-test simpler endpoint and delta representations under endpoint-only controls, hard holdouts, Procrustes nulls, and held-out alignment anchors, and we separately test whether ordered linguistic transformations exhibit noncommutative composition diagnostics.

Immediate Backlog Changes

1. Add RISE to every related-work section as the closest prior work.
2. Stop using "universal transformation subspaces" as an unqualified Track 3 title.

3. Rename Track 3 working title toward "stress-testing cross-model transformation transfer".
4. Extend the first RISE/MDV comparison with confidence intervals and, if feasible, a more faithful RISE implementation before promoting Track 3.
5. Extend movement-level composition with train-only step-size calibration and confidence intervals before making any complementarity claim.
6. Emphasize Track 2 composition/order diagnostics as the most distinct paper-level contribution.

Critical Review: Current Research State

Critical Review: Current Research State

Date: 2026-06-14

This note is a deliberately skeptical review of the repository after the RISE-aware UPAT work, hybrid transfer test, and spherical delta steering experiment.

Executive Verdict

The repository is now a credible research artifact and a useful exploratory program. It is not yet a submission-ready single paper.

The strongest scientific value is not one grand claim. It is the combination of controlled positive results and openly reported boundary conditions:

- delta vectors can add information beyond endpoint-only baselines
- syntax-perfect results are endpoint/surface leakage, not a triumph
- QMT-like composition shows local signed-permutation coherence
- negation breaks or destabilizes the composition story
- cross-model Procrustes transfer survives strong nulls
- RISE-style target reconstruction and delta-classifier transfer answer different questions
- simple hybridization does not automatically improve the main metrics

This is not nonsense. The work contains several real, falsifiable empirical facts. The main danger is over-narrating them as a unified theory before the controls are complete.

What Is Actually Done

Track 1: Endpoint-Controlled Delta Representations

Status: most mature draft.

Main result:

- In the main full-semantic multiseed setting, `delta` beats `y\_only` and `concat` across the original tested model set.
- McNemar tests support the result.
- BERT-large and DeBERTa-v3-base spot-checks support the delta advantage.
- Pooling/layerwise checks reduce the risk that the mean-pooling choice alone creates the effect.

Critical caveats:

- `y\_only` is still strong.
- syntax `1.0` is confirmed as target/surface leakage.
- UPAT hard-holdout does not show a reliable $\Delta > y_only$ advantage.
- The main multiseed result still needs confidence intervals and broader holdout ablation tables.

Best paper framing:

> Delta vectors add reproducible relational information beyond endpoint-only baselines in controlled synthetic transformation probes, but they are not pure linguistic operators.

Track 2: Signed-Permutation / Composition Diagnostics

Status: interesting and distinct, but needs stronger data generation.

Main result:

- Pairwise composition is order-sensitive.
- Semantic-equivalence controls show that the diagnostic is not just arbitrary wording noise.
- The third-order result is correctly reframed as signed-permutation coherence, not a Jacobi identity.
- `QMT` is the strongest local result across encoder and decoder checks.
- Grammar-generated pairwise composition controls now preserve below-null commutator coherence, but endpoint-only controls remain near perfect.

Critical caveats:

- Hand-written templates may contain stable lexical markers.
- Grammar-generated endpoints still contain explicit order/pair markers.
- Negation-containing triples are unstable.
- Endpoint-only controls for composition are still needed.
- Pairwise commutator norms need null baselines before being promoted.

Best paper framing:

> Some controlled ordered transformations exhibit local composition-sensitive geometry; the result is not a global Lie algebra.

Track 3: RISE-Aware Cross-Model Transfer Stress Tests

Status: promising but most exposed to related-work criticism.

Main result:

- Cross-model Procrustes transfer survives `N=1000` random-label, random-pairing, and random-orthogonal nulls.
- Held-out alignment anchors recover most of the full-anchor transfer effect.
- RISE/MDV-style target prediction is strong within-model but harder cross-model.
- Non-leaky hybrid prototype-score features do not beat aligned `delta\_only` for transformation-label F1.
- Spherical delta steering separates target cosine from retrieval label F1:
 - `linear\_delta` is best for target cosine
 - `rise\_only` is best for transformation-neighborhood retrieval
 - uncalibrated spherical delta movement slightly improves label F1 but lowers target cosine

Critical caveats:

- RISE is the closest and stronger neighboring paper for broad cross-model/cross-lingual transformation geometry.
- The current `rise\_style` implementation is only a simplified baseline, not a faithful reproduction.
- Anchor-domain robustness is not yet tested.
- Confidence intervals over directions and examples are missing.
- Step-size calibration for spherical movement is not done.

Best paper framing:

> A stress-test comparison showing that cross-model class-discriminative transfer, target reconstruction, and spherical movement are separable geometric questions.

What Is Not Ready

1. A unified "theory of linguistic operators" is not ready.
2. A claim of first discovery of geometric linguistic transformations is not valid because RISE already occupies that territory.
3. A strong natural-language generalization claim is not ready because the datasets are synthetic controlled templates.
4. A steering/generation claim is not ready because all current steering is embedding-space target prediction, not causal intervention in generation.

Current Research Directions

Direction A: Finish Track 1 as a Conservative Baseline Paper

This is the lowest-risk publication path.

Required:

- confidence intervals for `delta - y\_only` and `delta - concat`
- remaining holdout representation-ablation tables
- clean final bibliography
- self-review against endpoint leakage objections

Direction B: Make Track 2 a Composition Diagnostics Paper

This is the most distinct intellectual contribution.

Required:

- endpoint-balanced grammar generation for `N,Q,M,T`
- target-only controls for third-order composition endpoints
- stronger pairwise commutator nulls beyond the current random-pairing/norm-matched controls
- layer/pooling ablation

Direction C: Develop Track 3 as a RISE-Aware Stress-Test Paper

This is exciting but must be very carefully positioned.

Required:

- bootstrap confidence intervals
- direction-family summaries
- anchor-domain robustness
- train-only step-size calibration for spherical delta steering
- more faithful RISE implementation or explicit statement that the current baseline is simplified

Direction D: Move Toward Causal Editing

This is future work, not current evidence.

Required:

- residual-stream injection in a decoder model
- random-vector and wrong-class controls
- generation-level metrics
- careful separation from embedding retrieval experiments

Repository Hygiene Review

Strengths:

- Scripts and results are now mostly discoverable through `scripts/README.md` and

`results/README.md`.

- Main research state is documented in `paper/research\_program.md`, `paper/reviewer\_revision\_plan.md`, and `research/diary.md`.
- The Zenodo snapshot is ethically framed as software/research artifact, not a peer-reviewed paper.
- The repo includes code, CSVs, figures, requirements, citation metadata, and reports.

Weaknesses:

- The repository is large and still contains many historical exploratory result folders.
- Some old scripts are not clearly labeled as historical versus current.
- The main report date lagged behind the live research date; future reports should use the actual current date.
- Track 3 does not yet have its own article draft, even though it now has enough material for a structured outline.

Immediate cleanup recommendations:

- Add a Track 3 article folder only after calibration/CI, or add a placeholder outline clearly marked "not draft-ready."
- Add a `CURRENT\_RESULTS.md` or keep this critical review as the compact state summary.
- Keep old exploratory folders, but avoid citing them from current papers unless they are part of the active evidence chain.
- Treat `reports/2026-06-13\_reviewer\_revised\_report.pdf` as historical and build new live packets under the actual date.

Bottom Line

This is a real research program, not AI-flavored noise. The results are interesting because they are not uniformly positive. They expose a structured split:

- delta vectors help classification beyond endpoints in some controlled settings
- endpoint leakage is real and sometimes dominant
- composition structure exists locally but fails globally
- cross-model alignment is surprisingly robust under nulls
- RISE-like geometry and delta geometry optimize different notions of success

The next best move is not adding more broad claims. It is adding calibration and confidence intervals to Track 3, while keeping Track 1 and Track 2 clean enough to become separate papers.

2026-06-23 Update: Multilingual max audit

The new multilingual audit is a genuine strengthening of Track 2, but it also forces a narrative correction.

What got stronger:

- The signed-permutation diagnostic now survives a 7-language, 5-multilingual-encoder stress test.
- All four tested triples are below signed-null in every model-language cell.
- Source-only held-out-language controls are chance-like, so the result is not simply recoverable from source sentences.

What got weaker:

- The previous `QMT`-as-unique story is no longer globally valid. `QMT` remains the unique stable triple in the older English encoder/decoder table, but `NQM` is strongest in the multilingual audit and every triple is below null.
- Endpoint/delta/commutator controls remain strong, so the result is still not endpoint-independent algebraic evidence.
- Cross-language centroid consistency is moderate with high variance, so the multilingual result should not be sold as a clean universal operator space.

Updated skeptical verdict:

- > Controlled multilingual sentence encoders show robust third-order signed-permutation cancellation under synthetic `N,Q,M,T` templates, but endpoint controls and regime-dependent triple rankings prevent a Lie-algebra claim.

The next decisive experiment is endpoint-balanced multilingual generation with third-order target-only controls.

Code listing: run_lie_algebraic_identities.py (1)

```
0001: import gc
0002: import json
0003: import os
0004: import warnings
0005: from pathlib import Path
0006:
0007: import numpy as np
0008: import pandas as pd
0009: import torch
0010: import matplotlib.pyplot as plt
0011:
0012: from transformers import AutoTokenizer, AutoModel
0013: from sklearn.decomposition import PCA
0014: from sklearn.metrics.pairwise import cosine_similarity
0015:
0016: warnings.filterwarnings("ignore")
0017:
0018:
0019: class LieAlgebraicIdentitiesAudit:
0020:     def __init__(self):
0021:         self.out_dir = Path(os.getenv("LIE_ALGEBRA_OUT_DIR", "results/experiments/lie_algebraic_identities_results"))
0022:         self.csv_dir = self.out_dir / "csv"
0023:         self.fig_dir = self.out_dir / "figures"
0024:         self.csv_dir.mkdir(parents=True, exist_ok=True)
0025:         self.fig_dir.mkdir(parents=True, exist_ok=True)
0026:
0027:         self.device = "cuda" if torch.cuda.is_available() else "cpu"
0028:         self.hf_token = None
0029:         self.pca_dim = 64
0030:         self.n_jacobi_null = 1000
0031:         self.n_bootstrap = 2000
0032:
0033:         self.models = [
0034:             model.strip()
0035:             for model in os.getenv(
0036:                 "LIE_ALGEBRA_MODELS",
0037:                 "bert-base-uncased,distilroberta-base,roberta-base",
0038:             ).split(",")
0039:             if model.strip()
0040:         ]
0041:
0042:         self.ops = ["N", "Q", "M", "T"]
0043:
0044:         self.subjects = [
0045:             "scientist", "engineer", "teacher", "doctor", "programmer",
0046:             "researcher", "analyst", "manager", "wizard", "dragon",
0047:             "queen", "robot", "pirate", "oracle", "knight", "alien",
0048:         ]
0049:
0050:         self.actions = [
0051:             ("accepted", "accept", "the explanation"),
0052:             ("completed", "complete", "the repair"),
0053:             ("confirmed", "confirm", "the answer"),
0054:             ("approved", "approve", "the treatment"),
```

Code listing: run_lie_algebraic_identities.py (2)

```
0055:         ("fixed", "fix", "the bug"),
0056:         ("supported", "support", "the theory"),
0057:         ("verified", "verify", "the report"),
0058:         ("guarded", "guard", "the treasure"),
0059:         ("opened", "open", "the portal"),
0060:         ("signed", "sign", "the treaty"),
0061:     ]
0062:
0063:     def log(self, msg):
0064:         print(msg, flush=True)
0065:
0066:     def base_sentence(self, subject, past, obj):
0067:         return f"The {subject} {past} {obj}."
0068:
0069:     def compose(self, seq, subject, past, base, obj):
0070:         s = "".join(seq)
0071:
0072:         mapping = {
0073:             "": f"The {subject} {past} {obj}.",
0074:
0075:             "N": f"The {subject} failed to {base} {obj}.",
0076:             "Q": f"Could the {subject} {base} {obj}?",
0077:             "M": f"The {subject} allegedly {past} {obj}.",
0078:             "T": f"The {subject} will {base} {obj} tomorrow.",
0079:
0080:             "NQ": f"Could the {subject} fail to {base} {obj}?",
0081:             "QN": f"Is it false that the {subject} {past} {obj}?",
0082:
0083:             "NM": f"The {subject} allegedly failed to {base} {obj}.",
0084:             "MN": f"Allegedly, the {subject} failed to {base} {obj}.",
0085:
0086:             "NT": f"The {subject} will fail to {base} {obj} tomorrow.",
0087:             "TN": f"The {subject} failed to have {past} {obj} earlier.",
0088:
0089:             "QM": f"Could the {subject} allegedly {base} {obj}?",
0090:             "MQ": f"Is it alleged that the {subject} {past} {obj}?",
0091:
0092:             "QT": f"Could the {subject} {base} {obj} tomorrow?",
0093:             "TQ": f"Will the {subject} {base} {obj} tomorrow?",
0094:
0095:             "MT": f"The {subject} will allegedly {base} {obj} tomorrow.",
0096:             "TM": f"The {subject} was allegedly going to {base} {obj}.",
0097:         }
0098:
0099:         if s in mapping:
0100:             return mapping[s]
0101:
0102:         # canonical triple templates
0103:         if s == "NQM":
0104:             return f"Could the {subject} allegedly fail to {base} {obj}?"
0105:         if s == "QMN":
0106:             return f"Is it false that the {subject} allegedly {past} {obj}?"
0107:         if s == "MNQ":
0108:             return f"Is it alleged that the {subject} failed to {base} {obj}?"
```

Code listing: run_lie_algebraic_identities.py (3)

```
0109:
0110:     if s == "QNM":
0111:         return f"Could it be false that the {subject} allegedly {past} {obj}?"
0112:     if s == "NMQ":
0113:         return f"Could it be alleged that the {subject} failed to {base} {obj}?"
0114:     if s == "MQN":
0115:         return f"Allegedly, is it false that the {subject} {past} {obj}?"
0116:
0117:     if s == "NMT":
0118:         return f"The {subject} will allegedly fail to {base} {obj} tomorrow."
0119:     if s == "MTN":
0120:         return f"The {subject} was allegedly going to fail to {base} {obj}."
0121:     if s == "TNM":
0122:         return f"Earlier, the {subject} allegedly failed to have {past} {obj}."
0123:
0124:     if s == "NTM":
0125:         return f"Allegedly, the {subject} will fail to {base} {obj} tomorrow."
0126:     if s == "TMN":
0127:         return f"Earlier, it was alleged that the {subject} failed to have {past} {obj}."
0128:     if s == "MNT":
0129:         return f"It is alleged that the {subject} will fail to {base} {obj} tomorrow."
0130:
0131:     if s == "NQT":
0132:         return f"Could the {subject} fail to {base} {obj} tomorrow?"
0133:     if s == "QTN":
0134:         return f"Is it false that the {subject} will {base} {obj} tomorrow?"
0135:     if s == "TNQ":
0136:         return f"Could the {subject} have failed to {base} {obj} earlier?"
0137:
0138:     if s == "QNT":
0139:         return f"Could it be false that the {subject} will {base} {obj} tomorrow?"
0140:     if s == "NTQ":
0141:         return f"Could it be that the {subject} will fail to {base} {obj} tomorrow?"
0142:     if s == "TQN":
0143:         return f"Will it be false that the {subject} {base} {obj} tomorrow?"
0144:
0145:     if s == "QMT":
0146:         return f"Could the {subject} allegedly {base} {obj} tomorrow?"
0147:     if s == "MTQ":
0148:         return f"Could it be alleged that the {subject} will {base} {obj} tomorrow?"
0149:     if s == "TQM":
0150:         return f"Allegedly, will the {subject} {base} {obj} tomorrow?"
0151:
0152:     if s == "MQT":
0153:         return f"Will it be alleged that the {subject} {base} {obj} tomorrow?"
0154:     if s == "QTM":
0155:         return f"Allegedly, could the {subject} {base} {obj} tomorrow?"
0156:     if s == "TMQ":
0157:         return f"Could the {subject} have allegedly planned to {base} {obj}?"
0158:
0159:     raise ValueError(f"Unsupported composition: {s}")
0160:
0161: def build_dataset(self, n_templates=100, seed=42):
0162:     rng = np.random.default_rng(seed)
```

Code listing: run_lie_algebraic_identities.py (4)

```
0163:         combos = [(s, a) for s in self.subjects for a in self.actions]
0164:         rng.shuffle(combos)
0165:         combos = combos[:n_templates]
0166:
0167:         rows = []
0168:
0169:         pairs = [("N", "Q"), ("N", "M"), ("N", "T"), ("Q", "M"), ("Q", "T"), ("M", "T")]
0170:         triples = [("N", "Q", "M"), ("N", "Q", "T"), ("N", "M", "T"), ("Q", "M", "T")]
0171:
0172:         for i, (subject, action) in enumerate(combos):
0173:             past, base, obj = action
0174:             source = self.base_sentence(subject, past, obj)
0175:
0176:             for a, b in pairs:
0177:                 rows.append({
0178:                     "template_id": i,
0179:                     "kind": "antisymmetry",
0180:                     "source": source,
0181:                     "a": a,
0182:                     "b": b,
0183:                     "c": "",
0184:                     "ab": self.compose([a, b], subject, past, base, obj),
0185:                     "ba": self.compose([b, a], subject, past, base, obj),
0186:                 })
0187:
0188:             for a, b, c in triples:
0189:                 rows.append({
0190:                     "template_id": i,
0191:                     "kind": "jacobi",
0192:                     "source": source,
0193:                     "a": a,
0194:                     "b": b,
0195:                     "c": c,
0196:                     "abc": self.compose([a, b, c], subject, past, base, obj),
0197:                     "bca": self.compose([b, c, a], subject, past, base, obj),
0198:                     "cab": self.compose([c, a, b], subject, past, base, obj),
0199:                     "acb": self.compose([a, c, b], subject, past, base, obj),
0200:                     "cba": self.compose([c, b, a], subject, past, base, obj),
0201:                     "bac": self.compose([b, a, c], subject, past, base, obj),
0202:                 })
0203:
0204:         df = pd.DataFrame(rows)
0205:         df.to_csv(self.csv_dir / "lie_algebraic_identities_dataset.csv", index=False)
0206:         return df
0207:
0208:     @torch.no_grad()
0209:     def get_embeddings(self, model_name, texts, batch_size=16):
0210:         tokenizer = AutoTokenizer.from_pretrained(model_name, token=self.hf_token)
0211:
0212:         if tokenizer.pad_token is None:
0213:             tokenizer.pad_token = tokenizer.eos_token or tokenizer.unk_token or "[PAD]"
0214:
0215:         model = AutoModel.from_pretrained(model_name, token=self.hf_token).to(self.device)
0216:         model.eval()
```

Code listing: run_lie_algebraic_identities.py (5)

```
0217:
0218:         embs = []
0219:
0220:         for i in range(0, len(texts), batch_size):
0221:             batch = texts[i:i + batch_size]
0222:
0223:             enc = tokenizer(
0224:                 batch,
0225:                 padding=True,
0226:                 truncation=True,
0227:                 max_length=128,
0228:                 return_tensors="pt",
0229:             ).to(self.device)
0230:
0231:             out = model(**enc)
0232:             hidden = out.last_hidden_state
0233:             mask = enc["attention_mask"].unsqueeze(-1)
0234:
0235:             pooled = (hidden * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1)
0236:             embs.append(pooled.detach().cpu().numpy())
0237:
0238:         del model
0239:         gc.collect()
0240:
0241:         if self.device == "cuda":
0242:             torch.cuda.empty_cache()
0243:
0244:         return np.vstack(embs)
0245:
0246:     def cos(self, x, y):
0247:         return float(cosine_similarity(x.reshape(1, -1), y.reshape(1, -1))[0, 0])
0248:
0249:     def alternating_null_stats(self, vectors, observed_norm, scale, rng):
0250:         null_relative = []
0251:         n = len(vectors)
0252:
0253:         for _ in range(self.n_jacobi_null):
0254:             positive = set(rng.choice(n, size=n // 2, replace=False).tolist())
0255:             signed_sum = np.zeros_like(vectors[0])
0256:
0257:             for i, vector in enumerate(vectors):
0258:                 signed_sum += vector if i in positive else -vector
0259:
0260:             null_relative.append(np.linalg.norm(signed_sum) / scale)
0261:
0262:         null_relative = np.asarray(null_relative, dtype=float)
0263:         observed_relative = observed_norm / scale
0264:
0265:         return {
0266:             "null_mean_relative_jacobi_norm": float(null_relative.mean()),
0267:             "null_std_relative_jacobi_norm": float(null_relative.std(ddof=1)),
0268:             "jacobi_to_null_mean_ratio": float(observed_relative / (null_relative.mean() + 1e-12)),
0269:             "null_percentile_smaller_is_better": float((null_relative <= observed_relative).mean()),
0270:         }
```

Code listing: run_lie_algebraic_identities.py (6)

```
0271:
0272:     def bootstrap_mean_ci(self, values, rng, alpha=0.05):
0273:         values = np.asarray(values, dtype=float)
0274:         if len(values) == 0:
0275:             return np.nan, np.nan
0276:
0277:         draws = rng.choice(values, size=(self.n_bootstrap, len(values)), replace=True).mean(axis=1)
0278:         return (
0279:             float(np.quantile(draws, alpha / 2)),
0280:             float(np.quantile(draws, 1 - alpha / 2)),
0281:         )
0282:
0283:     def compute_for_model(self, model_name, df):
0284:         self.log("\n\n" + "=" * 80)
0285:         self.log(f"MODEL: {model_name}")
0286:         self.log("=" * 80)
0287:
0288:         text_cols = ["source", "ab", "ba", "abc", "bca", "cab", "acb", "cba", "bac"]
0289:         all_texts = set()
0290:
0291:         for col in text_cols:
0292:             if col in df.columns:
0293:                 all_texts |= set(df[col].dropna().values)
0294:
0295:         all_texts = sorted(all_texts)
0296:         idx = {t: i for i, t in enumerate(all_texts)}
0297:
0298:         raw = self.get_embeddings(model_name, all_texts)
0299:
0300:         dim = min(self.pca_dim, raw.shape[0] - 1, raw.shape[1])
0301:         pca = PCA(n_components=dim, random_state=42)
0302:         vecs = pca.fit_transform(raw)
0303:
0304:         anti_rows = []
0305:         jacobi_rows = []
0306:
0307:         anti_df = df[df["kind"] == "antisymmetry"]
0308:
0309:         for row in anti_df.itertuples():
0310:             x = vecs[idx[row.source]]
0311:             ab = vecs[idx[row.ab]]
0312:             ba = vecs[idx[row.ba]]
0313:
0314:             delta_ab = ab - x
0315:             delta_ba = ba - x
0316:
0317:             comm_ab = delta_ab - delta_ba
0318:             comm_ba = delta_ba - delta_ab
0319:
0320:             antisym_error = np.linalg.norm(comm_ab + comm_ba)
0321:             comm_norm = np.linalg.norm(comm_ab)
0322:
0323:             anti_rows.append({
0324:                 "model": model_name,
```

Code listing: run_lie_algebraic_identities.py (7)

```
0325:         "template_id": row.template_id,
0326:         "pair": f"{row.a}{row.b}_vs_{row.b}{row.a}",
0327:         "a": row.a,
0328:         "b": row.b,
0329:         "comm_norm": float(comm_norm),
0330:         "antisym_error": float(antisym_error),
0331:         "relative_antisym_error": float(antisym_error / (comm_norm + 1e-12)),
0332:         "cos_comm_ab_neg_comm_ba": self.cos(comm_ab, -comm_ba),
0333:     })
0334:
0335: jacobi_df = df[df["kind"] == "jacobi"]
0336: seed = sum((i + 1) * ord(ch) for i, ch in enumerate(model_name)) % (2 ** 32)
0337: rng = np.random.default_rng(seed)
0338:
0339: for row in jacobi_df.itertuples():
0340:     x = vecs[idx[row.source]]
0341:
0342:     abc = vecs[idx[row.abc]] - x
0343:     bca = vecs[idx[row.bca]] - x
0344:     cab = vecs[idx[row.cab]] - x
0345:
0346:     acb = vecs[idx[row.acb]] - x
0347:     cba = vecs[idx[row.cba]] - x
0348:     bac = vecs[idx[row.bac]] - x
0349:
0350:     # Jacobi-like alternating sum over permutations:
0351:     # J = ABC + BCA + CAB - ACB - CBA - BAC
0352:     jacobi = abc + bca + cab - acb - cba - bac
0353:
0354:     positive_norm = np.linalg.norm(abc) + np.linalg.norm(bca) + np.linalg.norm(cab)
0355:     negative_norm = np.linalg.norm(acb) + np.linalg.norm(cba) + np.linalg.norm(bac)
0356:     scale = 0.5 * (positive_norm + negative_norm) + 1e-12
0357:
0358:     jacobi_norm = np.linalg.norm(jacobi)
0359:     null_stats = self.alternating_null_stats(
0360:         [abc, bca, cab, acb, cba, bac],
0361:         jacobi_norm,
0362:         scale,
0363:         rng,
0364:     )
0365:
0366:     jacobi_rows.append({
0367:         "model": model_name,
0368:         "template_id": row.template_id,
0369:         "triple": f"{row.a}{row.b}{row.c}",
0370:         "a": row.a,
0371:         "b": row.b,
0372:         "c": row.c,
0373:         "jacobi_norm": float(jacobi_norm),
0374:         "relative_jacobi_norm": float(jacobi_norm / scale),
0375:         **null_stats,
0376:     })
0377:
0378: anti_result = pd.DataFrame(anti_rows)
```

Code listing: run_lie_algebraic_identities.py (8)

```
0379:         jacobi_result = pd.DataFrame(jacobi_rows)
0380:
0381:         anti_result.to_csv(self.csv_dir / f"antisymmetry_raw_{self.safe_name(model_name)}.csv", index=False)
0382:         jacobi_result.to_csv(self.csv_dir / f"jacobi_raw_{self.safe_name(model_name)}.csv", index=False)
0383:
0384:         return anti_result, jacobi_result
0385:
0386:     def safe_name(self, model_name):
0387:         return model_name.replace("/", "_").replace("-", "_")
0388:
0389:     def summarize(self, anti_all, jacobi_all):
0390:         anti_summary = (
0391:             anti_all.groupby(["model", "pair"])
0392:             .agg(
0393:                 mean_comm_norm=("comm_norm", "mean"),
0394:                 mean_relative_antisym_error=("relative_antisym_error", "mean"),
0395:                 mean_cos_comm_ab_neg_comm_ba=("cos_comm_ab_neg_comm_ba", "mean"),
0396:                 n=("pair", "count"),
0397:             )
0398:             .reset_index()
0399:         )
0400:
0401:         jacobi_summary = (
0402:             jacobi_all.groupby(["model", "triple"])
0403:             .agg(
0404:                 mean_jacobi_norm=("jacobi_norm", "mean"),
0405:                 mean_relative_jacobi_norm=("relative_jacobi_norm", "mean"),
0406:                 mean_null_relative_jacobi_norm=("null_mean_relative_jacobi_norm", "mean"),
0407:                 mean_jacobi_to_null_mean_ratio=("jacobi_to_null_mean_ratio", "mean"),
0408:                 mean_null_percentile_smaller_is_better=("null_percentile_smaller_is_better", "mean"),
0409:                 n=("triple", "count"),
0410:             )
0411:             .reset_index()
0412:         )
0413:
0414:         rng = np.random.default_rng(20260611)
0415:         ci_rows = []
0416:
0417:         for row in jacobi_summary.itertuples():
0418:             group = jacobi_all[
0419:                 (jacobi_all["model"] == row.model)
0420:                 & (jacobi_all["triple"] == row.triple)
0421:             ]
0422:
0423:             rel_low, rel_high = self.bootstrap_mean_ci(group["relative_jacobi_norm"].values, rng)
0424:             ratio_low, ratio_high = self.bootstrap_mean_ci(group["jacobi_to_null_mean_ratio"].values, rng)
0425:
0426:             ci_rows.append({
0427:                 "model": row.model,
0428:                 "triple": row.triple,
0429:                 "relative_jacobi_norm_ci95_low": rel_low,
0430:                 "relative_jacobi_norm_ci95_high": rel_high,
0431:                 "jacobi_to_null_mean_ratio_ci95_low": ratio_low,
0432:                 "jacobi_to_null_mean_ratio_ci95_high": ratio_high,
```


Code listing: run_lie_algebraic_identities.py (9)

```
0433:         })
0434:
0435:         jacobi_summary = jacobi_summary.merge(pd.DataFrame(ci_rows), on=["model", "triple"], how="left")
0436:
0437:         anti_summary.to_csv(self.csv_dir / "antisymmetry_summary.csv", index=False)
0438:         jacobi_summary.to_csv(self.csv_dir / "jacobi_summary.csv", index=False)
0439:
0440:         return anti_summary, jacobi_summary
0441:
0442:     def plot_heatmap(self, df, index_col, value_col, filename, title):
0443:         rows = sorted(df["model"].unique())
0444:         cols = sorted(df[index_col].unique())
0445:
0446:         mat = np.zeros((len(rows), len(cols)))
0447:
0448:         for i, model in enumerate(rows):
0449:             for j, col in enumerate(cols):
0450:                 val = df[(df["model"] == model) & (df[index_col] == col)][value_col].values
0451:                 mat[i, j] = val[0] if len(val) else np.nan
0452:
0453:         plt.figure(figsize=(10, 5))
0454:         plt.imshow(mat, aspect="auto")
0455:         plt.colorbar(label=value_col)
0456:
0457:         plt.xticks(np.arange(len(cols)), cols, rotation=35, ha="right")
0458:         plt.yticks(np.arange(len(rows)), rows)
0459:         plt.title(title)
0460:
0461:         for i in range(mat.shape[0]):
0462:             for j in range(mat.shape[1]):
0463:                 plt.text(j, i, f"{mat[i, j]:.2f}", ha="center", va="center")
0464:
0465:         path = self.fig_dir / filename
0466:         plt.tight_layout()
0467:         plt.savefig(path, dpi=220, bbox_inches="tight")
0468:         plt.close()
0469:         self.log(f"Saved figure: {path}")
0470:
0471:     def run(self):
0472:         self.log(f"DEVICE: {self.device}")
0473:         self.log(f"OUT DIR: {self.out_dir}")
0474:
0475:         df = self.build_dataset(n_templates=100, seed=42)
0476:
0477:         anti_results = []
0478:         jacobi_results = []
0479:
0480:         for model in self.models:
0481:             anti, jacobi = self.compute_for_model(model, df)
0482:             anti_results.append(anti)
0483:             jacobi_results.append(jacobi)
0484:
0485:         anti_all = pd.concat(anti_results, ignore_index=True)
0486:         jacobi_all = pd.concat(jacobi_results, ignore_index=True)
```

Code listing: run_lie_algebraic_identities.py (10)

```
0487:
0488:     anti_all.to_csv(self.csv_dir / "antisymmetry_raw_all_models.csv", index=False)
0489:     jacobi_all.to_csv(self.csv_dir / "jacobi_raw_all_models.csv", index=False)
0490:
0491:     anti_summary, jacobi_summary = self.summarize(anti_all, jacobi_all)
0492:
0493:     self.plot_heatmap(
0494:         anti_summary,
0495:         "pair",
0496:         "mean_cos_comm_ab_neg_comm_ba",
0497:         "01_antisymmetry_cosine_heatmap.png",
0498:         "Antisymmetry Check:  $\cos([A,B], -[B,A])$ ",
0499:     )
0500:
0501:     self.plot_heatmap(
0502:         anti_summary,
0503:         "pair",
0504:         "mean_relative_antisym_error",
0505:         "02_antisymmetry_error_heatmap.png",
0506:         "Antisymmetry Error",
0507:     )
0508:
0509:     self.plot_heatmap(
0510:         jacobi_summary,
0511:         "triple",
0512:         "mean_relative_jacobi_norm",
0513:         "03_jacobi_relative_norm_heatmap.png",
0514:         "Jacobi-like Alternating Sum Relative Norm",
0515:     )
0516:
0517:     self.plot_heatmap(
0518:         jacobi_summary,
0519:         "triple",
0520:         "mean_jacobi_to_null_mean_ratio",
0521:         "04_jacobi_vs_permutation_null_heatmap.png",
0522:         "Jacobi-like Norm / Permutation Null Mean",
0523:     )
0524:
0525:     metadata = {
0526:         "antisymmetry": "checks whether  $[A,B] \neq -[B,A]$ ",
0527:         "jacobi_like": "checks alternating sum  $ABC+BCA+CAB-ACB-CBA-BAC$ ",
0528:         "jacobi_null": (
0529:             "compares the observed alternating sum with random 3-positive/3-negative "
0530:             "sign assignments over the same six composition vectors; lower ratio and "
0531:             "lower percentile mean stronger Jacobi-like cancellation"
0532:         ),
0533:         "caution": (
0534:             "A literal nested-commutator Jacobi expansion over the same six embedded "
0535:             "composition endpoints cancels by construction, so this script reports a "
0536:             "non-tautological alternating third-order cancellation diagnostic instead."
0537:         ),
0538:         "note": "This is an empirical Lie-style diagnostic, not a formal proof of Lie algebra.",
0539:         "models": self.models,
0540:         "n_jacobi_null": self.n_jacobi_null,
```

Code listing: run_lie_algebraic_identities.py (11)

```
0541:         "n_bootstrap": self.n_bootstrap,
0542:         "jacobi_triples": ["NQM", "NQT", "NMT", "QMT"],
0543:     }
0544:
0545:     with open(self.out_dir / "metadata.json", "w", encoding="utf-8") as f:
0546:         json.dump(metadata, f, indent=2)
0547:
0548:     self.log("\n\nANTISYMMETRY SUMMARY")
0549:     self.log(str(anti_summary))
0550:
0551:     self.log("\n\nJACOBI SUMMARY")
0552:     self.log(str(jacobi_summary))
0553:
0554:     self.log("\n\nDONE")
0555:
0556:
0557: if __name__ == "__main__":
0558:     audit = LieAlgebraicIdentitiesAudit()
0559:     audit.run()
```

Code listing: run_lie_composition_grammar_controls.py (1)

```
0001: import gc
0002: import json
0003: import os
0004: import warnings
0005: from pathlib import Path
0006:
0007: import matplotlib.pyplot as plt
0008: import numpy as np
0009: import pandas as pd
0010: import torch
0011: from scipy.stats import percentileofscore
0012: from sklearn.decomposition import PCA
0013: from sklearn.linear_model import LogisticRegression
0014: from sklearn.metrics import accuracy_score, f1_score
0015: from sklearn.metrics.pairwise import cosine_similarity
0016: from sklearn.preprocessing import normalize
0017: from transformers import AutoModel, AutoTokenizer
0018:
0019: warnings.filterwarnings("ignore")
0020:
0021:
0022: class GrammarCompositionDataset:
0023:     SUBJECTS = [
0024:         "scientist", "engineer", "teacher", "doctor", "programmer",
0025:         "researcher", "analyst", "manager", "architect", "designer",
0026:         "editor", "planner", "auditor", "coordinator", "operator", "consultant",
0027:     ]
0028:
0029:     ACTIONS = [
0030:         ("accepted", "accept", "the explanation"),
0031:         ("completed", "complete", "the repair"),
0032:         ("confirmed", "confirm", "the answer"),
0033:         ("approved", "approve", "the treatment"),
0034:         ("fixed", "fix", "the bug"),
0035:         ("supported", "support", "the theory"),
0036:         ("verified", "verify", "the report"),
0037:         ("reviewed", "review", "the proposal"),
0038:         ("updated", "update", "the schedule"),
0039:         ("submitted", "submit", "the request"),
0040:     ]
0041:
0042:     CONTEXTS = [
0043:         "during the meeting",
0044:         "after the review",
0045:         "before the deadline",
0046:         "in the morning",
0047:         "at the office",
0048:         "for the committee",
0049:         "under the policy",
0050:         "with the team",
0051:     ]
0052:
0053:     NEG_FORMS = ["failed to {base}", "did not {base}", "refused to {base}"]
0054:     MOD_FORMS = ["allegedly", "reportedly", "apparently"]
```

Code listing: run_lie_composition_grammar_controls.py (2)

```
0055:     FUTURE_FORMS = ["will {base}", "is going to {base}", "plans to {base}"]
0056:     PAIRS = [{"N", "Q"}, ("N", "M"), ("N", "T"), ("Q", "M"), ("Q", "T"), ("M", "T")]
0057:
0058:     def __init__(self, n_templates=120, seed=20260614):
0059:         self.n_templates = n_templates
0060:         self.rng = np.random.default_rng(seed)
0061:
0062:     def sample_form(self):
0063:         return {
0064:             "neg": self.rng.choice(self.NEG_FORMS),
0065:             "mod": self.rng.choice(self.MOD_FORMS),
0066:             "future": self.rng.choice(self.FUTURE_FORMS),
0067:             "question": int(self.rng.integers(0, 3)),
0068:             "context": self.rng.choice(self.CONTEXTS),
0069:         }
0070:
0071:     def base_sentence(self, subject, past, obj, context):
0072:         return f"The {subject} {past} {obj} {context}."
0073:
0074:     def phrase(self, ops, subject, past, base, obj, form):
0075:         ops = list(ops)
0076:         context = form["context"]
0077:
0078:         negated = "N" in ops
0079:         modal = "M" in ops
0080:         future = "T" in ops
0081:         question = "Q" in ops
0082:
0083:         if future:
0084:             vp = form["future"].format(base=base)
0085:             vp_past = f"was going to {base}"
0086:         elif negated:
0087:             vp = form["neg"].format(base=base)
0088:             vp_past = f"failed to {base}"
0089:         else:
0090:             vp = base
0091:             vp_past = past
0092:
0093:         if negated and future:
0094:             if ops.index("N") < ops.index("T"):
0095:                 vp = f"will fail to {base}"
0096:                 vp_past = f"would fail to {base}"
0097:             else:
0098:                 vp = f"will not {base}"
0099:                 vp_past = f"would not {base}"
0100:
0101:         if modal:
0102:             marker = form["mod"]
0103:             if ops.index("M") == 0:
0104:                 vp = f"{marker} {vp}"
0105:                 vp_past = f"{marker} {vp_past}"
0106:             elif negated and not future:
0107:                 vp = f"did not {marker} {base}"
0108:                 vp_past = f"did not {marker} {base}"
```

Code listing: run_lie_composition_grammar_controls.py (3)

```
0109:         elif future:
0110:             vp = f"will {marker} {base}"
0111:             vp_past = f"was going to {marker} {base}"
0112:         else:
0113:             vp = f"{marker} {vp}"
0114:             vp_past = f"{marker} {vp_past}"
0115:
0116:     if question:
0117:         q_before_neg = negated and ops.index("Q") < ops.index("N")
0118:         q_before_mod = modal and ops.index("Q") < ops.index("M")
0119:         q_before_future = future and ops.index("Q") < ops.index("T")
0120:
0121:         if q_before_neg:
0122:             positive_vp = form["future"].format(base=base) if future else past
0123:             if modal:
0124:                 positive_vp = f"{form['mod']} {positive_vp}"
0125:             return f"Is it false that the {subject} {positive_vp} {obj} {context}?"
0126:
0127:         if q_before_mod:
0128:             positive_vp = form["future"].format(base=base) if future else (f"failed to {base}" if negated else past)
0129:             return f"Is it {form['mod']} true that the {subject} {positive_vp} {obj} {context}?"
0130:
0131:         if q_before_future:
0132:             positive_vp = f"fail to {base}" if negated else form["future"].format(base=base)
0133:             if modal:
0134:                 positive_vp = f"{form['mod']} {positive_vp}"
0135:             return f"Will it be true that the {subject} {positive_vp} {obj} {context}?"
0136:
0137:         if future:
0138:             q_vp = f"fail to {base}" if negated else base
0139:             if modal:
0140:                 q_vp = f"{form['mod']} {q_vp}"
0141:             return f"Will the {subject} {q_vp} {obj} {context}?"
0142:
0143:         if negated:
0144:             q_vp = f"fail to {base}"
0145:             if modal:
0146:                 q_vp = f"{form['mod']} {q_vp}"
0147:             return f"Did the {subject} {q_vp} {obj} {context}?"
0148:
0149:         if modal:
0150:             return f"Did the {subject} {form['mod']} {base} {obj} {context}?"
0151:
0152:         return f"Did the {subject} {base} {obj} {context}?"
0153:
0154:     sentence = f"The {subject} {vp} {obj} {context}."
0155:     return " ".join(sentence.split())
0156:
0157: def build(self):
0158:     combos = [(s, a) for s in self.SUBJECTS for a in self.ACTIONS]
0159:     self.rng.shuffle(combos)
0160:     combos = combos[: self.n_templates]
0161:
0162:     rows = []
```

Code listing: run_lie_composition_grammar_controls.py (4)

```
0163:         for template_id, (subject, action) in enumerate(combos):
0164:             past, base, obj = action
0165:             form = self.sample_form()
0166:             source = self.base_sentence(subject, past, obj, form["context"])
0167:
0168:             for a, b in self.PAIRS:
0169:                 ab = self.phrase([a, b], subject, past, base, obj, form)
0170:                 ba = self.phrase([b, a], subject, past, base, obj, form)
0171:                 rows.append({
0172:                     "template_id": template_id,
0173:                     "subject": subject,
0174:                     "action_base": base,
0175:                     "object": obj,
0176:                     "context": form["context"],
0177:                     "op_a": a,
0178:                     "op_b": b,
0179:                     "pair": f"{a}{b}_vs_{b}{a}",
0180:                     "source": source,
0181:                     "ab_text": ab,
0182:                     "ba_text": ba,
0183:                 })
0184:
0185:             return pd.DataFrame(rows)
0186:
0187:
0188: class GrammarCompositionAudit:
0189:     def __init__(self):
0190:         self.out_dir = Path(os.getenv(
0191:             "LIE_GRAMMAR_COMPOSITION_OUT_DIR",
0192:             "results/experiments/lie_composition_grammar_results",
0193:         ))
0194:         self.csv_dir = self.out_dir / "csv"
0195:         self.fig_dir = self.out_dir / "figures"
0196:         self.csv_dir.mkdir(parents=True, exist_ok=True)
0197:         self.fig_dir.mkdir(parents=True, exist_ok=True)
0198:
0199:         self.device = "cuda" if torch.cuda.is_available() else "cpu"
0200:         self.hf_token = None
0201:         self.pca_dim = 64
0202:         self.seed = 20260614
0203:         self.n_null = int(os.getenv("LIE_GRAMMAR_COMPOSITION_N_NULL", "1000"))
0204:         self.models = [
0205:             model.strip()
0206:             for model in os.getenv(
0207:                 "LIE_GRAMMAR_COMPOSITION_MODELS",
0208:                 "bert-base-uncased,distilroberta-base,roberta-base",
0209:             ).split(",")
0210:             if model.strip()
0211:         ]
0212:         self.rng = np.random.default_rng(self.seed)
0213:
0214:     def log(self, msg):
0215:         print(msg, flush=True)
0216:
```

Code listing: run_lie_composition_grammar_controls.py (5)

```
0217:     def safe_name(self, model_name):
0218:         return model_name.replace("/", "_").replace("-", "_")
0219:
0220:     @torch.no_grad()
0221:     def get_embeddings(self, model_name, texts, batch_size=16):
0222:         tokenizer = AutoTokenizer.from_pretrained(model_name, token=self.hf_token)
0223:         if tokenizer.pad_token is None:
0224:             tokenizer.pad_token = tokenizer.eos_token or tokenizer.unk_token or "[PAD]"
0225:
0226:         model = AutoModel.from_pretrained(model_name, token=self.hf_token).to(self.device)
0227:         model.eval()
0228:
0229:         embeddings = []
0230:         for i in range(0, len(texts), batch_size):
0231:             batch = texts[i:i + batch_size]
0232:             enc = tokenizer(
0233:                 batch,
0234:                 padding=True,
0235:                 truncation=True,
0236:                 max_length=128,
0237:                 return_tensors="pt",
0238:             ).to(self.device)
0239:             out = model(**enc)
0240:             hidden = out.last_hidden_state
0241:             mask = enc["attention_mask"].unsqueeze(-1)
0242:             pooled = (hidden * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1)
0243:             embeddings.append(pooled.detach().cpu().numpy())
0244:
0245:         del model
0246:         gc.collect()
0247:         if self.device == "cuda":
0248:             torch.cuda.empty_cache()
0249:
0250:         return np.vstack(embeddings)
0251:
0252:     def build_dataset(self):
0253:         df = GrammarCompositionDataset().build()
0254:         df.to_csv(self.csv_dir / "grammar_composition_dataset.csv", index=False)
0255:         self.log("\nDATASET")
0256:         self.log(str(df.groupby("pair").size()))
0257:         self.log(f"Total rows: {len(df)}")
0258:         return df
0259:
0260:     def compute_model_vectors(self, model_name, df):
0261:         self.log("\n" + "=" * 80)
0262:         self.log(f"MODEL: {model_name}")
0263:         self.log("=" * 80)
0264:
0265:         texts = sorted(set(df["source"]) | set(df["ab_text"]) | set(df["ba_text"]))
0266:         idx = {text: i for i, text in enumerate(texts)}
0267:         raw = self.get_embeddings(model_name, texts)
0268:
0269:         dim = min(self.pca_dim, raw.shape[0] - 1, raw.shape[1])
0270:         vecs = PCA(n_components=dim, random_state=42).fit_transform(raw)
```


Code listing: run_lie_composition_grammar_controls.py (6)

```
0271:
0272:     rows = []
0273:     features = {
0274:         "source": [],
0275:         "ab_endpoint": [],
0276:         "ba_endpoint": [],
0277:         "ab_delta": [],
0278:         "ba_delta": [],
0279:         "commutator": [],
0280:         "pair": [],
0281:         "template_id": [],
0282:     }
0283:
0284:     for row in df.itertuples(index=False):
0285:         source = vecs[idx[row.source]]
0286:         ab = vecs[idx[row.ab_text]]
0287:         ba = vecs[idx[row.ba_text]]
0288:         delta_ab = ab - source
0289:         delta_ba = ba - source
0290:         comm = delta_ab - delta_ba
0291:
0292:         norm_ab = float(np.linalg.norm(delta_ab))
0293:         norm_ba = float(np.linalg.norm(delta_ba))
0294:         comm_norm = float(np.linalg.norm(comm))
0295:         scale = 0.5 * (norm_ab + norm_ba) + 1e-12
0296:         cosine = float(cosine_similarity(delta_ab.reshape(1, -1), delta_ba.reshape(1, -1))[0, 0])
0297:
0298:         rows.append({
0299:             "model": model_name,
0300:             "template_id": row.template_id,
0301:             "pair": row.pair,
0302:             "op_a": row.op_a,
0303:             "op_b": row.op_b,
0304:             "source": row.source,
0305:             "ab_text": row.ab_text,
0306:             "ba_text": row.ba_text,
0307:             "norm_ab": norm_ab,
0308:             "norm_ba": norm_ba,
0309:             "commutator_norm": comm_norm,
0310:             "relative_commutator_norm": comm_norm / scale,
0311:             "cosine_ab_ba": cosine,
0312:             "noncommutativity_score": 1.0 - cosine,
0313:         })
0314:
0315:         features["source"].append(source)
0316:         features["ab_endpoint"].append(ab)
0317:         features["ba_endpoint"].append(ba)
0318:         features["ab_delta"].append(delta_ab)
0319:         features["ba_delta"].append(delta_ba)
0320:         features["commutator"].append(comm)
0321:         features["pair"].append(row.pair)
0322:         features["template_id"].append(row.template_id)
0323:
0324:     result = pd.DataFrame(rows)
```

Code listing: run_lie_composition_grammar_controls.py (7)

```
0325:         for key in ["source", "ab_endpoint", "ba_endpoint", "ab_delta", "ba_delta", "commutator"]:
0326:             features[key] = np.vstack(features[key])
0327:         features["pair"] = np.array(features["pair"])
0328:         features["template_id"] = np.array(features["template_id"])
0329:         return result, features
0330:
0331:     def endpoint_controls(self, model_name, features):
0332:         pairs = sorted(set(features["pair"]))
0333:         pair_to_id = {pair: i for i, pair in enumerate(pairs)}
0334:         y = np.array([pair_to_id[pair] for pair in features["pair"]])
0335:         template_ids = features["template_id"]
0336:
0337:         train_mask = template_ids < np.median(template_ids)
0338:         test_mask = ~train_mask
0339:
0340:         feature_sets = {
0341:             "source_only": features["source"],
0342:             "ab_endpoint_only": features["ab_endpoint"],
0343:             "ba_endpoint_only": features["ba_endpoint"],
0344:             "ab_delta_only": features["ab_delta"],
0345:             "ba_delta_only": features["ba_delta"],
0346:             "commutator_delta": features["commutator"],
0347:             "endpoint_concat": np.hstack([features["ab_endpoint"], features["ba_endpoint"]]),
0348:         }
0349:
0350:         rows = []
0351:         for name, x in feature_sets.items():
0352:             clf = LogisticRegression(max_iter=5000, C=1.0, random_state=42)
0353:             clf.fit(normalize(x[train_mask]), y[train_mask])
0354:             pred = clf.predict(normalize(x[test_mask]))
0355:             rows.append({
0356:                 "model": model_name,
0357:                 "feature": name,
0358:                 "accuracy": accuracy_score(y[test_mask], pred),
0359:                 "macro_f1": f1_score(y[test_mask], pred, average="macro"),
0360:                 "n_train": int(train_mask.sum()),
0361:                 "n_test": int(test_mask.sum()),
0362:                 "chance": 1.0 / len(pairs),
0363:             })
0364:
0365:         return pd.DataFrame(rows)
0366:
0367:     def commutator_nulls_from_features(self, model_name, features):
0368:         rows = []
0369:         for pair in sorted(set(features["pair"])):
0370:             mask = features["pair"] == pair
0371:             delta_ab = features["ab_delta"][mask]
0372:             delta_ba = features["ba_delta"][mask]
0373:             norm_ab = np.linalg.norm(delta_ab, axis=1)
0374:             norm_ba = np.linalg.norm(delta_ba, axis=1)
0375:             observed = np.linalg.norm(delta_ab - delta_ba, axis=1) / (0.5 * (norm_ab + norm_ba) + 1e-12)
0376:             observed_mean = float(observed.mean())
0377:
0378:             null_specs = {
```

Code listing: run_lie_composition_grammar_controls.py (8)

```
0379:         "random_pairing_same_pair": lambda: (delta_ab, delta_ba[self.rng.permutation(len(delta_ba))]),
0380:         "random_pairing_any_pair": lambda: (
0381:             features["ab_delta"][self.rng.choice(len(features["ab_delta"]), size=len(delta_ab), replace=False)],
0382:             features["ba_delta"][self.rng.choice(len(features["ba_delta"]), size=len(delta_ba), replace=False)],
0383:         ),
0384:         "random_direction_norm_matched": lambda: self.random_direction_vectors(norm_ab, norm_ba),
0385:     }
0386:
0387: for null_type, make_null in null_specs.items():
0388:     null_means = []
0389:     for _ in range(self.n_null):
0390:         random_ab, random_ba = make_null()
0391:         random_norm_ab = np.linalg.norm(random_ab, axis=1)
0392:         random_norm_ba = np.linalg.norm(random_ba, axis=1)
0393:         random_comm = np.linalg.norm(random_ab - random_ba, axis=1)
0394:         random_scale = 0.5 * (random_norm_ab + random_norm_ba) + 1e-12
0395:         null_means.append(float(np.mean(random_comm / random_scale)))
0396:
0397:     null = np.array(null_means)
0398:     rows.append({
0399:         "model": model_name,
0400:         "pair": pair,
0401:         "null_type": null_type,
0402:         "observed_mean_relative_commutator_norm": observed_mean,
0403:         "null_mean": float(null.mean()),
0404:         "null_std": float(null.std(ddof=1)),
0405:         "null_p05": float(np.percentile(null, 5)),
0406:         "null_p95": float(np.percentile(null, 95)),
0407:         "observed_percentile_smaller_is_more_coherent": float(percentileofscore(null, observed_mean, kind="mean")),
0408:         "p_smaller_or_equal_null": float(((null <= observed_mean).sum() + 1) / (len(null) + 1)),
0409:         "p_larger_or_equal_null": float(((null >= observed_mean).sum() + 1) / (len(null) + 1)),
0410:         "n_null": len(null),
0411:     })
0412:
0413: return pd.DataFrame(rows)
0414:
0415: def random_direction_vectors(self, norm_ab, norm_ba):
0416:     dim = self.pca_dim
0417:     u = self.rng.normal(size=(len(norm_ab), dim))
0418:     v = self.rng.normal(size=(len(norm_ba), dim))
0419:     u = normalize(u)
0420:     v = normalize(v)
0421:     return u * norm_ab[:, None], v * norm_ba[:, None]
0422:
0423: def summarize(self, raw_df):
0424:     return (
0425:         raw_df
0426:         .groupby(["model", "pair"])
0427:         .agg(
0428:             mean_relative_commutator_norm=("relative_commutator_norm", "mean"),
0429:             std_relative_commutator_norm=("relative_commutator_norm", "std"),
0430:             mean_noncommutativity=("noncommutativity_score", "mean"),
0431:             std_noncommutativity=("noncommutativity_score", "std"),
0432:             mean_cosine=("cosine_ab_ba", "mean"),
```

Code listing: run_lie_composition_grammar_controls.py (9)

```
0433:         n=("pair", "count"),
0434:     )
0435:     .reset_index()
0436: )
0437:
0438: def plot_summary(self, summary):
0439:     pairs = sorted(summary["pair"].unique())
0440:     models = sorted(summary["model"].unique())
0441:     mat = np.zeros((len(models), len(pairs)))
0442:     for i, model in enumerate(models):
0443:         for j, pair in enumerate(pairs):
0444:             val = summary[(summary["model"] == model) & (summary["pair"] == pair)]["mean_relative_commutator_norm"]
0445:             mat[i, j] = val.iloc[0] if len(val) else np.nan
0446:
0447:     plt.figure(figsize=(11, 5))
0448:     plt.imshow(mat, aspect="auto")
0449:     plt.colorbar(label="Mean relative commutator norm")
0450:     plt.xticks(np.arange(len(pairs)), pairs, rotation=35, ha="right")
0451:     plt.yticks(np.arange(len(models)), models)
0452:     plt.title("Grammar-generated composition: relative commutator norm")
0453:     for i in range(mat.shape[0]):
0454:         for j in range(mat.shape[1]):
0455:             plt.text(j, i, f"{mat[i, j]:.2f}", ha="center", va="center")
0456:     plt.tight_layout()
0457:     plt.savefig(self.fig_dir / "01_relative_commutator_norm_heatmap.png", dpi=220, bbox_inches="tight")
0458:     plt.close()
0459:
0460: def run(self):
0461:     self.log(f"DEVICE: {self.device}")
0462:     self.log(f"OUT DIR: {self.out_dir}")
0463:     df = self.build_dataset()
0464:
0465:     raw_parts = []
0466:     control_parts = []
0467:     null_parts = []
0468:     for model_name in self.models:
0469:         raw_model, features = self.compute_model_vectors(model_name, df)
0470:         raw_model.to_csv(self.csv_dir / f"grammar_composition_raw_{self.safe_name(model_name)}.csv", index=False)
0471:         raw_parts.append(raw_model)
0472:         control_parts.append(self.endpoint_controls(model_name, features))
0473:         null_parts.append(self.commutator_nulls_from_features(model_name, features))
0474:
0475:     raw_df = pd.concat(raw_parts, ignore_index=True)
0476:     controls = pd.concat(control_parts, ignore_index=True)
0477:     summary = self.summarize(raw_df)
0478:     nulls = pd.concat(null_parts, ignore_index=True)
0479:
0480:     raw_df.to_csv(self.csv_dir / "grammar_composition_raw_all_models.csv", index=False)
0481:     controls.to_csv(self.csv_dir / "grammar_endpoint_controls.csv", index=False)
0482:     summary.to_csv(self.csv_dir / "grammar_composition_summary.csv", index=False)
0483:     nulls.to_csv(self.csv_dir / "grammar_commutator_nulls.csv", index=False)
0484:     self.plot_summary(summary)
0485:
0486:     metadata = {
```

Code listing: run_lie_composition_grammar_controls.py (10)

```
0487:         "models": self.models,
0488:         "pca_dim": self.pca_dim,
0489:         "n_null": self.n_null,
0490:         "note": "Grammar-generated Track 2 control: endpoint controls plus random-direction commutator norm nulls.",
0491:     }
0492:     (self.out_dir / "metadata.json").write_text(json.dumps(metadata, indent=2), encoding="utf-8")
0493:
0494:     self.log("\nDONE")
0495:     self.log(f"Results saved to: {self.out_dir}")
0496:
0497:
0498: if __name__ == "__main__":
0499:     GrammarCompositionAudit().run()
```

Code listing: run_lie_multilingual_max_audit.py (1)

```
0001: import gc
0002: import json
0003: import math
0004: import os
0005: import sys
0006: import time
0007: import traceback
0008: from dataclasses import dataclass
0009: from pathlib import Path
0010:
0011: import matplotlib.pyplot as plt
0012: import numpy as np
0013: import pandas as pd
0014: import torch
0015: from scipy.stats import percentileofscore
0016: from sklearn.decomposition import PCA
0017: from sklearn.linear_model import LogisticRegression
0018: from sklearn.metrics import accuracy_score, f1_score
0019: from sklearn.preprocessing import normalize
0020: from transformers import AutoModel, AutoTokenizer
0021:
0022:
0023: OUT_DIR = Path("results/experiments/lie_multilingual_max_results")
0024: CSV_DIR = OUT_DIR / "csv"
0025: FIG_DIR = OUT_DIR / "figures"
0026: CHECKPOINT_DIR = OUT_DIR / "checkpoints"
0027: for directory in [CSV_DIR, FIG_DIR, CHECKPOINT_DIR]:
0028:     directory.mkdir(parents=True, exist_ok=True)
0029:
0030:
0031: PAIR_OPS = [("N", "Q"), ("N", "M"), ("N", "T"), ("Q", "M"), ("Q", "T"), ("M", "T")]
0032: TRIPLE_OPS = [("N", "Q", "M"), ("N", "Q", "T"), ("N", "M", "T"), ("Q", "M", "T")]
0033: CYCLIC = [(0, 1, 2), (1, 2, 0), (2, 0, 1)]
0034: ANTICYCLIC = [(0, 2, 1), (2, 1, 0), (1, 0, 2)]
0035:
0036:
0037: MODELS = [
0038:     "sentence-transformers/paraphrase-multilingual-mpnet-base-v2",
0039:     "sentence-transformers/LaBSE",
0040:     "intfloat/multilingual-e5-large",
0041:     "BAAI/bge-m3",
0042:     "bert-base-multilingual-cased",
0043: ]
0044:
0045:
0046: @dataclass
0047: class LanguageSpec:
0048:     code: str
0049:     subjects: list[str]
0050:     actions: list[tuple[str, str, str]]
0051:     contexts: list[str]
0052:     source: str
0053:     neg: list[str]
0054:     mod: list[str]
```

Code listing: run_lie_multilingual_max_audit.py (2)

```
0055:     future: list[str]
0056:     statement: str
0057:     question: str
0058:     false_question: str
0059:     modal_question: str
0060:     future_truth_question: str
0061:     future_question: str
0062:     did_fail_question: str
0063:     modal_did_question: str
0064:
0065:
0066: LANGS = [
0067:     LanguageSpec(
0068:         code="en",
0069:         subjects=["scientist", "engineer", "teacher", "doctor", "programmer", "researcher", "analyst", "manager"],
0070:         actions=[
0071:             ("accepted", "accept", "the explanation"),
0072:             ("completed", "complete", "the repair"),
0073:             ("confirmed", "confirm", "the answer"),
0074:             ("approved", "approve", "the treatment"),
0075:             ("fixed", "fix", "the bug"),
0076:             ("supported", "support", "the theory"),
0077:         ],
0078:         contexts=["during the meeting", "after the review", "before the deadline", "in the morning"],
0079:         source="The {subject} {past} {obj} {context}.",
0080:         neg=["failed to {base}", "did not {base}", "refused to {base}"],
0081:         mod=["allegedly", "reportedly", "apparently"],
0082:         future=["will {base}", "is going to {base}", "plans to {base}"],
0083:         statement="The {subject} {vp} {obj} {context}.",
0084:         question="Did the {subject} {base} {obj} {context}?",
0085:         false_question="Is it false that the {subject} {vp} {obj} {context}?",
0086:         modal_question="Is it {marker} true that the {subject} {vp} {obj} {context}?",
0087:         future_truth_question="Will it be true that the {subject} {vp} {obj} {context}?",
0088:         future_question="Will the {subject} {vp} {obj} {context}?",
0089:         did_fail_question="Did the {subject} fail to {base} {obj} {context}?",
0090:         modal_did_question="Did the {subject} {marker} {base} {obj} {context}?",
0091:     ),
0092:     LanguageSpec(
0093:         code="es",
0094:         subjects=["cientifico", "ingeniero", "profesor", "medico", "programador", "investigador", "analista", "gerente"],
0095:         actions=[
0096:             ("acepto", "aceptar", "la explicacion"),
0097:             ("completo", "completar", "la reparacion"),
0098:             ("confirmo", "confirmar", "la respuesta"),
0099:             ("aprobo", "aprobar", "el tratamiento"),
0100:             ("arreglo", "arreglar", "el error"),
0101:             ("apoyo", "apoyar", "la teoria"),
0102:         ],
0103:         contexts=["durante la reunion", "despues de la revision", "antes del plazo", "por la manana"],
0104:         source="El {subject} {past} {obj} {context}.",
0105:         neg=["no logro {base}", "no quiso {base}", "se nego a {base}"],
0106:         mod=["supuestamente", "al parecer", "segun se informa"],
0107:         future=["va a {base}", "planea {base}", "intentara {base}"],
0108:         statement="El {subject} {vp} {obj} {context}."
```

Code listing: run_lie_multilingual_max_audit.py (3)

```
0109:         question="El {subject} {base} {obj} {context}?",
0110:         false_question="Es falso que el {subject} {vp} {obj} {context}?",
0111:         modal_question="Es {marker} cierto que el {subject} {vp} {obj} {context}?",
0112:         future_truth_question="Sera cierto que el {subject} {vp} {obj} {context}?",
0113:         future_question="El {subject} va a {vp} {obj} {context}?",
0114:         did_fail_question="El {subject} no logro {base} {obj} {context}?",
0115:         modal_did_question="El {subject} {marker} {base} {obj} {context}?",
0116:     ),
0117:     LanguageSpec(
0118:         code="fr",
0119:         subjects=["scientifique", "ingenieur", "enseignant", "medecin", "programmeur", "chercheur", "analyste", "directeur"],
0120:         actions=[
0121:             ("a accepte", "accepter", "l'explication"),
0122:             ("a termine", "terminer", "la reparation"),
0123:             ("a confirme", "confirmer", "la reponse"),
0124:             ("a approuve", "approuver", "le traitement"),
0125:             ("a corrige", "corriger", "le bogue"),
0126:             ("a soutenu", "soutenir", "la theorie"),
0127:         ],
0128:         contexts=["pendant la reunion", "apres l'examen", "avant la date limite", "le matin"],
0129:         source="Le {subject} {past} {obj} {context}.",
0130:         neg=["n'a pas reussi a {base}", "n'a pas voulu {base}", "a refuse de {base}"],
0131:         mod=["pretendument", "apparemment", "selon le rapport"],
0132:         future=["va {base}", "prevoit de {base}", "tentera de {base}"],
0133:         statement="Le {subject} {vp} {obj} {context}.",
0134:         question="Le {subject} peut-il {base} {obj} {context}?",
0135:         false_question="Est-il faux que le {subject} {vp} {obj} {context}?",
0136:         modal_question="Est-il {marker} vrai que le {subject} {vp} {obj} {context}?",
0137:         future_truth_question="Sera-t-il vrai que le {subject} {vp} {obj} {context}?",
0138:         future_question="Le {subject} va-t-il {vp} {obj} {context}?",
0139:         did_fail_question="Le {subject} n'a-t-il pas reussi a {base} {obj} {context}?",
0140:         modal_did_question="Le {subject} a-t-il {marker} {base} {obj} {context}?",
0141:     ),
0142:     LanguageSpec(
0143:         code="de",
0144:         subjects=["Wissenschaftler", "Ingenieur", "Lehrer", "Arzt", "Programmierer", "Forscher", "Analyst", "Manager"],
0145:         actions=[
0146:             ("akzeptierte", "akzeptieren", "die Erklaerung"),
0147:             ("beendete", "beenden", "die Reparatur"),
0148:             ("bestaetigte", "bestaetigen", "die Antwort"),
0149:             ("genehmigte", "genehmigen", "die Behandlung"),
0150:             ("reparierte", "reparieren", "den Fehler"),
0151:             ("unterstuetzte", "unterstuetzen", "die Theorie"),
0152:         ],
0153:         contexts=["waehrend der Besprechung", "nach der Pruefung", "vor der Frist", "am Morgen"],
0154:         source="Der {subject} {past} {obj} {context}.",
0155:         neg=["schaffte es nicht zu {base}", "weigerte sich zu {base}", "konnte nicht {base}"],
0156:         mod=["angeblich", "offenbar", "berichten zufolge"],
0157:         future=["wird {base}", "plant zu {base}", "versucht zu {base}"],
0158:         statement="Der {subject} {vp} {obj} {context}.",
0159:         question="Kann der {subject} {base} {obj} {context}?",
0160:         false_question="Ist es falsch, dass der {subject} {vp} {obj} {context}?",
0161:         modal_question="Ist es {marker} wahr, dass der {subject} {vp} {obj} {context}?",
0162:         future_truth_question="Wird es wahr sein, dass der {subject} {vp} {obj} {context}?",
```


Code listing: run_lie_multilingual_max_audit.py (4)

```
0163:         future_question="Wird der {subject} {vp} {obj} {context}?",
0164:         did_fail_question="Schaffte es der {subject} nicht zu {base} {obj} {context}?",
0165:         modal_did_question="Hat der {subject} {marker} {base} {obj} {context}?",
0166:     ),
0167:     LanguageSpec(
0168:         code="ru",
0169:         subjects=["uchenyj", "inzhenery", "uchitel", "vrach", "programmist", "issledovatel", "analitik", "menedzher"],
0170:         actions=[
0171:             ("prinyal", "prinyat", "obyasnenie"),
0172:             ("zavershil", "zavershit", "remont"),
0173:             ("podtverdil", "podtverdit", "otvet"),
0174:             ("odobril", "odobrit", "lechenie"),
0175:             ("ispravil", "ispravit", "oshibku"),
0176:             ("podderzhal", "podderzhat", "teoriyu"),
0177:         ],
0178:         contexts=["vo vremya vstrechi", "posle proverki", "do sroka", "utrom"],
0179:         source="{subject} {past} {obj} {context}.",
0180:         neg=["ne smog {base}", "otkazalsya {base}", "ne stal {base}"],
0181:         mod=["predpolozhitelno", "po soobshcheniyam", "po-vidimomu"],
0182:         future=["budet {base}", "planiruet {base}", "poprobuetsya {base}"],
0183:         statement="{subject} {vp} {obj} {context}.",
0184:         question="{subject} mozhetsya {base} {obj} {context}?",
0185:         false_question="Nepravda li, chto {subject} {vp} {obj} {context}?",
0186:         modal_question="Pravda li, chto {subject} {marker} {vp} {obj} {context}?",
0187:         future_truth_question="Budet li pravday, chto {subject} {vp} {obj} {context}?",
0188:         future_question="Budet li {subject} {vp} {obj} {context}?",
0189:         did_fail_question="{subject} ne smog {base} {obj} {context}?",
0190:         modal_did_question="{subject} {marker} {base} {obj} {context}?",
0191:     ),
0192:     LanguageSpec(
0193:         code="zh",
0194:         subjects=["\u79d1\u5b66\u5bb6", "\u5de5\u7a0b\u5e08", "\u8001\u5e08", "\u533b\u751f", "\u7a0b\u5e8f\u5458", "\u7814\u79d1\u5bb6", "\u5de5\u7a0b\u5e08", "\u8001\u5e08", "\u533b\u751f", "\u7a0b\u5e8f\u5458", "\u7814\u79d1\u5bb6"],
0195:         actions=[
0196:             ("\u63a5\u53d7\u4e86", "\u63a5\u53d7", "\u89e3\u91ca"),
0197:             ("\u5b8c\u6210\u4e86", "\u5b8c\u6210", "\u4fee\u7406"),
0198:             ("\u786e\u8ba4\u4e86", "\u786e\u8ba4", "\u7b54\u6848"),
0199:             ("\u6279\u51c6\u4e86", "\u6279\u51c6", "\u6cbb\u7597"),
0200:             ("\u4fee\u590d\u4e86", "\u4fee\u590d", "\u9519\u8bef"),
0201:             ("\u652f\u6301\u4e86", "\u652f\u6301", "\u7406\u8bba"),
0202:         ],
0203:         contexts=["\u5728\u4f1a\u8bae\u671f\u95f4", "\u5728\u5ba1\u67e5\u4e4b\u540e", "\u5728\u622a\u66b6\u65e5\u671f\u4e4b\u540e", "\u5728\u622a\u66b6\u65e5\u671f\u4e4b\u540e", "\u5728\u622a\u66b6\u65e5\u671f\u4e4b\u540e"],
0204:         source="{subject}{context}{past}{obj}\u3002",
0205:         neg=["\u672a\u80fd\u5b8c\u6210", "\u6ca1\u6709", "\u62d2\u7edd"],
0206:         mod=["\u636e\u79f0", "\u636e\u62a5\u9053", "\u663e\u7136"],
0207:         future=["\u5c06\u53d6", "\u8ba1\u5212", "\u6253\u7b97"],
0208:         statement="{subject}{context}{vp}{obj}\u3002",
0209:         question="{subject}{context}{base}{obj}\u5417\u5417",
0210:         false_question="{subject}{context}{vp}{obj}\u662f\u5047\u7684\u5417",
0211:         modal_question="{subject}{context}{marker}{vp}{obj}\u662f\u771f\u7684\u5417\u5417",
0212:         future_truth_question="{subject}{context}{vp}{obj}\u5c06\u662f\u771f\u7684\u5417\u5417",
0213:         future_question="{subject}{context}\u4f1a\u53d6\u5417\u5417",
0214:         did_fail_question="{subject}{context}\u672a\u80fd\u5b8c\u6210\u5417\u5417",
0215:         modal_did_question="{subject}{context}{marker}{base}{obj}\u5417\u5417",
0216:     ),
```

Code listing: run_lie_multilingual_max_audit.py (5)

```
0217:     LanguageSpec(
0218:         code="ar",
0219:         subjects=["al-bahith", "al-muhandis", "al-muallim", "al-tabib", "al-mubarmij", "al-muhalil", "al-mudir", "al-muraji"]
0220:         actions=[
0221:             ("qabila", "yaqbal", "al-tafsir"),
0222:             ("akmala", "yukmil", "al-islah"),
0223:             ("akkada", "yuakkid", "al-ijaba"),
0224:             ("wafaqa ala", "yuwafiq ala", "al-ilaj"),
0225:             ("aslaha", "yuslih", "al-khata"),
0226:             ("daama", "yadaam", "al-nazariya"),
0227:         ],
0228:         contexts=["athna al-ijtimaa", "baad al-murajaa", "qabla al-mawed", "sabahan"],
0229:         source="{subject} {past} {obj} {context}.",
0230:         neg=["lam {base}", "fashila an {base}", "rafada an {base}"],
0231:         mod=["mazuman", "hasab al-taqrir", "ala ma yabdu"],
0232:         future=["sawfa {base}", "yukhatit an {base}", "sayuhawil an {base}"],
0233:         statement="{subject} {vp} {obj} {context}.",
0234:         question="hal {subject} {base} {obj} {context}?",
0235:         false_question="hal min al-khata anna {subject} {vp} {obj} {context}?",
0236:         modal_question="hal min al-sahih {marker} anna {subject} {vp} {obj} {context}?",
0237:         future_truth_question="hal sayakun sahihan anna {subject} {vp} {obj} {context}?",
0238:         future_question="hal sawfa {subject} {vp} {obj} {context}?",
0239:         did_fail_question="hal {subject} fashila an {base} {obj} {context}?",
0240:         modal_did_question="hal {subject} {marker} {base} {obj} {context}?",
0241:     ),
0242: ]
0243:
0244:
0245: def clean(text: str) -> str:
0246:     return " ".join(text.split())
0247:
0248:
0249: def format_vp(spec: LanguageSpec, ops: tuple[str, ...], past: str, base: str, form: dict) -> str:
0250:     negated = "N" in ops
0251:     modal = "M" in ops
0252:     future = "T" in ops
0253:
0254:     if future:
0255:         vp = form["future"].format(base=base)
0256:     elif negated:
0257:         vp = form["neg"].format(base=base)
0258:     else:
0259:         vp = past
0260:
0261:     if negated and future:
0262:         vp = form["future"].format(base=base)
0263:         if ops.index("N") < ops.index("T"):
0264:             vp = spec.neg[0].format(base=vp)
0265:         else:
0266:             vp = spec.neg[1].format(base=base)
0267:
0268:     if modal:
0269:         marker = form["mod"]
0270:         if spec.code == "zh":
```

Code listing: run_lie_multilingual_max_audit.py (6)

```
0271:         vp = marker + vp
0272:     elif ops.index("M") == 0:
0273:         vp = f"{marker} {vp}"
0274:     else:
0275:         vp = f"{vp} {marker}"
0276:
0277:     return clean(vp)
0278:
0279:
0280: def compose_text(spec: LanguageSpec, ops: tuple[str, ...], subject: str, past: str, base: str, obj: str, context: str, form: d
0281:     vp = format_vp(spec, ops, past, base, form)
0282:     question = "Q" in ops
0283:     negated = "N" in ops
0284:     modal = "M" in ops
0285:     future = "T" in ops
0286:
0287:     if not question:
0288:         return clean(spec.statement.format(subject=subject, vp=vp, obj=obj, context=context))
0289:
0290:     if negated and ops.index("Q") < ops.index("N"):
0291:         positive_vp = format_vp(spec, tuple(op for op in ops if op != "N"), past, base, form)
0292:         return clean(spec.false_question.format(subject=subject, vp=positive_vp, obj=obj, context=context))
0293:
0294:     if modal and ops.index("Q") < ops.index("M"):
0295:         positive_vp = format_vp(spec, tuple(op for op in ops if op != "M"), past, base, form)
0296:         return clean(spec.modal_question.format(subject=subject, marker=form["mod"], vp=positive_vp, obj=obj, context=context))
0297:
0298:     if future and ops.index("Q") < ops.index("T"):
0299:         positive_vp = format_vp(spec, tuple(op for op in ops if op != "T"), past, base, form)
0300:         return clean(spec.future_truth_question.format(subject=subject, vp=positive_vp, obj=obj, context=context))
0301:
0302:     if future:
0303:         q_vp = format_vp(spec, tuple(op for op in ops if op != "T"), past, base, form)
0304:         return clean(spec.future_question.format(subject=subject, vp=q_vp, obj=obj, context=context))
0305:
0306:     if negated:
0307:         return clean(spec.did_fail_question.format(subject=subject, base=base, obj=obj, context=context))
0308:
0309:     if modal:
0310:         return clean(spec.modal_did_question.format(subject=subject, marker=form["mod"], base=base, obj=obj, context=context))
0311:
0312:     return clean(spec.question.format(subject=subject, base=base, obj=obj, context=context))
0313:
0314:
0315: def build_dataset(n_templates_per_language: int, seed: int) -> pd.DataFrame:
0316:     rng = np.random.default_rng(seed)
0317:     rows = []
0318:     for spec in LANGS:
0319:         combos = [(subject, action, context) for subject in spec.subjects for action in spec.actions for context in spec.conte
0320:         rng.shuffle(combos)
0321:         combos = combos[:n_templates_per_language]
0322:         for template_id, (subject, action, context) in enumerate(combos):
0323:             past, base, obj = action
0324:             form = {
```

Code listing: run_lie_multilingual_max_audit.py (7)

```
0325:         "neg": rng.choice(spec.neg),
0326:         "mod": rng.choice(spec.mod),
0327:         "future": rng.choice(spec.future),
0328:     }
0329:     source = clean(spec.source.format(subject=subject, past=past, obj=obj, context=context))
0330:     for a, b in PAIR_OPS:
0331:         ab = compose_text(spec, (a, b), subject, past, base, obj, context, form)
0332:         ba = compose_text(spec, (b, a), subject, past, base, obj, context, form)
0333:         rows.append({
0334:             "language": spec.code,
0335:             "template_id": template_id,
0336:             "kind": "pair",
0337:             "label": f"{a}_{b}_vs_{b}_{a}",
0338:             "source": source,
0339:             "op_a": a,
0340:             "op_b": b,
0341:             "op_c": "",
0342:             "ab_text": ab,
0343:             "ba_text": ba,
0344:         })
0345:     for triple in TRIPLE_OPS:
0346:         row = {
0347:             "language": spec.code,
0348:             "template_id": template_id,
0349:             "kind": "triple",
0350:             "label": "".join(triple),
0351:             "source": source,
0352:             "op_a": triple[0],
0353:             "op_b": triple[1],
0354:             "op_c": triple[2],
0355:         }
0356:         for name, perm in [
0357:             ("abc", (0, 1, 2)),
0358:             ("bca", (1, 2, 0)),
0359:             ("cab", (2, 0, 1)),
0360:             ("acb", (0, 2, 1)),
0361:             ("cba", (2, 1, 0)),
0362:             ("bac", (1, 0, 2)),
0363:         ]:
0364:             seq = tuple(triple[i] for i in perm)
0365:             row[f"{name}_text"] = compose_text(spec, seq, subject, past, base, obj, context, form)
0366:         rows.append(row)
0367:     return pd.DataFrame(rows)
0368:
0369:
0370: def mean_pool(hidden, mask):
0371:     mask = mask.unsqueeze(-1)
0372:     return (hidden * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1)
0373:
0374:
0375: @torch.no_grad()
0376: def embed_texts(model_name: str, texts: list[str], device: str, batch_size: int) -> np.ndarray:
0377:     tokenizer = AutoTokenizer.from_pretrained(model_name)
0378:     if tokenizer.pad_token is None:
```

Code listing: run_lie_multilingual_max_audit.py (8)

```
0379:         tokenizer.pad_token = tokenizer.eos_token or tokenizer.unk_token or "[PAD]"
0380:     model = AutoModel.from_pretrained(model_name).to(device)
0381:     model.eval()
0382:
0383:     prefix = "query: " if "e5" in model_name.lower() else ""
0384:     embeddings = []
0385:     for start in range(0, len(texts), batch_size):
0386:         batch = [prefix + text for text in texts[start:start + batch_size]]
0387:         enc = tokenizer(batch, padding=True, truncation=True, max_length=128, return_tensors="pt").to(device)
0388:         out = model(**enc)
0389:         pooled = mean_pool(out.last_hidden_state, enc["attention_mask"])
0390:         embeddings.append(pooled.detach().cpu().numpy())
0391:     del model
0392:     gc.collect()
0393:     if device == "cuda":
0394:         torch.cuda.empty_cache()
0395:     return np.vstack(embeddings)
0396:
0397:
0398: def relative_norm(vec, scale_vecs):
0399:     scale = np.mean([np.linalg.norm(v) for v in scale_vecs]) + 1e-12
0400:     return float(np.linalg.norm(vec) / scale)
0401:
0402:
0403: def empirical_p(null_values, observed, smaller_is_better=True):
0404:     null_values = np.asarray(null_values)
0405:     if smaller_is_better:
0406:         return float(((null_values <= observed).sum() + 1) / (len(null_values) + 1))
0407:     return float(((null_values >= observed).sum() + 1) / (len(null_values) + 1))
0408:
0409:
0410: def fit_pca(raw: np.ndarray, pca_dim: int) -> np.ndarray:
0411:     dim = min(pca_dim, raw.shape[0] - 1, raw.shape[1])
0412:     return PCA(n_components=dim, random_state=42).fit_transform(raw)
0413:
0414:
0415: def classifier_controls(df, vecs_by_text, label_col, endpoint_cols, train_langs, test_langs):
0416:     rows = []
0417:     sub = df[df["kind"].eq("pair")].copy()
0418:     labels = sorted(sub[label_col].unique())
0419:     y_map = {label: i for i, label in enumerate(labels)}
0420:     y = sub[label_col].map(y_map).to_numpy()
0421:     train_mask = sub["language"].isin(train_langs).to_numpy()
0422:     test_mask = sub["language"].isin(test_langs).to_numpy()
0423:     if train_mask.sum() == 0 or test_mask.sum() == 0:
0424:         return pd.DataFrame()
0425:
0426:     feature_sets = {
0427:         "source_only": np.vstack([vecs_by_text[t] for t in sub["source"]]),
0428:         "ab_endpoint_only": np.vstack([vecs_by_text[t] for t in sub["ab_text"]]),
0429:         "ba_endpoint_only": np.vstack([vecs_by_text[t] for t in sub["ba_text"]]),
0430:         "ab_delta_only": np.vstack([vecs_by_text[row.ab_text] - vecs_by_text[row.source] for row in sub.itertuples()]),
0431:         "ba_delta_only": np.vstack([vecs_by_text[row.ba_text] - vecs_by_text[row.source] for row in sub.itertuples()]),
0432:         "commutator_delta": np.vstack([
```

Code listing: run_lie_multilingual_max_audit.py (9)

```
0433:         (vecs_by_text[row.ab_text] - vecs_by_text[row.source]) - (vecs_by_text[row.ba_text] - vecs_by_text[row.source])
0434:         for row in sub.itertuples()
0435:     ]),
0436: }
0437: for feature, x in feature_sets.items():
0438:     clf = LogisticRegression(max_iter=5000, C=1.0, random_state=42)
0439:     clf.fit(normalize(x[train_mask]), y[train_mask])
0440:     pred = clf.predict(normalize(x[test_mask]))
0441:     rows.append({
0442:         "feature": feature,
0443:         "train_languages": ", ".join(train_langs),
0444:         "test_languages": ", ".join(test_langs),
0445:         "accuracy": accuracy_score(y[test_mask], pred),
0446:         "macro_f1": f1_score(y[test_mask], pred, average="macro"),
0447:         "chance": 1.0 / len(labels),
0448:         "n_train": int(train_mask.sum()),
0449:         "n_test": int(test_mask.sum()),
0450:     })
0451: return pd.DataFrame(rows)
0452:
0453:
0454: def analyze_model(model_name: str, df: pd.DataFrame, args: dict):
0455:     safe = model_name.replace("/", "__").replace("-", "_")
0456:     done_marker = CHECKPOINT_DIR / f"{safe}.done.json"
0457:     if done_marker.exists() and not args["force"]:
0458:         print(f"[SKIP] {model_name} already done", flush=True)
0459:         return
0460:
0461:     print(f"\n=== MODEL {model_name} ===", flush=True)
0462:     text_cols = ["source", "ab_text", "ba_text", "abc_text", "bca_text", "cab_text", "acb_text", "cba_text", "bac_text"]
0463:     texts = sorted({str(v) for col in text_cols if col in df.columns for v in df[col].dropna().tolist()})
0464:     print(f"texts={len(texts)}", flush=True)
0465:
0466:     raw = embed_texts(model_name, texts, args["device"], args["batch_size"])
0467:     vecs = fit_pca(raw, args["pca_dim"])
0468:     vecs_by_text = {text: vecs[i] for i, text in enumerate(texts)}
0469:
0470:     pair_rows = []
0471:     for row in df[df["kind"].eq("pair")].itertuples(index=False):
0472:         x = vecs_by_text[row.source]
0473:         dab = vecs_by_text[row.ab_text] - x
0474:         dba = vecs_by_text[row.ba_text] - x
0475:         comm = dab - dba
0476:         rel = relative_norm(comm, [dab, dba])
0477:         cos = float(np.sum(normalize(dab.reshape(1, -1)) * normalize(dba.reshape(1, -1))))
0478:         pair_rows.append({
0479:             "model": model_name,
0480:             "language": row.language,
0481:             "template_id": row.template_id,
0482:             "pair": row.label,
0483:             "relative_commutator_norm": rel,
0484:             "commutator_norm": float(np.linalg.norm(comm)),
0485:             "cosine_ab_ba": cos,
0486:             "noncommutativity": 1.0 - cos,
```

Code listing: run_lie_multilingual_max_audit.py (10)

```
0487:         })
0488:     pair_raw = pd.DataFrame(pair_rows)
0489:
0490:     triple_rows = []
0491:     rng = np.random.default_rng(args["seed"])
0492:     for row in df[df["kind"].eq("triple")].itertuples(index=False):
0493:         x = vecs_by_text[row.source]
0494:         endpoint_names = ["abc_text", "bca_text", "cab_text", "acb_text", "cba_text", "bac_text"]
0495:         deltas = [vecs_by_text[getattr(row, name)] - x for name in endpoint_names]
0496:         signed = deltas[0] + deltas[1] + deltas[2] - deltas[3] - deltas[4] - deltas[5]
0497:         scale = np.mean([np.linalg.norm(delta) for delta in deltas]) + 1e-12
0498:         rel = float(np.linalg.norm(signed) / scale)
0499:         nulls = []
0500:         for _ in range(args["n_null"]):
0501:             pos = set(rng.choice(6, size=3, replace=False).tolist())
0502:             s = np.zeros_like(signed)
0503:             for i, delta in enumerate(deltas):
0504:                 s += delta if i in pos else -delta
0505:             nulls.append(float(np.linalg.norm(s) / scale))
0506:         nulls = np.asarray(nulls)
0507:         triple_rows.append({
0508:             "model": model_name,
0509:             "language": row.language,
0510:             "template_id": row.template_id,
0511:             "triple": row.label,
0512:             "relative_signed_permutation_norm": rel,
0513:             "null_mean": float(nulls.mean()),
0514:             "null_std": float(nulls.std(ddof=1)),
0515:             "ratio_to_null_mean": float(rel / (nulls.mean() + 1e-12)),
0516:             "null_percentile_smaller_is_better": float(percentileofscore(nulls, rel, kind="mean")),
0517:             "p_smaller_or_equal_null": empirical_p(nulls, rel, smaller_is_better=True),
0518:         })
0519:     triple_raw = pd.DataFrame(triple_rows)
0520:
0521:     pair_summary = pair_raw.groupby(["model", "language", "pair"]).agg(
0522:         mean_relative_commutator_norm=("relative_commutator_norm", "mean"),
0523:         std_relative_commutator_norm=("relative_commutator_norm", "std"),
0524:         mean_noncommutativity=("noncommutativity", "mean"),
0525:         n=("pair", "count"),
0526:     ).reset_index()
0527:
0528:     triple_summary = triple_raw.groupby(["model", "language", "triple"]).agg(
0529:         mean_relative_signed_permutation_norm=("relative_signed_permutation_norm", "mean"),
0530:         mean_ratio_to_null_mean=("ratio_to_null_mean", "mean"),
0531:         mean_null_percentile=("null_percentile_smaller_is_better", "mean"),
0532:         max_p_smaller_or_equal_null=("p_smaller_or_equal_null", "max"),
0533:         n=("triple", "count"),
0534:     ).reset_index()
0535:
0536:     cent_rows = []
0537:     for kind, raw_df, label_col, value_col in [
0538:         ("pair_commutator", pair_raw, "pair", "relative_commutator_norm"),
0539:         ("triple_signed", triple_raw, "triple", "relative_signed_permutation_norm"),
0540:     ]:
```

Code listing: run_lie_multilingual_max_audit.py (11)

```
0541:     labels = sorted(raw_df[label_col].unique())
0542:     langs = sorted(raw_df["language"].unique())
0543:     for label in labels:
0544:         lang_vecs = {}
0545:         for lang in langs:
0546:             sub_df = df[(df["language"].eq(lang)) & (df["kind"].eq("pair" if kind == "pair_commutator" else "triple"))]
0547:             if kind == "pair_commutator":
0548:                 sub_df = sub_df[sub_df["label"].eq(label)]
0549:                 vec_list = []
0550:                 for r in sub_df.itertuples(index=False):
0551:                     x = vecs_by_text[r.source]
0552:                     vec_list.append((vecs_by_text[r.ab_text] - x) - (vecs_by_text[r.ba_text] - x))
0553:             else:
0554:                 sub_df = sub_df[sub_df["label"].eq(label)]
0555:                 vec_list = []
0556:                 for r in sub_df.itertuples(index=False):
0557:                     x = vecs_by_text[r.source]
0558:                     d = [vecs_by_text[getattr(r, name)] - x for name in ["abc_text", "bca_text", "cab_text", "acb_text", "
0559:                     vec_list.append(d[0] + d[1] + d[2] - d[3] - d[4] - d[5])
0560:                 lang_vecs[lang] = np.mean(vec_list, axis=0)
0561:     for i, lang_a in enumerate(langs):
0562:         for lang_b in langs[i + 1:]:
0563:             va = normalize(lang_vecs[lang_a].reshape(1, -1))
0564:             vb = normalize(lang_vecs[lang_b].reshape(1, -1))
0565:             cent_rows.append({
0566:                 "model": model_name,
0567:                 "kind": kind,
0568:                 "label": label,
0569:                 "language_a": lang_a,
0570:                 "language_b": lang_b,
0571:                 "centroid_cosine": float(np.sum(va * vb)),
0572:             })
0573:
0574:     centroid_consistency = pd.DataFrame(cent_rows)
0575:     control_parts = []
0576:     all_langs = [spec.code for spec in LANGS]
0577:     for heldout in all_langs:
0578:         train_langs = [lang for lang in all_langs if lang != heldout]
0579:         controls = classifier_controls(df, vecs_by_text, "label", [], train_langs=train_langs, test_langs=[heldout])
0580:         if not controls.empty:
0581:             controls.insert(0, "model", model_name)
0582:             control_parts.append(controls)
0583:     controls = pd.concat(control_parts, ignore_index=True)
0584:
0585:     pair_raw.to_csv(CSV_DIR / f"pair_raw_{safe}.csv", index=False)
0586:     triple_raw.to_csv(CSV_DIR / f"triple_raw_{safe}.csv", index=False)
0587:     pair_summary.to_csv(CSV_DIR / f"pair_summary_{safe}.csv", index=False)
0588:     triple_summary.to_csv(CSV_DIR / f"triple_summary_{safe}.csv", index=False)
0589:     centroid_consistency.to_csv(CSV_DIR / f"cross_language_centroid_consistency_{safe}.csv", index=False)
0590:     controls.to_csv(CSV_DIR / f"endpoint_controls_{safe}.csv", index=False)
0591:     done_marker.write_text(json.dumps({"model": model_name, "finished_at": time.ctime()}, indent=2), encoding="utf-8")
0592:
0593:
0594: def aggregate_outputs(models: list[str]):
```


Code listing: run_lie_multilingual_max_audit.py (12)

```
0595:     def read_many(pattern):
0596:         paths = [
0597:             path for path in sorted(CSV_DIR.glob(pattern))
0598:             if "all_models" not in path.name and path.name != "endpoint_controls_summary.csv"
0599:         ]
0600:         if not paths:
0601:             return pd.DataFrame()
0602:         return pd.concat([pd.read_csv(path) for path in paths], ignore_index=True)
0603:
0604:     outputs = {
0605:         "pair_raw_all_models.csv": read_many("pair_raw_*.csv"),
0606:         "triple_raw_all_models.csv": read_many("triple_raw_*.csv"),
0607:         "pair_summary_all_models.csv": read_many("pair_summary_*.csv"),
0608:         "triple_summary_all_models.csv": read_many("triple_summary_*.csv"),
0609:         "cross_language_centroid_consistency_all_models.csv": read_many("cross_language_centroid_consistency_*.csv"),
0610:         "endpoint_controls_all_models.csv": read_many("endpoint_controls_*.csv"),
0611:     }
0612:     for filename, df in outputs.items():
0613:         if not df.empty:
0614:             df.to_csv(CSV_DIR / filename, index=False)
0615:
0616:     if not outputs["triple_summary_all_models.csv"].empty:
0617:         triples = outputs["triple_summary_all_models.csv"]
0618:         triple_global = (
0619:             triples.groupby("triple")
0620:             .agg(
0621:                 mean_ratio_to_null_mean=("mean_ratio_to_null_mean", "mean"),
0622:                 std_ratio_to_null_mean=("mean_ratio_to_null_mean", "std"),
0623:                 min_ratio_to_null_mean=("mean_ratio_to_null_mean", "min"),
0624:                 max_ratio_to_null_mean=("mean_ratio_to_null_mean", "max"),
0625:                 below_null_cells=("mean_ratio_to_null_mean", lambda s: int((s < 1.0).sum())),
0626:                 n_cells=("mean_ratio_to_null_mean", "count"),
0627:             )
0628:             .reset_index()
0629:         )
0630:         triple_global["frac_below_null_cells"] = triple_global["below_null_cells"] / triple_global["n_cells"]
0631:         triple_global.to_csv(CSV_DIR / "triple_global_summary.csv", index=False)
0632:
0633:         triple_by_model = (
0634:             triples.groupby(["model", "triple"])
0635:             .agg(
0636:                 mean_ratio_to_null_mean=("mean_ratio_to_null_mean", "mean"),
0637:                 std_ratio_to_null_mean=("mean_ratio_to_null_mean", "std"),
0638:                 min_ratio_to_null_mean=("mean_ratio_to_null_mean", "min"),
0639:                 max_ratio_to_null_mean=("mean_ratio_to_null_mean", "max"),
0640:                 below_null_languages=("mean_ratio_to_null_mean", lambda s: int((s < 1.0).sum())),
0641:                 n_languages=("mean_ratio_to_null_mean", "count"),
0642:             )
0643:             .reset_index()
0644:         )
0645:         triple_by_model.to_csv(CSV_DIR / "triple_by_model_summary.csv", index=False)
0646:
0647:         triple_by_language = (
0648:             triples.groupby(["language", "triple"])
```

Code listing: run_lie_multilingual_max_audit.py (13)

```
0649:         .agg(
0650:             mean_ratio_to_null_mean=("mean_ratio_to_null_mean", "mean"),
0651:             std_ratio_to_null_mean=("mean_ratio_to_null_mean", "std"),
0652:             min_ratio_to_null_mean=("mean_ratio_to_null_mean", "min"),
0653:             max_ratio_to_null_mean=("mean_ratio_to_null_mean", "max"),
0654:             below_null_models=("mean_ratio_to_null_mean", lambda s: int((s < 1.0).sum())),
0655:             n_models=("mean_ratio_to_null_mean", "count"),
0656:         )
0657:         .reset_index()
0658:     )
0659: triple_by_language.to_csv(CSV_DIR / "triple_by_language_summary.csv", index=False)
0660:
0661: pivot = triples.pivot_table(index=["model", "language"], columns="triple", values="mean_ratio_to_null_mean")
0662: plt.figure(figsize=(10, max(4, len(pivot) * 0.25)))
0663: plt.imshow(pivot.values, aspect="auto", vmin=0, vmax=max(1.5, np.nanmax(pivot.values)))
0664: plt.colorbar(label="Mean ratio to signed-null mean")
0665: plt.xticks(np.arange(len(pivot.columns)), pivot.columns, rotation=35, ha="right")
0666: plt.yticks(np.arange(len(pivot.index)), [f"{m}\\n{l}" for m, l in pivot.index], fontsize=6)
0667: plt.title("Multilingual third-order signed permutation ratios")
0668: plt.tight_layout()
0669: plt.savefig(FIG_DIR / "01_multilingual_signed_permutation_ratios.png", dpi=220, bbox_inches="tight")
0670: plt.close()
0671:
0672: ordered = triple_global.sort_values("mean_ratio_to_null_mean")
0673: plt.figure(figsize=(7, 4))
0674: plt.bar(ordered["triple"], ordered["mean_ratio_to_null_mean"], yerr=ordered["std_ratio_to_null_mean"], capsize=4)
0675: plt.axhline(1.0, color="black", linestyle="--", linewidth=1)
0676: plt.ylabel("Mean ratio to signed-null mean")
0677: plt.title("Global multilingual signed-permutation ratio by triple")
0678: plt.tight_layout()
0679: plt.savefig(FIG_DIR / "02_triple_global_ratio_summary.png", dpi=220, bbox_inches="tight")
0680: plt.close()
0681:
0682: if not outputs["pair_summary_all_models.csv"].empty:
0683:     pairs = outputs["pair_summary_all_models.csv"]
0684:     pair_global = (
0685:         pairs.groupby("pair")
0686:         .agg(
0687:             mean_relative_commutator_norm=("mean_relative_commutator_norm", "mean"),
0688:             std_relative_commutator_norm=("mean_relative_commutator_norm", "std"),
0689:             min_relative_commutator_norm=("mean_relative_commutator_norm", "min"),
0690:             max_relative_commutator_norm=("mean_relative_commutator_norm", "max"),
0691:             n_cells=("mean_relative_commutator_norm", "count"),
0692:         )
0693:         .reset_index()
0694:     )
0695:     pair_global.to_csv(CSV_DIR / "pair_global_summary.csv", index=False)
0696:
0697: if not outputs["endpoint_controls_all_models.csv"].empty:
0698:     controls = outputs["endpoint_controls_all_models.csv"]
0699:     summary = controls.groupby(["model", "feature"]).agg(mean_macro_f1=("macro_f1", "mean"), min_macro_f1=("macro_f1", "mi
0700:     summary.to_csv(CSV_DIR / "endpoint_controls_summary.csv", index=False)
0701:
0702:     control_pivot = summary.pivot_table(index="model", columns="feature", values="mean_macro_f1")
```

Code listing: run_lie_multilingual_max_audit.py (14)

```
0703:         plt.figure(figsize=(10, 4.8))
0704:         plt.imshow(control_pivot.values, aspect="auto", vmin=0, vmax=1)
0705:         plt.colorbar(label="Mean macro F1 across held-out languages")
0706:         plt.xticks(np.arange(len(control_pivot.columns)), control_pivot.columns, rotation=35, ha="right")
0707:         plt.yticks(np.arange(len(control_pivot.index)), control_pivot.index, fontsize=7)
0708:         plt.title("Endpoint/control feature transfer across held-out languages")
0709:         plt.tight_layout()
0710:         plt.savefig(FIG_DIR / "03_endpoint_control_macro_f1.png", dpi=220, bbox_inches="tight")
0711:         plt.close()
0712:
0713: if not outputs["cross_language_centroid_consistency_all_models.csv"].empty:
0714:     consistency = outputs["cross_language_centroid_consistency_all_models.csv"]
0715:     consistency_summary = (
0716:         consistency.groupby("kind")
0717:         .agg(
0718:             mean_centroid_cosine=("centroid_cosine", "mean"),
0719:             std_centroid_cosine=("centroid_cosine", "std"),
0720:             min_centroid_cosine=("centroid_cosine", "min"),
0721:             max_centroid_cosine=("centroid_cosine", "max"),
0722:             n=("centroid_cosine", "count"),
0723:         )
0724:         .reset_index()
0725:     )
0726:     consistency_summary.to_csv(CSV_DIR / "cross_language_centroid_summary.csv", index=False)
0727:
0728:     labels = list(consistency_summary["kind"])
0729:     data = [consistency.loc[consistency["kind"] == label, "centroid_cosine"].values for label in labels]
0730:     plt.figure(figsize=(6, 4))
0731:     plt.boxplot(data, tick_labels=labels, showfliers=False)
0732:     plt.axhline(0, color="black", linestyle="--", linewidth=1)
0733:     plt.ylabel("Cross-language centroid cosine")
0734:     plt.title("Cross-language consistency of composition centroids")
0735:     plt.tight_layout()
0736:     plt.savefig(FIG_DIR / "04_cross_language_centroid_consistency.png", dpi=220, bbox_inches="tight")
0737:     plt.close()
0738:
0739:
0740: def main():
0741:     args = {
0742:         "seed": int(os.getenv("LIE_MULTI_SEED", "20260622")),
0743:         "n_templates_per_language": int(os.getenv("LIE_MULTI_TEMPLATES_PER_LANGUAGE", "48")),
0744:         "n_null": int(os.getenv("LIE_MULTI_N_NULL", "2000")),
0745:         "pca_dim": int(os.getenv("LIE_MULTI_PCA_DIM", "96")),
0746:         "batch_size": int(os.getenv("LIE_MULTI_BATCH_SIZE", "8")),
0747:         "device": "cuda" if torch.cuda.is_available() else "cpu",
0748:         "force": os.getenv("LIE_MULTI_FORCE", "0") == "1",
0749:     }
0750:     model_env = os.getenv("LIE_MULTI_MODELS", "")
0751:     models = [m.strip() for m in model_env.split(",") if m.strip()] or MODELS
0752:
0753:     print("LIE MULTILINGUAL MAX AUDIT")
0754:     print(json.dumps(**args, "models": models, "languages": [spec.code for spec in LANGS]), indent=2))
0755:     OUT_DIR.mkdir(parents=True, exist_ok=True)
0756:     (OUT_DIR / "run_config.json").write_text(json.dumps(**args, "models": models, "languages": [spec.code for spec in LANGS])
```

Code listing: run_lie_multilingual_max_audit.py (15)

```
0757:
0758:     df = build_dataset(args["n_templates_per_language"], args["seed"])
0759:     df.to_csv(CSV_DIR / "multilingual_composition_dataset.csv", index=False)
0760:     print("dataset rows", len(df), "texts", len({v for col in df.columns if col.endswith("_text") or col == "source" for v in
0761:
0762:     failures = []
0763:     for model in models:
0764:         try:
0765:             analyze_model(model, df, args)
0766:             aggregate_outputs(models)
0767:         except Exception as exc:
0768:             print(f"[FAIL] {model}: {exc}", flush=True)
0769:             traceback.print_exc()
0770:             failures.append({"model": model, "error": repr(exc), "traceback": traceback.format_exc()})
0771:             (OUT_DIR / "failures.json").write_text(json.dumps(failures, indent=2), encoding="utf-8")
0772:             continue
0773:
0774:     aggregate_outputs(models)
0775:     status = {
0776:         "finished_at": time.ctime(),
0777:         "failures": failures,
0778:         "completed_markers": [p.name for p in CHECKPOINT_DIR.glob("*.done.json")],
0779:     }
0780:     (OUT_DIR / "run_status.json").write_text(json.dumps(status, indent=2), encoding="utf-8")
0781:     print("DONE")
0782:     print(json.dumps(status, indent=2))
0783:     return 0 if not failures else 2
0784:
0785:
0786: if __name__ == "__main__":
0787:     sys.exit(main())
```

Code listing: run_syntax_representation_ablation.py (1)

```
0001: import gc
0002: import os
0003: from pathlib import Path
0004:
0005: import numpy as np
0006: import pandas as pd
0007:
0008: from sklearn.linear_model import LogisticRegression
0009: from sklearn.metrics import accuracy_score
0010: from sklearn.pipeline import make_pipeline
0011: from sklearn.preprocessing import StandardScaler
0012: from sklearn.svm import LinearSVC
0013:
0014: import lie_llm_syntax_holdout_experiment as syntax
0015:
0016:
0017: OUT_DIR = Path(os.getenv("SYNTAX_ABLATION_OUT_DIR", "results/experiments/syntax_representation_ablation_results"))
0018: OUT_DIR.mkdir(parents=True, exist_ok=True)
0019:
0020: MODELS = [m.strip() for m in os.getenv(
0021:     "SYNTAX_ABLATION_MODELS",
0022:     "bert-base-uncased,roberta-base,distilroberta-base,gpt2,distilgpt2",
0023: ).split(",") if m.strip()]
0024:
0025: N_BASE = int(os.getenv("SYNTAX_ABLATION_N_BASE", str(syntax.N_BASE)))
0026:
0027:
0028: def run_classifiers(X, labels, train_mask, test_mask):
0029:     classifiers = {
0030:         "logreg": make_pipeline(
0031:             StandardScaler(),
0032:             LogisticRegression(max_iter=5000, class_weight="balanced", n_jobs=-1),
0033:         ),
0034:         "linear_svc": make_pipeline(
0035:             StandardScaler(),
0036:             LinearSVC(class_weight="balanced", max_iter=20000),
0037:         ),
0038:     }
0039:
0040:     rows = []
0041:     for clf_name, clf in classifiers.items():
0042:         clf.fit(X[train_mask], labels[train_mask])
0043:         pred = clf.predict(X[test_mask])
0044:         rows.append({
0045:             "classifier": clf_name,
0046:             "accuracy": float(accuracy_score(labels[test_mask], pred)),
0047:             "n_train": int(train_mask.sum()),
0048:             "n_test": int(test_mask.sum()),
0049:         })
0050:
0051:     return rows
0052:
0053:
0054: def main():
```

Code listing: run_syntax_representation_ablation.py (2)

```
0055:     syntax.OUT_DIR = str(OUT_DIR)
0056:     os.makedirs(syntax.OUT_DIR, exist_ok=True)
0057:
0058:     base_df = syntax.make_base_rows(N_BASE)
0059:     pair_df = syntax.build_pairs(base_df)
0060:     prompts, pair_df = syntax.index_prompts(pair_df)
0061:
0062:     base_df.to_csv(OUT_DIR / "base_sentences.csv", index=False)
0063:     pair_df.to_csv(OUT_DIR / "transformation_pairs.csv", index=False)
0064:     pd.DataFrame({"prompt_id": range(len(prompts)), "prompt": prompts}).to_csv(
0065:         OUT_DIR / "prompts.csv",
0066:         index=False,
0067:     )
0068:
0069:     labels = pair_df["class"].astype(str).to_numpy()
0070:     syntax_family = pair_df["syntax_family"].astype(str).to_numpy()
0071:     train_mask = np.isin(syntax_family, ["simple_svo", "with_context"])
0072:     test_mask = np.isin(syntax_family, ["relative_clause", "temporal_clause", "reported_statement", "conditional"])
0073:     src_idx = pair_df["source_idx"].to_numpy()
0074:     tgt_idx = pair_df["target_idx"].to_numpy()
0075:
0076:     all_rows = []
0077:
0078:     for model_name in MODELS:
0079:         print(f"\n=== MODEL: {model_name} ===")
0080:         vectors = syntax.get_vectors(model_name, prompts)
0081:
0082:         x_src = vectors[src_idx]
0083:         x_tgt = vectors[tgt_idx]
0084:         representations = {
0085:             "x_only": x_src,
0086:             "y_only": x_tgt,
0087:             "delta": x_tgt - x_src,
0088:             "concat": np.concatenate([x_src, x_tgt], axis=1),
0089:         }
0090:
0091:         for rep_name, X in representations.items():
0092:             for row in run_classifiers(X, labels, train_mask, test_mask):
0093:                 row.update({
0094:                     "model": model_name,
0095:                     "representation": rep_name,
0096:                     "random_baseline": 1 / len(set(labels)),
0097:                     "n_base": N_BASE,
0098:                 })
0099:                 print(model_name, rep_name, row["classifier"], f"{row['accuracy']:.4f}")
0100:                 all_rows.append(row)
0101:
0102:         del vectors, x_src, x_tgt, representations
0103:         gc.collect()
0104:
0105:     result = pd.DataFrame(all_rows)
0106:     result.to_csv(OUT_DIR / "syntax_representation_ablation.csv", index=False)
0107:
0108:     pivot = result.pivot_table(
```

Code listing: run_syntax_representation_ablation.py (3)

```
0109:         index=["model", "classifier"],
0110:         columns="representation",
0111:         values="accuracy",
0112:     ).reset_index()
0113:     pivot.to_csv(OUT_DIR / "syntax_representation_ablation_pivot.csv", index=False)
0114:
0115:     print("\nSUMMARY")
0116:     print(pivot)
0117:     print("Saved to", OUT_DIR)
0118:
0119:
0120: if __name__ == "__main__":
0121:     main()
```

Code listing: run_layerwise_pooling_ablation.py (1)

```
0001: import gc
0002: import os
0003: import time
0004: from pathlib import Path
0005:
0006: import numpy as np
0007: import pandas as pd
0008: import torch
0009: from sklearn.linear_model import LogisticRegression
0010: from sklearn.metrics import accuracy_score
0011: from sklearn.pipeline import make_pipeline
0012: from sklearn.preprocessing import StandardScaler
0013: from sklearn.svm import LinearSVC
0014: from transformers import AutoModel, AutoTokenizer
0015:
0016: import lie_llm_syntax_holdout_experiment as syntax
0017:
0018:
0019: OUT_DIR = Path(os.getenv("LAYERWISE_OUT_DIR", "results/experiments/layerwise_pooling_ablation_results"))
0020: OUT_DIR.mkdir(parents=True, exist_ok=True)
0021:
0022: MODEL_NAME = os.getenv("LAYERWISE_MODEL", "bert-base-uncased")
0023: N_BASE = int(os.getenv("LAYERWISE_N_BASE", "300"))
0024: BATCH_SIZE = int(os.getenv("LAYERWISE_BATCH_SIZE", "16"))
0025:
0026:
0027: def pool(hidden, mask, mode):
0028:     if mode == "mean":
0029:         m = mask.unsqueeze(-1).float()
0030:         return (hidden * m).sum(dim=1) / m.sum(dim=1).clamp_min(1e-9)
0031:     if mode == "cls":
0032:         return hidden[:, 0, :]
0033:     if mode == "last_token":
0034:         idx = mask.sum(dim=1).clamp_min(1) - 1
0035:         return hidden[torch.arange(hidden.shape[0]), idx]
0036:     raise ValueError(mode)
0037:
0038:
0039: def extract_layer_vectors(prompts):
0040:     tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
0041:     if tokenizer.pad_token is None:
0042:         tokenizer.pad_token = tokenizer.eos_token or tokenizer.unk_token or "[PAD]"
0043:
0044:     model = AutoModel.from_pretrained(MODEL_NAME, output_hidden_states=True)
0045:     model.eval()
0046:
0047:     rows_by_key = {}
0048:     started = time.time()
0049:
0050:     for start in range(0, len(prompts), BATCH_SIZE):
0051:         batch = prompts[start:start + BATCH_SIZE]
0052:         enc = tokenizer(batch, padding=True, truncation=True, max_length=160, return_tensors="pt")
0053:
0054:         with torch.no_grad():
```


Code listing: run_layerwise_pooling_ablation.py (2)

```
0055:         out = model(**enc)
0056:
0057:         for layer_idx, hidden in enumerate(out.hidden_states):
0058:             for pooling in ["mean", "cls", "last_token"]:
0059:                 key = (layer_idx, pooling)
0060:                 rows_by_key.setdefault(key, []).append(pool(hidden, enc["attention_mask"], pooling).cpu().numpy())
0061:
0062:         done = start + len(batch)
0063:         print(f"{MODEL_NAME}: {done}/{len(prompts)}, {time.time() - started:.1f}s")
0064:
0065:     result = {key: np.vstack(parts) for key, parts in rows_by_key.items()}
0066:
0067:     del model
0068:     gc.collect()
0069:     return result
0070:
0071:
0072: def fit_score(X, labels, train_mask, test_mask):
0073:     classifiers = {
0074:         "logreg": make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000, class_weight="balanced", n_jobs=-1)),
0075:         "linear_svc": make_pipeline(StandardScaler(), LinearSVC(class_weight="balanced", max_iter=20000)),
0076:     }
0077:
0078:     rows = []
0079:     for clf_name, clf in classifiers.items():
0080:         clf.fit(X[train_mask], labels[train_mask])
0081:         pred = clf.predict(X[test_mask])
0082:         rows.append({
0083:             "classifier": clf_name,
0084:             "accuracy": float(accuracy_score(labels[test_mask], pred)),
0085:         })
0086:     return rows
0087:
0088:
0089: def main():
0090:     base_df = syntax.make_base_rows(N_BASE)
0091:     pair_df = syntax.build_pairs(base_df)
0092:     prompts, pair_df = syntax.index_prompts(pair_df)
0093:
0094:     pair_df.to_csv(OUT_DIR / "transformation_pairs.csv", index=False)
0095:     pd.DataFrame({"prompt_id": range(len(prompts)), "prompt": prompts}).to_csv(OUT_DIR / "prompts.csv", index=False)
0096:
0097:     labels = pair_df["class"].astype(str).to_numpy()
0098:     syntax_family = pair_df["syntax_family"].astype(str).to_numpy()
0099:     train_mask = np.isin(syntax_family, ["simple_svo", "with_context"])
0100:     test_mask = np.isin(syntax_family, ["relative_clause", "temporal_clause", "reported_statement", "conditional"])
0101:     src_idx = pair_df["source_idx"].to_numpy()
0102:     tgt_idx = pair_df["target_idx"].to_numpy()
0103:
0104:     vectors_by_key = extract_layer_vectors(prompts)
0105:
0106:     all_rows = []
0107:     for (layer_idx, pooling), vectors in vectors_by_key.items():
0108:         x_src = vectors[src_idx]
```

Code listing: run_layerwise_pooling_ablation.py (3)

```
0109:         x_tgt = vectors[tgt_idx]
0110:
0111:     for rep_name, X in {
0112:         "delta": x_tgt - x_src,
0113:         "y_only": x_tgt,
0114:     }.items():
0115:         for row in fit_score(X, labels, train_mask, test_mask):
0116:             row.update({
0117:                 "model": MODEL_NAME,
0118:                 "layer": layer_idx,
0119:                 "pooling": pooling,
0120:                 "representation": rep_name,
0121:                 "n_base": N_BASE,
0122:                 "n_train": int(train_mask.sum()),
0123:                 "n_test": int(test_mask.sum()),
0124:             })
0125:             print(row)
0126:             all_rows.append(row)
0127:
0128:     result = pd.DataFrame(all_rows)
0129:     result.to_csv(OUT_DIR / "layerwise_pooling_ablation.csv", index=False)
0130:
0131:     best = (
0132:         result[result["classifier"] == "linear_svc"]
0133:         .sort_values("accuracy", ascending=False)
0134:         .head(20)
0135:     )
0136:     best.to_csv(OUT_DIR / "layerwise_pooling_ablation_top20.csv", index=False)
0137:     print(best)
0138:     print("Saved to", OUT_DIR)
0139:
0140:
0141: if __name__ == "__main__":
0142:     main()
```

Code listing: run_track1_spotcheck.py (1)

```
0001: import gc
0002: import os
0003: from pathlib import Path
0004:
0005: import numpy as np
0006: import pandas as pd
0007: from sklearn.linear_model import LogisticRegression
0008: from sklearn.metrics import accuracy_score
0009: from sklearn.pipeline import make_pipeline
0010: from sklearn.preprocessing import StandardScaler
0011: from sklearn.svm import LinearSVC
0012:
0013: import lie_llm_full_semantic_holdout_experiment as full
0014:
0015:
0016: OUT_DIR = Path(os.getenv("SPOTCHECK_OUT_DIR", "results/experiments/track1_spotcheck_results"))
0017: OUT_DIR.mkdir(parents=True, exist_ok=True)
0018:
0019: MODELS = [m.strip() for m in os.getenv("SPOTCHECK_MODELS", "bert-large-uncased").split(",") if m.strip()]
0020: N_BASE = int(os.getenv("SPOTCHECK_N_BASE", "100"))
0021:
0022:
0023: def run_classifiers(X, labels, split):
0024:     train_mask = split == "train"
0025:     test_mask = split == "test"
0026:     classifiers = {
0027:         "logreg": make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000, class_weight="balanced", n_jobs=-1)),
0028:         "linear_svc": make_pipeline(StandardScaler(), LinearSVC(class_weight="balanced", max_iter=20000)),
0029:     }
0030:     rows = []
0031:     for clf_name, clf in classifiers.items():
0032:         clf.fit(X[train_mask], labels[train_mask])
0033:         pred = clf.predict(X[test_mask])
0034:         rows.append({
0035:             "classifier": clf_name,
0036:             "accuracy": float(accuracy_score(labels[test_mask], pred)),
0037:             "n_train": int(train_mask.sum()),
0038:             "n_test": int(test_mask.sum()),
0039:         })
0040:     return rows
0041:
0042:
0043: def main():
0044:     full.OUT_DIR = str(OUT_DIR)
0045:     full.N_BASE = N_BASE
0046:     os.makedirs(full.OUT_DIR, exist_ok=True)
0047:
0048:     pair_df = full.build_dataset()
0049:     prompts, pair_df = full.index_prompts(pair_df)
0050:     pair_df.to_csv(OUT_DIR / "transformation_pairs.csv", index=False)
0051:     pd.DataFrame({"prompt_id": range(len(prompts)), "prompt": prompts}).to_csv(OUT_DIR / "prompts.csv", index=False)
0052:
0053:     labels = pair_df["class"].astype(str).to_numpy()
0054:     split = pair_df["split"].astype(str).to_numpy()
```

Code listing: run_track1_spotcheck.py (2)

```
0055:     src_idx = pair_df["source_idx"].to_numpy()
0056:     tgt_idx = pair_df["target_idx"].to_numpy()
0057:
0058:     rows = []
0059:     for model_name in MODELS:
0060:         print(f"\n=== MODEL: {model_name} ===")
0061:         vectors = full.get_vectors(model_name, prompts)
0062:         x_src = vectors[src_idx]
0063:         x_tgt = vectors[tgt_idx]
0064:         reps = {
0065:             "x_only": x_src,
0066:             "y_only": x_tgt,
0067:             "delta": x_tgt - x_src,
0068:             "concat": np.concatenate([x_src, x_tgt], axis=1),
0069:         }
0070:         for rep_name, X in reps.items():
0071:             for row in run_classifiers(X, labels, split):
0072:                 row.update({
0073:                     "model": model_name,
0074:                     "representation": rep_name,
0075:                     "random_baseline": 1 / len(set(labels)),
0076:                     "n_base": N_BASE,
0077:                 })
0078:                 print(model_name, rep_name, row["classifier"], f"{row['accuracy']:.4f}")
0079:                 rows.append(row)
0080:     del vectors, reps
0081:     gc.collect()
0082:
0083:     result = pd.DataFrame(rows)
0084:     result.to_csv(OUT_DIR / "spotcheck_representation_ablation.csv", index=False)
0085:     pivot = result.pivot_table(index=["model", "classifier"], columns="representation", values="accuracy").reset_index()
0086:     pivot.to_csv(OUT_DIR / "spotcheck_representation_ablation_pivot.csv", index=False)
0087:     print(pivot)
0088:
0089:
0090: if __name__ == "__main__":
0091:     main()
```

Code listing: run_full_semantic_pooling_ablation.py (1)

```
0001: import gc
0002: import os
0003: import random
0004: from pathlib import Path
0005:
0006: import numpy as np
0007: import pandas as pd
0008: import torch
0009: from sklearn.linear_model import LogisticRegression
0010: from sklearn.metrics import accuracy_score
0011: from sklearn.pipeline import make_pipeline
0012: from sklearn.preprocessing import StandardScaler
0013: from sklearn.svm import LinearSVC
0014: from transformers import AutoModel, AutoTokenizer
0015:
0016: import lie_llm_full_semantic_holdout_experiment as full
0017:
0018:
0019: OUT_DIR = Path(os.getenv("POOLING_OUT_DIR", "results/experiments/full_semantic_pooling_ablation_results"))
0020: MODELS = [m.strip() for m in os.getenv("POOLING_MODELS", "bert-base-uncased,roberta-base,gpt2").split(",") if m.strip()]
0021: N_BASE = int(os.getenv("POOLING_N_BASE", "150"))
0022: BATCH_SIZE = int(os.getenv("POOLING_BATCH_SIZE", "32"))
0023: SEED = int(os.getenv("POOLING_SEED", "42"))
0024:
0025:
0026: def reset_full_dataset_seed():
0027:     full.N_BASE = N_BASE
0028:     full.random.seed(SEED)
0029:     full.np.random.seed(SEED)
0030:     random.seed(SEED)
0031:     np.random.seed(SEED)
0032:     torch.manual_seed(SEED)
0033:
0034:
0035: def safe_name(model_name):
0036:     return model_name.replace("/", "__")
0037:
0038:
0039: def last_token_pool(hidden, attention_mask):
0040:     lengths = attention_mask.sum(dim=1).clamp(min=1) - 1
0041:     batch_idx = torch.arange(hidden.shape[0], device=hidden.device)
0042:     return hidden[batch_idx, lengths]
0043:
0044:
0045: def get_pooled_vectors(model_name, prompts):
0046:     tokenizer = AutoTokenizer.from_pretrained(model_name)
0047:     if tokenizer.pad_token is None:
0048:         tokenizer.pad_token = tokenizer.eos_token or tokenizer.unk_token or "[PAD]"
0049:
0050:     model = AutoModel.from_pretrained(model_name, output_hidden_states=True)
0051:     model.eval()
0052:
0053:     pooled = {"mean": [], "cls": [], "last_token": []}
0054:
```

Code listing: run_full_semantic_pooling_ablation.py (2)

```
0055:     for start in range(0, len(prompts), BATCH_SIZE):
0056:         batch = prompts[start:start + BATCH_SIZE]
0057:         inputs = tokenizer(
0058:             batch,
0059:             padding=True,
0060:             truncation=True,
0061:             max_length=180,
0062:             return_tensors="pt",
0063:         )
0064:
0065:         with torch.no_grad():
0066:             out = model(**inputs)
0067:
0068:             hidden = out.hidden_states[-1]
0069:             mask = inputs["attention_mask"]
0070:
0071:             pooled["mean"].append(full.mean_pool(hidden, mask).cpu().numpy())
0072:             pooled["cls"].append(hidden[:, 0, :].cpu().numpy())
0073:             pooled["last_token"].append(last_token_pool(hidden, mask).cpu().numpy())
0074:
0075:             done = start + len(batch)
0076:             print(f"  {model_name}: {done}/{len(prompts)}", flush=True)
0077:
0078:     del model
0079:     gc.collect()
0080:
0081:     return {k: np.vstack(v) for k, v in pooled.items()}
0082:
0083:
0084: def build_features(vectors, pair_df, representation):
0085:     src = vectors[pair_df["source_idx"].to_numpy()]
0086:     tgt = vectors[pair_df["target_idx"].to_numpy()]
0087:
0088:     if representation == "x_only":
0089:         return src
0090:     if representation == "y_only":
0091:         return tgt
0092:     if representation == "delta":
0093:         return tgt - src
0094:     if representation == "concat":
0095:         return np.hstack([src, tgt])
0096:     raise ValueError(representation)
0097:
0098:
0099: def run_one(model_name, pooling, representation, vectors, pair_df):
0100:     X = build_features(vectors, pair_df, representation)
0101:     y = pair_df["class"].astype(str).to_numpy()
0102:     split = pair_df["split"].astype(str).to_numpy()
0103:
0104:     train = split == "train"
0105:     test = split == "test"
0106:
0107:     classifiers = {
0108:         "logreg": make_pipeline(
```

Code listing: run_full_semantic_pooling_ablation.py (3)

```
0109:         StandardScaler(),
0110:         LogisticRegression(max_iter=5000, class_weight="balanced"),
0111:     ),
0112:     "linear_svc": make_pipeline(
0113:         StandardScaler(),
0114:         LinearSVC(class_weight="balanced", max_iter=20000),
0115:     ),
0116: }
0117:
0118: rows = []
0119: for clf_name, clf in classifiers.items():
0120:     clf.fit(X[train], y[train])
0121:     pred = clf.predict(X[test])
0122:     rows.append({
0123:         "model": model_name,
0124:         "pooling": pooling,
0125:         "representation": representation,
0126:         "classifier": clf_name,
0127:         "accuracy": float(accuracy_score(y[test], pred)),
0128:         "n_base": N_BASE,
0129:         "n_train": int(train.sum()),
0130:         "n_test": int(test.sum()),
0131:     })
0132: return rows
0133:
0134:
0135: def main():
0136:     OUT_DIR.mkdir(parents=True, exist_ok=True)
0137:     reset_full_dataset_seed()
0138:
0139:     pair_df = full.build_dataset()
0140:     prompts, pair_df = full.index_prompts(pair_df)
0141:     pair_df.to_csv(OUT_DIR / "transformation_pairs.csv", index=False)
0142:     pd.DataFrame({"prompt_id": range(len(prompts)), "prompt": prompts}).to_csv(OUT_DIR / "prompts.csv", index=False)
0143:
0144:     rows = []
0145:     for model_name in MODELS:
0146:         print(f"\n=== MODEL: {model_name} ===", flush=True)
0147:         pooled_vectors = get_pooled_vectors(model_name, prompts)
0148:
0149:         for pooling, vectors in pooled_vectors.items():
0150:             for representation in ["x_only", "y_only", "concat", "delta"]:
0151:                 rows.extend(run_one(model_name, pooling, representation, vectors, pair_df))
0152:
0153:     result = pd.DataFrame(rows)
0154:     result.to_csv(OUT_DIR / "full_semantic_pooling_ablation.csv", index=False)
0155:     pivot = result.pivot_table(
0156:         index=["model", "pooling", "classifier"],
0157:         columns="representation",
0158:         values="accuracy",
0159:     ).reset_index()
0160:     pivot.to_csv(OUT_DIR / "full_semantic_pooling_ablation_pivot.csv", index=False)
0161:     print(pivot)
0162:
```

Code listing: run_full_semantic_pooling_ablation.py (4)

```
0163:
0164: if __name__ == "__main__":
0165:     main()
```


Code listing: run_upat_procrustes_nulls.py (1)

```
0001: import argparse
0002: import os
0003: import sys
0004: from pathlib import Path
0005:
0006: import matplotlib.pyplot as plt
0007: import numpy as np
0008: import pandas as pd
0009: from scipy.linalg import svd
0010: from sklearn.metrics import accuracy_score, f1_score
0011: from sklearn.preprocessing import normalize
0012:
0013: from run_upat_large import UPATAudit
0014:
0015:
0016: OUT_DIR = Path("results/experiments/upat_large_results")
0017: CSV_DIR = OUT_DIR / "csv"
0018: FIG_DIR = OUT_DIR / "figures"
0019: CSV_DIR.mkdir(parents=True, exist_ok=True)
0020: FIG_DIR.mkdir(parents=True, exist_ok=True)
0021:
0022: N_NULL = int(os.getenv("UPAT_PROCRUSTES_NULL_REPEATS", "30"))
0023: SEED = int(os.getenv("UPAT_PROCRUSTES_NULL_SEED", "1729"))
0024:
0025:
0026: def parse_args():
0027:     parser = argparse.ArgumentParser(
0028:         description="Run UPAT cross-model Procrustes null controls."
0029:     )
0030:     parser.add_argument(
0031:         "--n-null",
0032:         type=int,
0033:         default=N_NULL,
0034:         help="Number of null repeats per source-target direction and null type.",
0035:     )
0036:     parser.add_argument(
0037:         "--seed",
0038:         type=int,
0039:         default=SEED,
0040:         help="Base random seed for null controls.",
0041:     )
0042:     return parser.parse_args()
0043:
0044:
0045: def make_direction_context(audit, train_model, test_model):
0046:     source_sp = audit.spaces[train_model]
0047:     target_sp = audit.spaces[test_model]
0048:
0049:     common_texts = sorted(set(source_sp["texts"]) & set(target_sp["texts"]))
0050:     source_idx = np.array([source_sp["idx"][text] for text in common_texts])
0051:     target_idx = np.array([target_sp["idx"][text] for text in common_texts])
0052:
0053:     min_dim = min(source_sp["raw"].shape[1], target_sp["raw"].shape[1])
0054:     source_anchor = source_sp["raw"][source_idx, :min_dim]
```

Code listing: run_upat_procrustes_nulls.py (2)

```
0055:     target_anchor = target_sp["raw"][target_idx, :min_dim]
0056:     source_all = source_sp["raw"][:, :min_dim]
0057:     target_all = target_sp["raw"][:, :min_dim]
0058:
0059:     source_mean = source_anchor.mean(axis=0, keepdims=True)
0060:     target_mean = target_anchor.mean(axis=0, keepdims=True)
0061:
0062:     source_anchor_c = source_anchor - source_mean
0063:     target_anchor_c = target_anchor - target_mean
0064:     source_all_c = source_all - source_mean
0065:     target_all_c = target_all - target_mean
0066:
0067:     return {
0068:         "source_anchor_c": source_anchor_c,
0069:         "target_anchor_c": target_anchor_c,
0070:         "source_all_c": source_all_c,
0071:         "target_all_c": target_all_c,
0072:     }
0073:
0074:
0075: def apply_fast_procrustes(context, target_order=None):
0076:     target_anchor_c = context["target_anchor_c"]
0077:     if target_order is not None:
0078:         target_anchor_c = target_anchor_c[target_order]
0079:
0080:     u, _, vt = svd(context["source_anchor_c"].T @ target_anchor_c, full_matrices=False, check_finite=False)
0081:     r = u @ vt
0082:     return context["source_all_c"] @ r, context["target_all_c"]
0083:
0084:
0085: def apply_fast_random_orthogonal(context, seed):
0086:     rng = np.random.default_rng(seed)
0087:     dim = context["source_all_c"].shape[1]
0088:     q, _ = np.linalg.qr(rng.normal(size=(dim, dim)))
0089:     return context["source_all_c"] @ q, context["target_all_c"]
0090:
0091:
0092: def evaluate_transfer(audit, train_model, test_model, context, *, pairing="matched", shuffle_labels=False, random_orthogonal=F
0093:     rng = np.random.default_rng(seed)
0094:
0095:     if pairing == "random_pairing":
0096:         target_order = rng.permutation(len(context["target_anchor_c"]))
0097:     elif pairing != "matched":
0098:         raise ValueError(f"Unknown pairing mode: {pairing}")
0099:     else:
0100:         target_order = None
0101:
0102:     if random_orthogonal:
0103:         source_aligned, target_aligned = apply_fast_random_orthogonal(context, seed=seed)
0104:     else:
0105:         source_aligned, target_aligned = apply_fast_procrustes(context, target_order=target_order)
0106:
0107:     source_deltas = normalize(audit.make_raw_deltas(train_model, source_aligned))
0108:     target_deltas = normalize(audit.make_raw_deltas(test_model, target_aligned))
```

Code listing: run_upat_procrustes_nulls.py (3)

```
0109:
0110:     y_train = audit.y[audit.train_mask].copy()
0111:     if shuffle_labels:
0112:         y_train = rng.permutation(y_train)
0113:
0114:     pred = audit.fit_predict(
0115:         source_deltas[audit.train_mask],
0116:         y_train,
0117:         target_deltas[audit.test_mask],
0118:         seed=seed,
0119:     )
0120:
0121:     return {
0122:         "acc": accuracy_score(audit.y[audit.test_mask], pred),
0123:         "f1": f1_score(audit.y[audit.test_mask], pred, average="macro"),
0124:     }
0125:
0126:
0127: def run_nulls(audit):
0128:     observed = pd.read_csv(CSV_DIR / "cross_model_procrustes.csv")
0129:     rows = []
0130:     summary_rows = []
0131:
0132:     for obs in observed.itertuples(index=False):
0133:         train_model = obs.train_model
0134:         test_model = obs.test_model
0135:         if train_model == test_model:
0136:             continue
0137:
0138:         print(f"\n\nNulls: {train_model} -> {test_model}", flush=True)
0139:         context = make_direction_context(audit, train_model, test_model)
0140:
0141:         for null_type in ["random_pairing", "random_labels", "random_orthogonal"]:
0142:             f1_values = []
0143:             acc_values = []
0144:
0145:             for repeat in range(N_NULL):
0146:                 seed = SEED + repeat
0147:                 if null_type == "random_pairing":
0148:                     metrics = evaluate_transfer(
0149:                         audit,
0150:                         train_model,
0151:                         test_model,
0152:                         context,
0153:                         pairing="random_pairing",
0154:                         shuffle_labels=False,
0155:                         seed=seed,
0156:                     )
0157:                 else:
0158:                     metrics = evaluate_transfer(
0159:                         audit,
0160:                         train_model,
0161:                         test_model,
0162:                         context,
```

Code listing: run_upat_procrustes_nulls.py (4)

```
0163:         pairing="matched",
0164:         shuffle_labels=null_type == "random_labels",
0165:         random_orthogonal=null_type == "random_orthogonal",
0166:         seed=seed,
0167:     )
0168:
0169:     f1_values.append(metrics["f1"])
0170:     acc_values.append(metrics["acc"])
0171:     rows.append({
0172:         "train_model": train_model,
0173:         "test_model": test_model,
0174:         "null_type": null_type,
0175:         "repeat": repeat,
0176:         "null_acc": metrics["acc"],
0177:         "null_f1": metrics["f1"],
0178:         "observed_aligned_f1": obs.aligned_f1,
0179:         "observed_raw_f1": obs.raw_f1,
0180:         "observed_gain_f1": obs.gain_f1,
0181:     })
0182:
0183:     if (repeat + 1) % 100 == 0 or repeat + 1 == N_NULL:
0184:         print(f"  {null_type}: {repeat + 1}/{N_NULL}", flush=True)
0185:
0186:     f1_arr = np.array(f1_values)
0187:     acc_arr = np.array(acc_values)
0188:     summary_rows.append({
0189:         "train_model": train_model,
0190:         "test_model": test_model,
0191:         "null_type": null_type,
0192:         "n_null": N_NULL,
0193:         "observed_raw_f1": obs.raw_f1,
0194:         "observed_aligned_f1": obs.aligned_f1,
0195:         "observed_gain_f1": obs.gain_f1,
0196:         "null_mean_f1": f1_arr.mean(),
0197:         "null_std_f1": f1_arr.std(ddof=1) if len(f1_arr) > 1 else 0.0,
0198:         "null_max_f1": f1_arr.max(),
0199:         "null_mean_acc": acc_arr.mean(),
0200:         "p_empirical_null_ge_observed": ((f1_arr >= obs.aligned_f1).sum() + 1) / (N_NULL + 1),
0201:         "observed_minus_null_mean_f1": obs.aligned_f1 - f1_arr.mean(),
0202:     })
0203:
0204:     pd.DataFrame(rows).to_csv(CSV_DIR / "procrustes_null_raw.partial.csv", index=False)
0205:     pd.DataFrame(summary_rows).to_csv(CSV_DIR / "procrustes_null_summary.partial.csv", index=False)
0206:
0207:     raw_df = pd.DataFrame(rows)
0208:     summary_df = pd.DataFrame(summary_rows)
0209:
0210:     raw_df.to_csv(CSV_DIR / "procrustes_null_raw.csv", index=False)
0211:     summary_df.to_csv(CSV_DIR / "procrustes_null_summary.csv", index=False)
0212:     (CSV_DIR / "procrustes_null_raw.partial.csv").unlink(missing_ok=True)
0213:     (CSV_DIR / "procrustes_null_summary.partial.csv").unlink(missing_ok=True)
0214:     return raw_df, summary_df
0215:
0216:
```

Code listing: run_upat_procrustes_nulls.py (5)

```
0217: def plot_summary(summary_df):
0218:     if summary_df.empty:
0219:         return
0220:
0221:     for null_type, df in summary_df.groupby("null_type"):
0222:         labels = [
0223:             f"{row.train_model.split('/')[-1]}\n->{row.test_model.split('/')[-1]}"
0224:             for row in df.itertuples(index=False)
0225:         ]
0226:         x = np.arange(len(df))
0227:
0228:         plt.figure(figsize=(max(12, len(df) * 0.45), 6))
0229:         plt.bar(x, df["observed_aligned_f1"], label="observed aligned F1")
0230:         plt.bar(x, df["null_mean_f1"], alpha=0.65, label=f"{null_type} mean F1")
0231:         plt.xticks(x, labels, rotation=70, ha="right", fontsize=7)
0232:         plt.ylabel("Macro F1")
0233:         plt.title(f"UPAT Procrustes null baseline: {null_type}")
0234:         plt.legend()
0235:         plt.tight_layout()
0236:         plt.savefig(FIG_DIR / f"l0_procrustes_null_{null_type}.png", dpi=220, bbox_inches="tight")
0237:         plt.close()
0238:
0239:
0240: def main():
0241:     global N_NULL, SEED
0242:     args = parse_args()
0243:     N_NULL = args.n_null
0244:     SEED = args.seed
0245:     print(f"Procrustes null repeats: {N_NULL}; seed: {SEED}", flush=True)
0246:
0247:     audit = UPATAudit()
0248:     audit.build_dataset()
0249:     audit.extract_all_spaces()
0250:
0251:     cross_path = CSV_DIR / "cross_model_procrustes.csv"
0252:     if not cross_path.exists():
0253:         audit.run_cross_model_procrustes()
0254:
0255:     _, summary_df = run_nulls(audit)
0256:     plot_summary(summary_df)
0257:     print(f"\nSaved: {CSV_DIR / 'procrustes_null_raw.csv'}")
0258:     print(f"Saved: {CSV_DIR / 'procrustes_null_summary.csv'}")
0259:
0260:
0261: if __name__ == "__main__":
0262:     sys.exit(main())
```

Code listing: run_upat_alignment_size_heldout.py (1)

```
0001: import argparse
0002: import sys
0003: from pathlib import Path
0004:
0005: import matplotlib.pyplot as plt
0006: import numpy as np
0007: import pandas as pd
0008: from scipy.linalg import svd
0009: from sklearn.metrics import accuracy_score, f1_score
0010: from sklearn.preprocessing import normalize
0011:
0012: from run_upat_large import UPATAudit
0013: from upat_dataset import UPATDataset, UPATDatasetConfig
0014:
0015:
0016: OUT_DIR = Path("results/experiments/upat_large_results")
0017: CSV_DIR = OUT_DIR / "csv"
0018: FIG_DIR = OUT_DIR / "figures"
0019: CSV_DIR.mkdir(parents=True, exist_ok=True)
0020: FIG_DIR.mkdir(parents=True, exist_ok=True)
0021:
0022:
0023: def parse_args():
0024:     parser = argparse.ArgumentParser(
0025:         description="Run held-out anchor alignment-size curves for UPAT cross-model Procrustes transfer."
0026:     )
0027:     parser.add_argument(
0028:         "--sizes",
0029:         default="25,50,100,250,500,1000",
0030:         help="Comma-separated held-out alignment anchor sizes.",
0031:     )
0032:     parser.add_argument(
0033:         "--repeats",
0034:         type=int,
0035:         default=10,
0036:         help="Repeats per source-target direction and alignment size.",
0037:     )
0038:     parser.add_argument(
0039:         "--anchor-target-count",
0040:         type=int,
0041:         default=1200,
0042:         help="Number of unique held-out anchor texts to generate before subsampling.",
0043:     )
0044:     parser.add_argument(
0045:         "--seed",
0046:         type=int,
0047:         default=31415,
0048:         help="Base random seed.",
0049:     )
0050:     return parser.parse_args()
0051:
0052:
0053: def collect_texts(df):
0054:     return set(df["source"]) | set(df["target"])
```

Code listing: run_upat_alignment_size_heldout.py (2)

```
0055:
0056:
0057: def build_heldout_anchor_texts(main_texts, target_count, seed):
0058:     anchor_texts = []
0059:     seen = set(main_texts)
0060:
0061:     for offset in range(1000):
0062:         cfg = UPATDatasetConfig(
0063:             train_templates=200,
0064:             test_templates=200,
0065:             seed_train=seed + 2 * offset,
0066:             seed_test=seed + 2 * offset + 1,
0067:         )
0068:         df = UPATDataset(cfg).build()
0069:
0070:         for text in sorted(collect_texts(df)):
0071:             if text in seen:
0072:                 continue
0073:             seen.add(text)
0074:             anchor_texts.append(text)
0075:             if len(anchor_texts) >= target_count:
0076:                 return anchor_texts
0077:
0078:         raise RuntimeError(
0079:             f"Only generated {len(anchor_texts)} held-out anchor texts; requested {target_count}."
0080:         )
0081:
0082:
0083: def fit_procrustes(source_anchor, target_anchor, source_all, target_all):
0084:     source_mean = source_anchor.mean(axis=0, keepdims=True)
0085:     target_mean = target_anchor.mean(axis=0, keepdims=True)
0086:
0087:     source_anchor_c = source_anchor - source_mean
0088:     target_anchor_c = target_anchor - target_mean
0089:     source_all_c = source_all - source_mean
0090:     target_all_c = target_all - target_mean
0091:
0092:     min_dim = min(
0093:         source_anchor_c.shape[1],
0094:         target_anchor_c.shape[1],
0095:         source_all_c.shape[1],
0096:         target_all_c.shape[1],
0097:     )
0098:     source_anchor_c = source_anchor_c[:, :min_dim]
0099:     target_anchor_c = target_anchor_c[:, :min_dim]
0100:     source_all_c = source_all_c[:, :min_dim]
0101:     target_all_c = target_all_c[:, :min_dim]
0102:
0103:     u, _, vt = svd(source_anchor_c.T @ target_anchor_c, full_matrices=False, check_finite=False)
0104:     r = u @ vt
0105:     return source_all_c @ r, target_all_c
0106:
0107:
0108: def evaluate_direction(audit, train_model, test_model, source_anchor, target_anchor, seed):
```

Code listing: run_upat_alignment_size_heldout.py (3)

```
0109:     source_sp = audit.spaces[train_model]
0110:     target_sp = audit.spaces[test_model]
0111:
0112:     source_aligned, target_aligned = fit_procrustes(
0113:         source_anchor=source_anchor,
0114:         target_anchor=target_anchor,
0115:         source_all=source_sp["raw"],
0116:         target_all=target_sp["raw"],
0117:     )
0118:
0119:     source_deltas = normalize(audit.make_raw_deltas(train_model, source_aligned))
0120:     target_deltas = normalize(audit.make_raw_deltas(test_model, target_aligned))
0121:
0122:     pred = audit.fit_predict(
0123:         source_deltas[audit.train_mask],
0124:         audit.y[audit.train_mask],
0125:         target_deltas[audit.test_mask],
0126:         seed=seed,
0127:     )
0128:
0129:     return {
0130:         "acc": accuracy_score(audit.y[audit.test_mask], pred),
0131:         "f1": f1_score(audit.y[audit.test_mask], pred, average="macro"),
0132:     }
0133:
0134:
0135: def run_curve(audit, anchor_spaces, anchor_texts, sizes, repeats, seed):
0136:     observed = pd.read_csv(CSV_DIR / "cross_model_procrustes.csv")
0137:     observed = observed[observed["train_model"] != observed["test_model"]]
0138:     rng = np.random.default_rng(seed)
0139:     rows = []
0140:
0141:     for obs in observed.itertuples(index=False):
0142:         train_model = obs.train_model
0143:         test_model = obs.test_model
0144:         print(f"\nHeld-out alignment: {train_model} -> {test_model}", flush=True)
0145:
0146:         source_anchor_all = anchor_spaces[train_model]
0147:         target_anchor_all = anchor_spaces[test_model]
0148:
0149:         for size in sizes:
0150:             real_size = min(size, len(anchor_texts))
0151:             for repeat in range(repeats):
0152:                 sample = rng.choice(len(anchor_texts), size=real_size, replace=False)
0153:                 metrics = evaluate_direction(
0154:                     audit=audit,
0155:                     train_model=train_model,
0156:                     test_model=test_model,
0157:                     source_anchor=source_anchor_all[sample],
0158:                     target_anchor=target_anchor_all[sample],
0159:                     seed=seed + 1000 * repeat + real_size,
0160:                 )
0161:                 rows.append({
0162:                     "train_model": train_model,
```


Code listing: run_upat_alignment_size_heldout.py (4)

```
0163:         "test_model": test_model,
0164:         "alignment_size": real_size,
0165:         "repeat": repeat,
0166:         "heldout_anchor_pool_size": len(anchor_texts),
0167:         "observed_full_anchor_f1": obs.aligned_f1,
0168:         "observed_raw_f1": obs.raw_f1,
0169:         "heldout_acc": metrics["acc"],
0170:         "heldout_f1": metrics["f1"],
0171:         "heldout_minus_raw_f1": metrics["f1"] - obs.raw_f1,
0172:         "heldout_minus_full_anchor_f1": metrics["f1"] - obs.aligned_f1,
0173:     })
0174:
0175:     print(f"    size {real_size}: {repeats}/{repeats}", flush=True)
0176:
0177:     pd.DataFrame(rows).to_csv(CSV_DIR / "heldout_alignment_curve_raw.partial.csv", index=False)
0178:
0179:     raw_df = pd.DataFrame(rows)
0180:     summary_df = (
0181:         raw_df
0182:         .groupby("alignment_size")
0183:         .agg(
0184:             mean_f1=("heldout_f1", "mean"),
0185:             std_f1=("heldout_f1", "std"),
0186:             mean_acc=("heldout_acc", "mean"),
0187:             mean_full_anchor_f1=("observed_full_anchor_f1", "mean"),
0188:             mean_raw_f1=("observed_raw_f1", "mean"),
0189:             mean_gap_to_full_anchor=("heldout_minus_full_anchor_f1", "mean"),
0190:             mean_gain_over_raw=("heldout_minus_raw_f1", "mean"),
0191:             n=("heldout_f1", "count"),
0192:         )
0193:         .reset_index()
0194:     )
0195:     direction_summary_df = (
0196:         raw_df
0197:         .groupby(["train_model", "test_model", "alignment_size"])
0198:         .agg(
0199:             mean_f1=("heldout_f1", "mean"),
0200:             std_f1=("heldout_f1", "std"),
0201:             mean_acc=("heldout_acc", "mean"),
0202:             observed_full_anchor_f1=("observed_full_anchor_f1", "first"),
0203:             observed_raw_f1=("observed_raw_f1", "first"),
0204:             mean_gap_to_full_anchor=("heldout_minus_full_anchor_f1", "mean"),
0205:             mean_gain_over_raw=("heldout_minus_raw_f1", "mean"),
0206:             n=("heldout_f1", "count"),
0207:         )
0208:         .reset_index()
0209:     )
0210:
0211:     raw_df.to_csv(CSV_DIR / "heldout_alignment_curve_raw.csv", index=False)
0212:     summary_df.to_csv(CSV_DIR / "heldout_alignment_curve_summary.csv", index=False)
0213:     direction_summary_df.to_csv(CSV_DIR / "heldout_alignment_curve_by_direction.csv", index=False)
0214:     (CSV_DIR / "heldout_alignment_curve_raw.partial.csv").unlink(missing_ok=True)
0215:     return raw_df, summary_df, direction_summary_df
0216:
```

Code listing: run_upat_alignment_size_heldout.py (5)

```
0217:
0218: def plot_curve(summary_df, direction_summary_df):
0219:     plt.figure(figsize=(8, 5))
0220:     ordered = summary_df.sort_values("alignment_size")
0221:     plt.errorbar(
0222:         ordered["alignment_size"],
0223:         ordered["mean_f1"],
0224:         yerr=ordered["std_f1"],
0225:         marker="o",
0226:         capsize=4,
0227:         label="held-out anchor Procrustes",
0228:     )
0229:     plt.plot(
0230:         ordered["alignment_size"],
0231:         ordered["mean_raw_f1"],
0232:         linestyle="--",
0233:         label="raw cross-model mean",
0234:     )
0235:     plt.plot(
0236:         ordered["alignment_size"],
0237:         ordered["mean_full_anchor_f1"],
0238:         linestyle=":",
0239:         label="full-anchor Procrustes mean",
0240:     )
0241:     plt.xscale("log")
0242:     plt.xlabel("Held-out alignment anchor texts")
0243:     plt.ylabel("Macro F1")
0244:     plt.title("UPAT held-out alignment-size curve")
0245:     plt.grid(True, which="both", alpha=0.35)
0246:     plt.legend()
0247:     plt.tight_layout()
0248:     plt.savefig(FIG_DIR / "11_heldout_alignment_size_curve.png", dpi=220, bbox_inches="tight")
0249:     plt.close()
0250:
0251:     largest_size = direction_summary_df["alignment_size"].max()
0252:     largest = direction_summary_df[direction_summary_df["alignment_size"] == largest_size].copy()
0253:     labels = [
0254:         f"{row.train_model.split('/')[0]}\\n->{row.test_model.split('/')[0]}"
0255:         for row in largest.itertuples(index=False)
0256:     ]
0257:     x = np.arange(len(largest))
0258:     plt.figure(figsize=(max(12, len(largest) * 0.45), 6))
0259:     plt.bar(x, largest["observed_raw_f1"], label="raw F1")
0260:     plt.bar(x, largest["mean_f1"], alpha=0.75, label=f"held-out alignment F1, n={largest_size}")
0261:     plt.bar(x, largest["observed_full_anchor_f1"], alpha=0.45, label="full-anchor alignment F1")
0262:     plt.xticks(x, labels, rotation=70, ha="right", fontsize=7)
0263:     plt.ylabel("Macro F1")
0264:     plt.title("UPAT held-out alignment by direction")
0265:     plt.legend()
0266:     plt.tight_layout()
0267:     plt.savefig(FIG_DIR / "11_heldout_alignment_by_direction.png", dpi=220, bbox_inches="tight")
0268:     plt.close()
0269:
0270:
```

Code listing: run_upat_alignment_size_heldout.py (6)

```
0271: def main():
0272:     args = parse_args()
0273:     sizes = [int(x.strip()) for x in args.sizes.split(",") if x.strip()]
0274:     if not sizes:
0275:         raise ValueError("At least one alignment size is required.")
0276:
0277:     audit = UPATAudit()
0278:     audit.build_dataset()
0279:
0280:     main_texts = collect_texts(audit.df)
0281:     anchor_target_count = max(args.anchor_target_count, max(sizes))
0282:     anchor_texts = build_heldout_anchor_texts(
0283:         main_texts=main_texts,
0284:         target_count=anchor_target_count,
0285:         seed=args.seed,
0286:     )
0287:     pd.DataFrame({"text": anchor_texts}).to_csv(CSV_DIR / "heldout_alignment_anchor_texts.csv", index=False)
0288:     print(f"Held-out anchor texts: {len(anchor_texts)}", flush=True)
0289:
0290:     audit.extract_all_spaces()
0291:
0292:     cross_path = CSV_DIR / "cross_model_procrustes.csv"
0293:     if not cross_path.exists():
0294:         audit.run_cross_model_procrustes()
0295:
0296:     anchor_spaces = {}
0297:     for model_name in audit.procrustes_models:
0298:         print(f"\nEXTRACTING HELD-OUT ANCHORS {model_name}", flush=True)
0299:         anchor_spaces[model_name] = audit.get_embeddings(model_name, anchor_texts)
0300:
0301:     _, summary_df, direction_summary_df = run_curve(
0302:         audit=audit,
0303:         anchor_spaces=anchor_spaces,
0304:         anchor_texts=anchor_texts,
0305:         sizes=sizes,
0306:         repeats=args.repeats,
0307:         seed=args.seed,
0308:     )
0309:     plot_curve(summary_df, direction_summary_df)
0310:     print(f"\nSaved: {CSV_DIR / 'heldout_alignment_curve_raw.csv'}")
0311:     print(f"Saved: {CSV_DIR / 'heldout_alignment_curve_summary.csv'}")
0312:     print(f"Saved: {CSV_DIR / 'heldout_alignment_curve_by_direction.csv'}")
0313:
0314:
0315: if __name__ == "__main__":
0316:     sys.exit(main())
```

Code listing: run_upat_rise_aware_comparison.py (1)

```
0001: import argparse
0002: import sys
0003: from pathlib import Path
0004:
0005: import matplotlib.pyplot as plt
0006: import numpy as np
0007: import pandas as pd
0008: from scipy.linalg import svd
0009: from sklearn.metrics import accuracy_score, f1_score
0010: from sklearn.preprocessing import normalize
0011:
0012: from run_upat_large import UPATAudit
0013:
0014:
0015: OUT_DIR = Path("results/experiments/upat_large_results")
0016: CSV_DIR = OUT_DIR / "csv"
0017: FIG_DIR = OUT_DIR / "figures"
0018: CSV_DIR.mkdir(parents=True, exist_ok=True)
0019: FIG_DIR.mkdir(parents=True, exist_ok=True)
0020:
0021:
0022: def parse_args():
0023:     parser = argparse.ArgumentParser(
0024:         description="Compare delta, MDV, and RISE-style prototype baselines on UPAT."
0025:     )
0026:     parser.add_argument("--seed", type=int, default=2718)
0027:     return parser.parse_args()
0028:
0029:
0030: def unit(x):
0031:     return normalize(x)
0032:
0033:
0034: def cosine_rows(a, b):
0035:     return np.sum(unit(a) * unit(b), axis=1)
0036:
0037:
0038: def log_map_sphere(x, y, eps=1e-7):
0039:     dot = np.clip(np.sum(x * y, axis=1, keepdims=True), -1 + eps, 1 - eps)
0040:     theta = np.arccos(dot)
0041:     tangent = y - dot * x
0042:     sin_theta = np.sin(theta)
0043:     return tangent * (theta / np.maximum(sin_theta, eps))
0044:
0045:
0046: def exp_map_sphere(x, tangent, eps=1e-7):
0047:     norm = np.linalg.norm(tangent, axis=1, keepdims=True)
0048:     direction = tangent / np.maximum(norm, eps)
0049:     out = np.cos(norm) * x + np.sin(norm) * direction
0050:     small = norm[:, 0] < eps
0051:     if np.any(small):
0052:         out[small] = x[small]
0053:     return unit(out)
0054:
```

Code listing: run_upat_rise_aware_comparison.py (2)

```
0055:
0056: def householder_to_e1_apply(x, z, eps=1e-7):
0057:     e1 = np.zeros_like(x)
0058:     e1[:, 0] = 1.0
0059:     u = x - e1
0060:     denom = np.sum(u * u, axis=1, keepdims=True)
0061:     out = z.copy()
0062:
0063:     regular = denom[:, 0] > eps
0064:     if np.any(regular):
0065:         u_reg = u[regular]
0066:         z_reg = z[regular]
0067:         denom_reg = denom[regular]
0068:         out[regular] = z_reg - 2.0 * u_reg * (np.sum(u_reg * z_reg, axis=1, keepdims=True) / denom_reg)
0069:
0070:     antipodal = (~regular) & (x[:, 0] < 0)
0071:     if np.any(antipodal):
0072:         out[antipodal, 0] *= -1.0
0073:
0074:     return out
0075:
0076:
0077: def fit_procrustes(source_anchor, target_anchor, source_all, target_all):
0078:     source_mean = source_anchor.mean(axis=0, keepdims=True)
0079:     target_mean = target_anchor.mean(axis=0, keepdims=True)
0080:
0081:     source_anchor_c = source_anchor - source_mean
0082:     target_anchor_c = target_anchor - target_mean
0083:     source_all_c = source_all - source_mean
0084:     target_all_c = target_all - target_mean
0085:
0086:     min_dim = min(source_anchor_c.shape[1], target_anchor_c.shape[1], source_all_c.shape[1], target_all_c.shape[1])
0087:     source_anchor_c = source_anchor_c[:, :min_dim]
0088:     target_anchor_c = target_anchor_c[:, :min_dim]
0089:     source_all_c = source_all_c[:, :min_dim]
0090:     target_all_c = target_all_c[:, :min_dim]
0091:
0092:     u, _, vt = svd(source_anchor_c.T @ target_anchor_c, full_matrices=False, check_finite=False)
0093:     r = u @ vt
0094:     return source_all_c @ r, target_all_c
0095:
0096:
0097: def make_full_anchor_alignment(audit, train_model, test_model):
0098:     source_sp = audit.spaces[train_model]
0099:     target_sp = audit.spaces[test_model]
0100:     common_texts = sorted(set(source_sp["texts"]) & set(target_sp["texts"]))
0101:     source_idx = np.array([source_sp["idx"][text] for text in common_texts])
0102:     target_idx = np.array([target_sp["idx"][text] for text in common_texts])
0103:     return fit_procrustes(
0104:         source_anchor=source_sp["raw"][source_idx],
0105:         target_anchor=target_sp["raw"][target_idx],
0106:         source_all=source_sp["raw"],
0107:         target_all=target_sp["raw"],
0108:     )
```

Code listing: run_upat_rise_aware_comparison.py (3)

```
0109:
0110:
0111: def rows_from_vectors(audit, model_name, vectors):
0112:     sp = audit.spaces[model_name]
0113:     idx = sp["idx"]
0114:     x = np.array([vectors[idx[row.source]] for row in audit.df.itertuples(index=False)])
0115:     y = np.array([vectors[idx[row.target]] for row in audit.df.itertuples(index=False)])
0116:     return x, y
0117:
0118:
0119: def learn_mdv_raw(x_train, y_train, labels_train, class_ids):
0120:     deltas = y_train - x_train
0121:     return {
0122:         cls: deltas[labels_train == cls].mean(axis=0)
0123:         for cls in class_ids
0124:     }
0125:
0126:
0127: def predict_mdv_raw(x_test, prototypes, labels_test):
0128:     return np.vstack([x + prototypes[label] for x, label in zip(x_test, labels_test)])
0129:
0130:
0131: def learn_mdv_unit(x_train, y_train, labels_train, class_ids):
0132:     x_u = unit(x_train)
0133:     y_u = unit(y_train)
0134:     deltas = y_u - x_u
0135:     return {
0136:         cls: deltas[labels_train == cls].mean(axis=0)
0137:         for cls in class_ids
0138:     }
0139:
0140:
0141: def predict_mdv_unit(x_test, prototypes, labels_test):
0142:     x_u = unit(x_test)
0143:     return unit(np.vstack([x + prototypes[label] for x, label in zip(x_u, labels_test)]))
0144:
0145:
0146: def learn_rise_style(x_train, y_train, labels_train, class_ids):
0147:     x_u = unit(x_train)
0148:     y_u = unit(y_train)
0149:     tangent = log_map_sphere(x_u, y_u)
0150:     canonical = householder_to_e1_apply(x_u, tangent)
0151:     prototypes = {}
0152:     for cls in class_ids:
0153:         proto = canonical[labels_train == cls].mean(axis=0)
0154:         proto[0] = 0.0
0155:         prototypes[cls] = proto
0156:     return prototypes
0157:
0158:
0159: def predict_rise_style(x_test, prototypes, labels_test):
0160:     x_u = unit(x_test)
0161:     proto = np.vstack([prototypes[label] for label in labels_test])
0162:     tangent = householder_to_e1_apply(x_u, proto)
```

Code listing: run_upat_rise_aware_comparison.py (4)

```
0163:     return exp_map_sphere(x_u, tangent)
0164:
0165:
0166: def nearest_target_metrics(y_pred, y_true, labels_true):
0167:     y_pred_u = unit(y_pred)
0168:     y_true_u = unit(y_true)
0169:     sim = y_pred_u @ y_true_u.T
0170:     nearest = sim.argmax(axis=1)
0171:     pred_labels = labels_true[nearest]
0172:     return {
0173:         "retrieval_top1_acc": float(np.mean(nearest == np.arange(len(nearest)))),
0174:         "retrieval_label_acc": accuracy_score(labels_true, pred_labels),
0175:         "retrieval_label_f1": f1_score(labels_true, pred_labels, average="macro"),
0176:     }
0177:
0178:
0179: def classifier_transfer_metrics(audit, x_train, y_train, x_test, y_test, seed):
0180:     train_delta = normalize(y_train - x_train)
0181:     test_delta = normalize(y_test - x_test)
0182:     pred = audit.fit_predict(
0183:         train_delta[audit.train_mask],
0184:         audit.y[audit.train_mask],
0185:         test_delta[audit.test_mask],
0186:         seed=seed,
0187:     )
0188:     return {
0189:         "delta_classifier_acc": accuracy_score(audit.y[audit.test_mask], pred),
0190:         "delta_classifier_f1": f1_score(audit.y[audit.test_mask], pred, average="macro"),
0191:     }
0192:
0193:
0194: def evaluate_prototype_method(audit, method, train_model, test_model, train_x, train_y, test_x, test_y, seed):
0195:     train_idx = np.where(audit.train_mask)[0]
0196:     test_idx = np.where(audit.test_mask)[0]
0197:     class_ids = np.unique(audit.y)
0198:
0199:     if method == "mdv_raw":
0200:         prototypes = learn_mdv_raw(train_x[train_idx], train_y[train_idx], audit.y[train_idx], class_ids)
0201:         pred_y = predict_mdv_raw(test_x[test_idx], prototypes, audit.y[test_idx])
0202:     elif method == "mdv_unit":
0203:         prototypes = learn_mdv_unit(train_x[train_idx], train_y[train_idx], audit.y[train_idx], class_ids)
0204:         pred_y = predict_mdv_unit(test_x[test_idx], prototypes, audit.y[test_idx])
0205:     elif method == "rise_style":
0206:         prototypes = learn_rise_style(train_x[train_idx], train_y[train_idx], audit.y[train_idx], class_ids)
0207:         pred_y = predict_rise_style(test_x[test_idx], prototypes, audit.y[test_idx])
0208:     else:
0209:         raise ValueError(f"Unknown method: {method}")
0210:
0211:     y_true = test_y[test_idx]
0212:     cos = cosine_rows(pred_y, y_true)
0213:     retrieval = nearest_target_metrics(pred_y, y_true, audit.y[test_idx])
0214:
0215:     return {
0216:         "train_model": train_model,
```

Code listing: run_upat_rise_aware_comparison.py (5)

```
0217:         "test_model": test_model,
0218:         "method": method,
0219:         "mean_target_cosine": cos.mean(),
0220:         "std_target_cosine": cos.std(ddof=1),
0221:         "median_target_cosine": np.median(cos),
0222:         **retrieval,
0223:     }
0224:
0225:
0226: def run_within_model(audit, seed):
0227:     rows = []
0228:     for model_name in audit.procrustes_models:
0229:         print(f"\n\nWithin-model prototype comparison: {model_name}", flush=True)
0230:         sp = audit.spaces[model_name]
0231:         train_x = sp["x_raw"]
0232:         train_y = sp["y_raw"]
0233:         test_x = sp["x_raw"]
0234:         test_y = sp["y_raw"]
0235:         clf = classifier_transfer_metrics(audit, train_x, train_y, test_x, test_y, seed=seed)
0236:         for method in ["mdv_raw", "mdv_unit", "rise_style"]:
0237:             row = evaluate_prototype_method(
0238:                 audit, method, model_name, model_name, train_x, train_y, test_x, test_y, seed
0239:             )
0240:             row.update({"comparison": "within_model", **clf})
0241:             rows.append(row)
0242:     return rows
0243:
0244:
0245: def run_cross_model(audit, seed):
0246:     observed = pd.read_csv(CSV_DIR / "cross_model_procrustes.csv")
0247:     rows = []
0248:
0249:     for obs in observed.itertuples(index=False):
0250:         train_model = obs.train_model
0251:         test_model = obs.test_model
0252:         if train_model == test_model:
0253:             continue
0254:
0255:         print(f"\n\nCross-model prototype comparison: {train_model} -> {test_model}", flush=True)
0256:         source_aligned, target_aligned = make_full_anchor_alignment(audit, train_model, test_model)
0257:         train_x, train_y = rows_from_vectors(audit, train_model, source_aligned)
0258:         test_x, test_y = rows_from_vectors(audit, test_model, target_aligned)
0259:         clf = classifier_transfer_metrics(audit, train_x, train_y, test_x, test_y, seed=seed)
0260:
0261:         for method in ["mdv_raw", "mdv_unit", "rise_style"]:
0262:             row = evaluate_prototype_method(
0263:                 audit, method, train_model, test_model, train_x, train_y, test_x, test_y, seed
0264:             )
0265:             row.update({
0266:                 "comparison": "cross_model_full_anchor",
0267:                 "observed_raw_f1": obs.raw_f1,
0268:                 "observed_aligned_f1": obs.aligned_f1,
0269:                 **clf,
0270:             })
```


Code listing: run_upat_rise_aware_comparison.py (6)

```
0271:         rows.append(row)
0272:     return rows
0273:
0274:
0275: def summarize(raw_df):
0276:     return (
0277:         raw_df
0278:         .groupby(["comparison", "method"])
0279:         .agg(
0280:             mean_target_cosine=("mean_target_cosine", "mean"),
0281:             std_target_cosine=("mean_target_cosine", "std"),
0282:             mean_retrieval_top1_acc=("retrieval_top1_acc", "mean"),
0283:             mean_retrieval_label_acc=("retrieval_label_acc", "mean"),
0284:             mean_retrieval_label_f1=("retrieval_label_f1", "mean"),
0285:             mean_delta_classifier_f1=("delta_classifier_f1", "mean"),
0286:             n=("mean_target_cosine", "count"),
0287:         )
0288:         .reset_index()
0289:     )
0290:
0291:
0292: def plot_summary(summary_df):
0293:     for metric, ylabel, filename in [
0294:         ("mean_target_cosine", "Mean target cosine", "12_rise_aware_target_cosine.png"),
0295:         ("mean_retrieval_label_f1", "Nearest-target label macro F1", "12_rise_aware_retrieval_f1.png"),
0296:     ]:
0297:         pivot = summary_df.pivot(index="method", columns="comparison", values=metric)
0298:         ax = pivot.plot(kind="bar", figsize=(8, 5))
0299:         ax.set_ylabel(ylabel)
0300:         ax.set_title("UPAT RISE-aware prototype comparison")
0301:         ax.grid(axis="y", alpha=0.35)
0302:         plt.xticks(rotation=25, ha="right")
0303:         plt.tight_layout()
0304:         plt.savefig(FIG_DIR / filename, dpi=220, bbox_inches="tight")
0305:         plt.close()
0306:
0307:
0308: def main():
0309:     args = parse_args()
0310:     audit = UPATAudit()
0311:     audit.build_dataset()
0312:     audit.extract_all_spaces()
0313:
0314:     cross_path = CSV_DIR / "cross_model_procrustes.csv"
0315:     if not cross_path.exists():
0316:         audit.run_cross_model_procrustes()
0317:
0318:     raw_rows = []
0319:     raw_rows.extend(run_within_model(audit, seed=args.seed))
0320:     raw_rows.extend(run_cross_model(audit, seed=args.seed))
0321:
0322:     raw_df = pd.DataFrame(raw_rows)
0323:     summary_df = summarize(raw_df)
0324:     raw_df.to_csv(CSV_DIR / "rise_aware_comparison_raw.csv", index=False)
```

Code listing: run_upat_rise_aware_comparison.py (7)

```
0325:     summary_df.to_csv(CSV_DIR / "rise_aware_comparison_summary.csv", index=False)
0326:     plot_summary(summary_df)
0327:
0328:     print(f"\nSaved: {CSV_DIR / 'rise_aware_comparison_raw.csv'}")
0329:     print(f"Saved: {CSV_DIR / 'rise_aware_comparison_summary.csv'}")
0330:
0331:
0332: if __name__ == "__main__":
0333:     sys.exit(main())
```

Code listing: run_upat_hybrid_rise_procrustes.py (1)

```
0001: import argparse
0002: import sys
0003:
0004: import matplotlib.pyplot as plt
0005: import numpy as np
0006: import pandas as pd
0007: from sklearn.metrics import accuracy_score, f1_score
0008: from sklearn.preprocessing import StandardScaler, normalize
0009:
0010: from run_upat_large import UPATAudit
0011: from run_upat_rise_aware_comparison import (
0012:     CSV_DIR,
0013:     FIG_DIR,
0014:     learn_mdv_raw,
0015:     learn_mdv_unit,
0016:     learn_rise_style,
0017:     make_full_anchor_alignment,
0018:     predict_mdv_raw,
0019:     predict_mdv_unit,
0020:     predict_rise_style,
0021:     rows_from_vectors,
0022: )
0023:
0024:
0025: def parse_args():
0026:     parser = argparse.ArgumentParser(
0027:         description="Run non-leaky Hybrid RISE-Procrustes Transformation Transfer on UPAT."
0028:     )
0029:     parser.add_argument("--seed", type=int, default=4242)
0030:     return parser.parse_args()
0031:
0032:
0033: def cosine_rows(a, b):
0034:     return np.sum(normalize(a) * normalize(b), axis=1)
0035:
0036:
0037: def fit_predict_feature(audit, train_features, test_features, seed):
0038:     pred = audit.fit_predict(
0039:         train_features[audit.train_mask],
0040:         audit.y[audit.train_mask],
0041:         test_features[audit.test_mask],
0042:         seed=seed,
0043:     )
0044:     return {
0045:         "acc": accuracy_score(audit.y[audit.test_mask], pred),
0046:         "f1": f1_score(audit.y[audit.test_mask], pred, average="macro"),
0047:     }
0048:
0049:
0050: def learn_prototypes(method, x_train, y_train, labels_train, class_ids):
0051:     if method == "mdv_raw":
0052:         return learn_mdv_raw(x_train, y_train, labels_train, class_ids)
0053:     if method == "mdv_unit":
0054:         return learn_mdv_unit(x_train, y_train, labels_train, class_ids)
```

Code listing: run_upat_hybrid_rise_procrustes.py (2)

```
0055:     if method == "rise_style":
0056:         return learn_rise_style(x_train, y_train, labels_train, class_ids)
0057:     raise ValueError(f"Unknown prototype method: {method}")
0058:
0059:
0060: def predict_all_classes(method, x_all, prototypes, class_ids):
0061:     predictions = []
0062:     for cls in class_ids:
0063:         labels = np.full(x_all.shape[0], cls)
0064:         if method == "mdv_raw":
0065:             pred = predict_mdv_raw(x_all, prototypes, labels)
0066:         elif method == "mdv_unit":
0067:             pred = predict_mdv_unit(x_all, prototypes, labels)
0068:         elif method == "rise_style":
0069:             pred = predict_rise_style(x_all, prototypes, labels)
0070:         else:
0071:             raise ValueError(f"Unknown prototype method: {method}")
0072:         predictions.append(pred)
0073:     return np.stack(predictions, axis=1)
0074:
0075:
0076: def prototype_score_features(method, x_train, y_train, x_eval, y_eval, labels_train, class_ids):
0077:     """Score each pair against every class prototype without using its true label."""
0078:     prototypes = learn_prototypes(
0079:         method=method,
0080:         x_train=x_train,
0081:         y_train=y_train,
0082:         labels_train=labels_train,
0083:         class_ids=class_ids,
0084:     )
0085:     pred_all = predict_all_classes(method, x_eval, prototypes, class_ids)
0086:
0087:     true_delta = y_eval - x_eval
0088:     pred_delta = pred_all - x_eval[:, None, :]
0089:
0090:     target_cos = np.column_stack([
0091:         cosine_rows(pred_all[:, i, :], y_eval)
0092:         for i in range(len(class_ids))
0093:     ])
0094:     delta_cos = np.column_stack([
0095:         cosine_rows(pred_delta[:, i, :], true_delta)
0096:         for i in range(len(class_ids))
0097:     ])
0098:     residual_norm = np.column_stack([
0099:         np.linalg.norm(y_eval - pred_all[:, i, :], axis=1)
0100:         for i in range(len(class_ids))
0101:     ])
0102:
0103:     return np.hstack([target_cos, delta_cos, residual_norm])
0104:
0105:
0106: def standardize_from_train(audit, train_features, test_features):
0107:     scaler = StandardScaler()
0108:     scaler.fit(train_features[audit.train_mask])
```

Code listing: run_upat_hybrid_rise_procrustes.py (3)

```
0109:     return scaler.transform(train_features), scaler.transform(test_features)
0110:
0111:
0112: def build_feature_sets(audit, method, train_x, train_y, test_x, test_y):
0113:     train_idx = np.where(audit.train_mask)[0]
0114:     class_ids = np.unique(audit.y)
0115:
0116:     delta_train = normalize(train_y - train_x)
0117:     delta_test = normalize(test_y - test_x)
0118:
0119:     score_train = prototype_score_features(
0120:         method=method,
0121:         x_train=train_x[train_idx],
0122:         y_train=train_y[train_idx],
0123:         x_eval=train_x,
0124:         y_eval=train_y,
0125:         labels_train=audit.y[train_idx],
0126:         class_ids=class_ids,
0127:     )
0128:     score_test = prototype_score_features(
0129:         method=method,
0130:         x_train=train_x[train_idx],
0131:         y_train=train_y[train_idx],
0132:         x_eval=test_x,
0133:         y_eval=test_y,
0134:         labels_train=audit.y[train_idx],
0135:         class_ids=class_ids,
0136:     )
0137:     score_train, score_test = standardize_from_train(audit, score_train, score_test)
0138:
0139:     return {
0140:         "delta_only": (delta_train, delta_test),
0141:         f"{method}_prototype_scores": (score_train, score_test),
0142:         f"{method}_hybrid_delta_scores": (
0143:             normalize(np.hstack([delta_train, score_train])),
0144:             normalize(np.hstack([delta_test, score_test])),
0145:         ),
0146:     }
0147:
0148:
0149: def evaluate_feature_sets(audit, comparison, train_model, test_model, method, train_x, train_y, test_x, test_y, seed, observed
0150:     rows = []
0151:     for feature, (train_features, test_features) in build_feature_sets(
0152:         audit, method, train_x, train_y, test_x, test_y
0153:     ).items():
0154:         metrics = fit_predict_feature(audit, train_features, test_features, seed)
0155:         row = {
0156:             "comparison": comparison,
0157:             "train_model": train_model,
0158:             "test_model": test_model,
0159:             "prototype_method": method,
0160:             "feature": feature,
0161:             "acc": metrics["acc"],
0162:             "f1": metrics["f1"],
```

Code listing: run_upat_hybrid_rise_procrustes.py (4)

```
0163:         }
0164:         if observed is not None:
0165:             row.update({
0166:                 "observed_raw_f1": observed.raw_f1,
0167:                 "observed_aligned_f1": observed.aligned_f1,
0168:                 "gain_vs_observed_aligned_f1": metrics["f1"] - observed.aligned_f1,
0169:             })
0170:         rows.append(row)
0171:     return rows
0172:
0173:
0174: def run_within_model(audit, seed):
0175:     rows = []
0176:     for model_name in audit.procrustes_models:
0177:         print(f"\n\nHybrid within-model: {model_name}", flush=True)
0178:         sp = audit.spaces[model_name]
0179:         for method in ["mdv_raw", "mdv_unit", "rise_style"]:
0180:             rows.extend(evaluate_feature_sets(
0181:                 audit=audit,
0182:                 comparison="within_model",
0183:                 train_model=model_name,
0184:                 test_model=model_name,
0185:                 method=method,
0186:                 train_x=sp["x_raw"],
0187:                 train_y=sp["y_raw"],
0188:                 test_x=sp["x_raw"],
0189:                 test_y=sp["y_raw"],
0190:                 seed=seed,
0191:             ))
0192:     return rows
0193:
0194:
0195: def run_cross_model(audit, seed):
0196:     observed = pd.read_csv(CSV_DIR / "cross_model_procrustes.csv")
0197:     rows = []
0198:
0199:     for obs in observed.itertuples(index=False):
0200:         train_model = obs.train_model
0201:         test_model = obs.test_model
0202:         if train_model == test_model:
0203:             continue
0204:
0205:         print(f"\n\nHybrid cross-model: {train_model} -> {test_model}", flush=True)
0206:         source_aligned, target_aligned = make_full_anchor_alignment(audit, train_model, test_model)
0207:         train_x, train_y = rows_from_vectors(audit, train_model, source_aligned)
0208:         test_x, test_y = rows_from_vectors(audit, test_model, target_aligned)
0209:
0210:         for method in ["mdv_raw", "mdv_unit", "rise_style"]:
0211:             rows.extend(evaluate_feature_sets(
0212:                 audit=audit,
0213:                 comparison="cross_model_full_anchor",
0214:                 train_model=train_model,
0215:                 test_model=test_model,
0216:                 method=method,
```

Code listing: run_upat_hybrid_rise_procrustes.py (5)

```
0217:         train_x=train_x,
0218:         train_y=train_y,
0219:         test_x=test_x,
0220:         test_y=test_y,
0221:         seed=seed,
0222:         observed=obs,
0223:     ))
0224:     return rows
0225:
0226:
0227: def summarize(raw_df):
0228:     return (
0229:         raw_df
0230:         .groupby(["comparison", "prototype_method", "feature"])
0231:         .agg(
0232:             mean_acc=("acc", "mean"),
0233:             mean_f1=("f1", "mean"),
0234:             std_f1=("f1", "std"),
0235:             mean_gain_vs_observed_aligned_f1=("gain_vs_observed_aligned_f1", "mean"),
0236:             n=("f1", "count"),
0237:         )
0238:         .reset_index()
0239:         .sort_values(["comparison", "mean_f1"], ascending=[True, False])
0240:     )
0241:
0242:
0243: def plot_summary(summary_df):
0244:     cross = summary_df[summary_df["comparison"] == "cross_model_full_anchor"].copy()
0245:     cross["label"] = cross["prototype_method"] + "\\n" + cross["feature"].str.replace("_", " ")
0246:     cross = cross.sort_values("mean_f1", ascending=False)
0247:
0248:     plt.figure(figsize=(max(12, len(cross) * 0.55), 6))
0249:     x = np.arange(len(cross))
0250:     plt.bar(x, cross["mean_f1"])
0251:     delta_mean = cross[cross["feature"] == "delta_only"]["mean_f1"].iloc[0]
0252:     plt.axhline(delta_mean, linestyle="--", label="delta_only")
0253:     plt.xticks(x, cross["label"], rotation=70, ha="right", fontsize=7)
0254:     plt.ylabel("Cross-model transformation label macro F1")
0255:     plt.title("Hybrid RISE-Procrustes Transformation Transfer")
0256:     plt.grid(axis="y", alpha=0.35)
0257:     plt.legend()
0258:     plt.tight_layout()
0259:     plt.savefig(FIG_DIR / "13_hybrid_rise_procrustes_f1.png", dpi=220, bbox_inches="tight")
0260:     plt.close()
0261:
0262:     heat = cross.pivot_table(index="prototype_method", columns="feature", values="mean_f1")
0263:     plt.figure(figsize=(10, 4))
0264:     plt.imshow(heat.values, aspect="auto")
0265:     plt.colorbar(label="Macro F1")
0266:     plt.xticks(np.arange(len(heat.columns)), heat.columns, rotation=35, ha="right", fontsize=8)
0267:     plt.yticks(np.arange(len(heat.index)), heat.index)
0268:     plt.title("Hybrid feature comparison")
0269:     for i in range(heat.shape[0]):
0270:         for j in range(heat.shape[1]):
```

Code listing: run_upat_hybrid_rise_procrustes.py (6)

```
0271:         plt.text(j, i, f"{heat.values[i, j]:.3f}", ha="center", va="center", fontsize=8)
0272:     plt.tight_layout()
0273:     plt.savefig(FIG_DIR / "13_hybrid_rise_procrustes_heatmap.png", dpi=220, bbox_inches="tight")
0274:     plt.close()
0275:
0276:
0277: def main():
0278:     args = parse_args()
0279:     audit = UPATAudit()
0280:     audit.build_dataset()
0281:     audit.extract_all_spaces()
0282:
0283:     cross_path = CSV_DIR / "cross_model_procrustes.csv"
0284:     if not cross_path.exists():
0285:         audit.run_cross_model_procrustes()
0286:
0287:     rows = []
0288:     rows.extend(run_within_model(audit, seed=args.seed))
0289:     rows.extend(run_cross_model(audit, seed=args.seed))
0290:
0291:     raw_df = pd.DataFrame(rows)
0292:     summary_df = summarize(raw_df)
0293:     raw_df.to_csv(CSV_DIR / "hybrid_rise_procrustes_raw.csv", index=False)
0294:     summary_df.to_csv(CSV_DIR / "hybrid_rise_procrustes_summary.csv", index=False)
0295:     plot_summary(summary_df)
0296:
0297:     print(f"\nSaved: {CSV_DIR / 'hybrid_rise_procrustes_raw.csv'}")
0298:     print(f"Saved: {CSV_DIR / 'hybrid_rise_procrustes_summary.csv'}")
0299:
0300:
0301: if __name__ == "__main__":
0302:     sys.exit(main())
```


Code listing: run_upat_spherical_delta_steering.py (1)

```
0001: import argparse
0002: import sys
0003:
0004: import matplotlib.pyplot as plt
0005: import numpy as np
0006: import pandas as pd
0007: from sklearn.metrics import accuracy_score, f1_score
0008:
0009: from run_upat_large import UPATAudit
0010: from run_upat_rise_aware_comparison import (
0011:     CSV_DIR,
0012:     FIG_DIR,
0013:     cosine_rows,
0014:     exp_map_sphere,
0015:     learn_rise_style,
0016:     make_full_anchor_alignment,
0017:     nearest_target_metrics,
0018:     predict_rise_style,
0019:     rows_from_vectors,
0020:     unit,
0021: )
0022:
0023:
0024: def parse_args():
0025:     parser = argparse.ArgumentParser(
0026:         description="Run movement-level spherical delta steering tests on UPAT."
0027:     )
0028:     parser.add_argument("--seed", type=int, default=777)
0029:     return parser.parse_args()
0030:
0031:
0032: def tangent_project(x_unit, direction):
0033:     return direction - np.sum(direction * x_unit, axis=1, keepdims=True) * x_unit
0034:
0035:
0036: def learn_unit_delta_centroids(x_train, y_train, labels_train, class_ids):
0037:     deltas = unit(y_train) - unit(x_train)
0038:     return {
0039:         cls: deltas[labels_train == cls].mean(axis=0)
0040:         for cls in class_ids
0041:     }
0042:
0043:
0044: def predict_linear_delta(x_eval, centroids, labels_eval):
0045:     x_u = unit(x_eval)
0046:     delta = np.vstack([centroids[label] for label in labels_eval])
0047:     return unit(x_u + delta)
0048:
0049:
0050: def predict_spherical_delta(x_eval, centroids, labels_eval):
0051:     x_u = unit(x_eval)
0052:     delta = np.vstack([centroids[label] for label in labels_eval])
0053:     tangent = tangent_project(x_u, delta)
0054:     return exp_map_sphere(x_u, tangent)
```

Code listing: run_upat_spherical_delta_steering.py (2)

```
0055:
0056:
0057: def learn_residual_centroids(base_pred_train, y_train, labels_train, class_ids):
0058:     base_u = unit(base_pred_train)
0059:     residual = unit(y_train) - base_u
0060:     return {
0061:         cls: residual[labels_train == cls].mean(axis=0)
0062:         for cls in class_ids
0063:     }
0064:
0065:
0066: def apply_residual_spherical(base_pred, residual_centroids, labels_eval):
0067:     base_u = unit(base_pred)
0068:     residual = np.vstack([residual_centroids[label] for label in labels_eval])
0069:     tangent = tangent_project(base_u, residual)
0070:     return exp_map_sphere(base_u, tangent)
0071:
0072:
0073: def evaluate_prediction(method, comparison, train_model, test_model, y_pred, y_true, labels_true):
0074:     cos = cosine_rows(y_pred, y_true)
0075:     retrieval = nearest_target_metrics(y_pred, y_true, labels_true)
0076:     return {
0077:         "comparison": comparison,
0078:         "train_model": train_model,
0079:         "test_model": test_model,
0080:         "method": method,
0081:         "mean_target_cosine": float(cos.mean()),
0082:         "std_target_cosine": float(cos.std(ddof=1)),
0083:         "median_target_cosine": float(np.median(cos)),
0084:         **retrieval,
0085:     }
0086:
0087:
0088: def evaluate_direction(audit, comparison, train_model, test_model, train_x, train_y, test_x, test_y):
0089:     train_idx = np.where(audit.train_mask)[0]
0090:     test_idx = np.where(audit.test_mask)[0]
0091:     class_ids = np.unique(audit.y)
0092:
0093:     labels_train = audit.y[train_idx]
0094:     labels_test = audit.y[test_idx]
0095:
0096:     train_x_fit = train_x[train_idx]
0097:     train_y_fit = train_y[train_idx]
0098:     test_x_eval = test_x[test_idx]
0099:     test_y_eval = test_y[test_idx]
0100:
0101:     delta_centroids = learn_unit_delta_centroids(train_x_fit, train_y_fit, labels_train, class_ids)
0102:     rise_prototypes = learn_rise_style(train_x_fit, train_y_fit, labels_train, class_ids)
0103:
0104:     train_linear = predict_linear_delta(train_x_fit, delta_centroids, labels_train)
0105:     train_spherical = predict_spherical_delta(train_x_fit, delta_centroids, labels_train)
0106:     train_rise = predict_rise_style(train_x_fit, rise_prototypes, labels_train)
0107:
0108:     linear_pred = predict_linear_delta(test_x_eval, delta_centroids, labels_test)
```

Code listing: run_upat_spherical_delta_steering.py (3)

```
0109:     spherical_pred = predict_spherical_delta(test_x_eval, delta_centroids, labels_test)
0110:     rise_pred = predict_rise_style(test_x_eval, rise_prototypes, labels_test)
0111:
0112:     rise_residual = learn_residual_centroids(train_rise, train_y_fit, labels_train, class_ids)
0113:     linear_residual = learn_residual_centroids(train_linear, train_y_fit, labels_train, class_ids)
0114:     spherical_residual = learn_residual_centroids(train_spherical, train_y_fit, labels_train, class_ids)
0115:
0116:     rise_then_ours = apply_residual_spherical(rise_pred, rise_residual, labels_test)
0117:     linear_then_rise_residual = apply_residual_spherical(linear_pred, linear_residual, labels_test)
0118:     spherical_then_rise_residual = apply_residual_spherical(spherical_pred, spherical_residual, labels_test)
0119:     hybrid_average = unit((spherical_pred + rise_pred) / 2.0)
0120:
0121:     predictions = {
0122:         "linear_delta": linear_pred,
0123:         "spherical_delta": spherical_pred,
0124:         "rise_only": rise_pred,
0125:         "rise_then_ours_residual": rise_then_ours,
0126:         "linear_then_residual": linear_then_rise_residual,
0127:         "spherical_then_residual": spherical_then_rise_residual,
0128:         "hybrid_average": hybrid_average,
0129:     }
0130:
0131:     return [
0132:         evaluate_prediction(
0133:             method=method,
0134:             comparison=comparison,
0135:             train_model=train_model,
0136:             test_model=test_model,
0137:             y_pred=pred,
0138:             y_true=test_y_eval,
0139:             labels_true=labels_test,
0140:         )
0141:         for method, pred in predictions.items()
0142:     ]
0143:
0144:
0145: def run_within_model(audit):
0146:     rows = []
0147:     for model_name in audit.procrustes_models:
0148:         print(f"\nSpherical steering within-model: {model_name}", flush=True)
0149:         sp = audit.spaces[model_name]
0150:         rows.extend(evaluate_direction(
0151:             audit=audit,
0152:             comparison="within_model",
0153:             train_model=model_name,
0154:             test_model=model_name,
0155:             train_x=sp["x_raw"],
0156:             train_y=sp["y_raw"],
0157:             test_x=sp["x_raw"],
0158:             test_y=sp["y_raw"],
0159:         ))
0160:     return rows
0161:
0162:
```

Code listing: run_upat_spherical_delta_steering.py (4)

```
0163: def run_cross_model(audit):
0164:     observed = pd.read_csv(CSV_DIR / "cross_model_procrustes.csv")
0165:     rows = []
0166:
0167:     for obs in observed.itertuples(index=False):
0168:         train_model = obs.train_model
0169:         test_model = obs.test_model
0170:         if train_model == test_model:
0171:             continue
0172:
0173:         print(f"\nSpherical steering cross-model: {train_model} -> {test_model}", flush=True)
0174:         source_aligned, target_aligned = make_full_anchor_alignment(audit, train_model, test_model)
0175:         train_x, train_y = rows_from_vectors(audit, train_model, source_aligned)
0176:         test_x, test_y = rows_from_vectors(audit, test_model, target_aligned)
0177:         direction_rows = evaluate_direction(
0178:             audit=audit,
0179:             comparison="cross_model_full_anchor",
0180:             train_model=train_model,
0181:             test_model=test_model,
0182:             train_x=train_x,
0183:             train_y=train_y,
0184:             test_x=test_x,
0185:             test_y=test_y,
0186:         )
0187:         for row in direction_rows:
0188:             row.update({
0189:                 "observed_raw_f1": obs.raw_f1,
0190:                 "observed_aligned_f1": obs.aligned_f1,
0191:             })
0192:         rows.extend(direction_rows)
0193:     return rows
0194:
0195:
0196: def summarize(raw_df):
0197:     return (
0198:         raw_df
0199:         .groupby(["comparison", "method"])
0200:         .agg(
0201:             mean_target_cosine=("mean_target_cosine", "mean"),
0202:             std_target_cosine=("mean_target_cosine", "std"),
0203:             mean_retrieval_top1_acc=("retrieval_top1_acc", "mean"),
0204:             mean_retrieval_label_acc=("retrieval_label_acc", "mean"),
0205:             mean_retrieval_label_f1=("retrieval_label_f1", "mean"),
0206:             n=("mean_target_cosine", "count"),
0207:         )
0208:         .reset_index()
0209:         .sort_values(["comparison", "mean_target_cosine"], ascending=[True, False])
0210:     )
0211:
0212:
0213: def plot_summary(summary_df):
0214:     for metric, ylabel, filename in [
0215:         ("mean_target_cosine", "Mean target cosine", "14_spherical_delta_target_cosine.png"),
0216:         ("mean_retrieval_top1_acc", "Nearest-target top-1 accuracy", "14_spherical_delta_retrieval_top1.png"),
```

Code listing: run_upat_spherical_delta_steering.py (5)

```
0217:         ("mean_retrieval_label_f1", "Nearest-target label macro F1", "14_spherical_delta_retrieval_label_f1.png"),
0218:     ]:
0219:         pivot = summary_df.pivot(index="method", columns="comparison", values=metric)
0220:         ax = pivot.plot(kind="bar", figsize=(10, 5))
0221:         ax.set_ylabel(ylabel)
0222:         ax.set_title("UPAT Spherical Delta Steering")
0223:         ax.grid(axis="y", alpha=0.35)
0224:         plt.xticks(rotation=30, ha="right")
0225:         plt.tight_layout()
0226:         plt.savefig(FIG_DIR / filename, dpi=220, bbox_inches="tight")
0227:         plt.close()
0228:
0229:
0230: def main():
0231:     parse_args()
0232:     audit = UPATAudit()
0233:     audit.build_dataset()
0234:     audit.extract_all_spaces()
0235:
0236:     cross_path = CSV_DIR / "cross_model_procrustes.csv"
0237:     if not cross_path.exists():
0238:         audit.run_cross_model_procrustes()
0239:
0240:     rows = []
0241:     rows.extend(run_within_model(audit))
0242:     rows.extend(run_cross_model(audit))
0243:
0244:     raw_df = pd.DataFrame(rows)
0245:     summary_df = summarize(raw_df)
0246:
0247:     raw_df.to_csv(CSV_DIR / "spherical_delta_steering_raw.csv", index=False)
0248:     summary_df.to_csv(CSV_DIR / "spherical_delta_steering_summary.csv", index=False)
0249:     plot_summary(summary_df)
0250:
0251:     print(f"\nSaved: {CSV_DIR / 'spherical_delta_steering_raw.csv'}")
0252:     print(f"Saved: {CSV_DIR / 'spherical_delta_steering_summary.csv'}")
0253:
0254:
0255: if __name__ == "__main__":
0256:     sys.exit(main())
```

Code listing: analyze_confusion_negation.py (1)

```
0001: from pathlib import Path
0002:
0003: import pandas as pd
0004:
0005:
0006: ROOT = Path(__file__).resolve().parents[1]
0007: EXP = ROOT / "results" / "experiments"
0008: OUT = ROOT / "results"
0009: OUT.mkdir(exist_ok=True)
0010:
0011:
0012: def parse_name(path):
0013:     stem = path.stem.replace("confusion_", "")
0014:     if stem.endswith("_linear_svc"):
0015:         return stem[:-11].replace("__", "/"), "linear_svc"
0016:     if stem.endswith("_logreg"):
0017:         return stem[:-7].replace("__", "/"), "logreg"
0018:     raise ValueError(path.name)
0019:
0020:
0021: def load_confusion(path):
0022:     model, classifier = parse_name(path)
0023:     cm = pd.read_csv(path, index_col=0)
0024:     labels = list(cm.index)
0025:
0026:     rows = []
0027:     errors = []
0028:
0029:     for label in labels:
0030:         total = float(cm.loc[label].sum())
0031:         correct = float(cm.loc[label, label])
0032:         recall = correct / total if total else 0.0
0033:         rows.append({
0034:             "source": "full_semantic",
0035:             "model": model,
0036:             "classifier": classifier,
0037:             "class": label,
0038:             "support": int(total),
0039:             "correct": int(correct),
0040:             "recall": recall,
0041:             "error_rate": 1.0 - recall,
0042:             "is_negation": label == "negation",
0043:         })
0044:
0045:     for pred in labels:
0046:         if pred == label:
0047:             continue
0048:         count = int(cm.loc[label, pred])
0049:         if count:
0050:             errors.append({
0051:                 "source": "full_semantic",
0052:                 "model": model,
0053:                 "classifier": classifier,
0054:                 "true_class": label,
```

Code listing: analyze_confusion_negation.py (2)

```
0055:         "predicted_class": pred,
0056:         "count": count,
0057:         "true_support": int(total),
0058:         "share_of_true": count / total if total else 0.0,
0059:         "involves_negation": label == "negation" or pred == "negation",
0060:     })
0061:
0062:     return rows, errors
0063:
0064:
0065: def main():
0066:     paths = sorted((EXP / "lie_llm_full_semantic_holdout_results").glob("confusion_*.csv"))
0067:     all_rows = []
0068:     all_errors = []
0069:
0070:     for path in paths:
0071:         rows, errors = load_confusion(path)
0072:         all_rows.extend(rows)
0073:         all_errors.extend(errors)
0074:
0075:     class_df = pd.DataFrame(all_rows)
0076:     error_df = pd.DataFrame(all_errors)
0077:
0078:     class_df.to_csv(OUT / "confusion_class_recall.csv", index=False)
0079:     error_df.sort_values(["count", "share_of_true"], ascending=False).to_csv(
0080:         OUT / "confusion_top_errors.csv",
0081:         index=False,
0082:     )
0083:
0084:     neg_summary = (
0085:         class_df
0086:         .assign(group=lambda d: d["is_negation"].map({True: "negation", False: "non_negation"}))
0087:         .groupby(["source", "model", "classifier", "group"])
0088:         .agg(
0089:             mean_recall=("recall", "mean"),
0090:             mean_error_rate=("error_rate", "mean"),
0091:             n_classes=("class", "count"),
0092:         )
0093:         .reset_index()
0094:     )
0095:     neg_summary.to_csv(OUT / "confusion_negation_summary.csv", index=False)
0096:
0097:     rank_df = class_df.copy()
0098:     rank_df["recall_rank_low_is_hard"] = rank_df.groupby(["model", "classifier"])["recall"].rank(method="min")
0099:     rank_df.sort_values(["model", "classifier", "recall"]).to_csv(
0100:         OUT / "confusion_class_recall_ranked.csv",
0101:         index=False,
0102:     )
0103:
0104:     print("Saved:")
0105:     for name in [
0106:         "confusion_class_recall.csv",
0107:         "confusion_top_errors.csv",
0108:         "confusion_negation_summary.csv",
```

Code listing: analyze_confusion_negation.py (3)

```
0109:         "confusion_class_recall_ranked.csv",
0110:     ]:
0111:         print(OUT / name)
0112:
0113:
0114: if __name__ == "__main__":
0115:     main()
```


Code listing: build_signed_permutation_multiple_testing.py (1)

```
0001: from pathlib import Path
0002:
0003: import numpy as np
0004: import pandas as pd
0005:
0006:
0007: ROOT = Path(__file__).resolve().parents[1]
0008: EXP = ROOT / "results" / "experiments"
0009: OUT = EXP / "lie_algebraic_identities_results" / "csv"
0010:
0011:
0012: def benjamini_hochberg(p_values):
0013:     p = np.asarray(p_values, dtype=float)
0014:     n = len(p)
0015:     order = np.argsort(p)
0016:     ranked = p[order]
0017:     adj = np.empty(n, dtype=float)
0018:     prev = 1.0
0019:     for i in range(n - 1, -1, -1):
0020:         rank = i + 1
0021:         val = min(prev, ranked[i] * n / rank)
0022:         prev = val
0023:         adj[order[i]] = val
0024:     return np.clip(adj, 0, 1)
0025:
0026:
0027: def bootstrap_p_less_than_one(values, n_boot=20000, seed=42):
0028:     rng = np.random.default_rng(seed)
0029:     values = np.asarray(values, dtype=float)
0030:     idx = rng.integers(0, len(values), size=(n_boot, len(values)))
0031:     means = values[idx].mean(axis=1)
0032:     p = (np.sum(means >= 1.0) + 1) / (n_boot + 1)
0033:     return float(p)
0034:
0035:
0036: def main():
0037:     files = [
0038:         EXP / "lie_algebraic_identities_results" / "csv" / "jacobi_raw_all_models.csv",
0039:         EXP / "lie_algebraic_identities_decoder_results" / "csv" / "jacobi_raw_all_models.csv",
0040:     ]
0041:     raw = pd.concat([pd.read_csv(path) for path in files if path.exists()], ignore_index=True)
0042:
0043:     rows = []
0044:     for (model, triple), group in raw.groupby(["model", "triple"]):
0045:         ratios = group["jacobi_to_null_mean_ratio"].to_numpy()
0046:         mean_ratio = float(ratios.mean())
0047:         p_less = bootstrap_p_less_than_one(ratios)
0048:         rows.append({
0049:             "model": model,
0050:             "triple": triple,
0051:             "n": int(len(group)),
0052:             "mean_ratio_to_null": mean_ratio,
0053:             "bootstrap_p_ratio_less_than_1": p_less,
0054:             "direction": "below_null" if mean_ratio < 1 else "above_null",
```

Code listing: build_signed_permutation_multiple_testing.py (2)

```
0055:         })
0056:
0057:     result = pd.DataFrame(rows).sort_values(["bootstrap_p_ratio_less_than_1", "model", "triple"])
0058:     result["bonferroni_p"] = np.minimum(result["bootstrap_p_ratio_less_than_1"] * len(result), 1.0)
0059:     result["bh_fdr_p"] = benjamini_hochberg(result["bootstrap_p_ratio_less_than_1"].to_numpy())
0060:     result["passes_bonferroni_0_05"] = result["bonferroni_p"] < 0.05
0061:     result["passes_bh_fdr_0_05"] = result["bh_fdr_p"] < 0.05
0062:
0063:     out_path = OUT / "signed_permutation_multiple_testing.csv"
0064:     result.to_csv(out_path, index=False)
0065:     print(out_path)
0066:     print(result)
0067:
0068:
0069: if __name__ == "__main__":
0070:     main()
```